User Guide

Linux APSS For Ensemble Fusion Processors

Version 2.1.0

1. Alif DevKit Basics and Minimal Linux Setup

This chapter introduces the Alif DevKit and the basics of running a minimal Linux configuration on it. Whether you're exploring embedded systems for the first time or aiming to harness the Alif Ensemble's efficiency, this guide offers a clear starting point for building and deploying a lightweight Linux environment.

1.1 The Alif DevKit: A Platform for Minimal Linux

The Alif DevKit is a development platform centred on the Alif Ensemble, a multi-core embedded device engineered for high performance and low power consumption. This single-chip solution integrates Cortex-M and Cortex-A processors, making it versatile for embedded applications. At its heart, the Cortex-A32 processor—a 32-bit ARMv8-A core—delivers exceptional efficiency for computing tasks. The Application Processor Subsystem (APSS), powered by one or more Cortex-A32 cores, supports advanced operating systems like Linux, enabling a streamlined, minimal configuration ideal for resource-constrained environments.

Key features of the APSS in the DevKit Ensemble include:

- CPU core(s) running at 800 MHz
- 32 KB L1 instruction cache and 32 KB L1 data cache per CPU core.
- 512 KB total L2 cache.
- 8 MB Stitched SRAM (Static Random-Access Memory).
- 6 MB MRAM (Magneto resistive Random-Access Memory).
- Advanced SIMD and floating-point capabilities.
- GIC-400 (Generic Interrupt Controller) support.
- Full support for execution states (Exception Levels EL0 to EL3).
- Performance Monitoring Unit (PMU).
- Embedded Trace Macrocell (ETM).
- These capabilities make the Alif DevKit a powerful yet efficient platform for running a minimal Linux system, optimized for tasks where performance and power efficiency are critical.

1.1.1 Minimal Linux Configuration Explained

A minimal Linux configuration is a lightweight operating system tailored for embedded devices like the Alif DevKit. Unlike full-featured Linux distributions, it includes only essential components—such as a

compact kernel, a basic filesystem, and minimal utilities—to reduce memory footprint and boot time. On the Alif DevKit, this means leveraging Execute-in-Place (XiP) techniques to run the kernel and applications directly from flash memory, conserving RAM while maintaining functionality. This guide uses a tiny Linux distribution with *BusyBox*—a single executable providing core Unix utilities—as an example of such a setup.

1.1.2 Purpose of This Guide

This guide will help you set up and run a minimal embedded Linux application on the Alif Ensemble device. We will start with a sample application that continuously prints "Linux Hello World" to the console. This will demonstrate the basics of booting and running code. You can customize or replace this with any embedded application to meet your project's needs.

Here's what we'll cover:

- Prerequisites for building Linux images for the APSS on the DevKit.
- An overview of the APSS Linux Project, which manages the minimal Linux build process.
- Essential software development tools for the APSS Linux Project.
- A schematic build flow for creating Linux images.
- Steps to build Linux images on a Linux host (e.g., Ubuntu) or a Linux distribution like Ubuntu on Windows 11.
- How to program Linux images into MRAM flash and boot the APSS with a lightweight Linux distribution featuring *BusyBox.*
- By the end, you'll have a working minimal Linux setup on the Alif DevKit and the knowledge to adapt it for your own applications.

1.2 Overview of the APSS Linux Project

The APSS Linux Project uses the Yocto Project—an open-source framework for building embedded Linux distributions—to create Linux images for the Application Processor Subsystem (APSS) on the Ensemble Development Kit. It leverages the OpenEmbedded build system and the BitBake tool. OpenEmbedded provides a build automation framework and cross-compilation environment for embedded Linux distributions. BitBake, a task execution engine, runs shell and Python tasks efficiently in parallel,

managing complex dependencies. Like GNU Make with makefiles, BitBake uses recipes to control the build process.

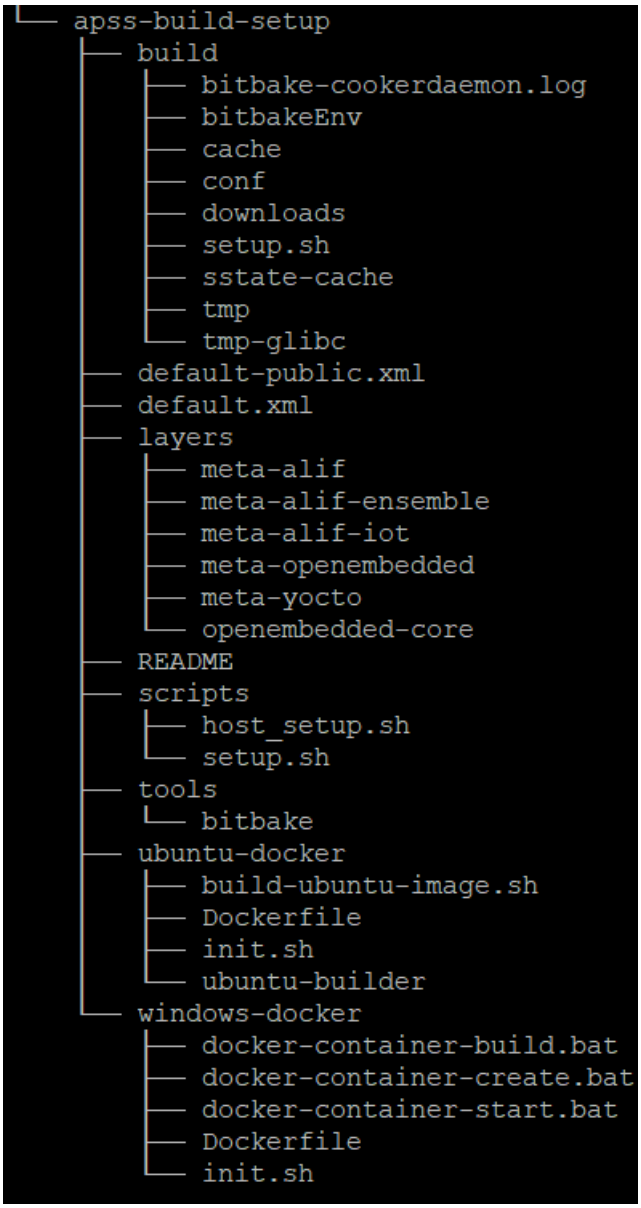


Figure 1 Directory Structure of the APSS Linux Project

Here's a breakdown of the key components in the APSS Linux Project directory:

Entry	Description
build	Stores the Linux kernel, user-space packages, and root filesystem (rootfs) during compilation. Created by the setup.sh script, which sets up the environment for BitBake.
default.xml	A manifest file for the repo tool, used to manage multiple Git repositories.
layers	Holds Yocto layers, including open-source and custom layers like meta-alif, meta-alif-ensemble, and meta-alif-iot. Each layer contains recipes—procedures to build packages from source code.
meta-alif	Includes recipes to build an embedded Linux application (recipes-core/images/alif-helloworld-image.bb) and a tiny Linux distribution (recipes-core/images/alif-tiny-image.bb). Uses a read-only XiP cramfs filesystem. Cramfs is a simple, compact, compressed filesystem. XiP (eXecute-in-Place) allows files to run directly from NOR flash or ROM, mapping them to physical memory via the MMU instead of loading them into RAM, saving memory.
meta-alif-ensemble	Contains the machine configuration file (conf/machine/devkit-e8.conf), Linux kernel recipe, and APSS-specific settings. Defines architecture, bootloader boot address, Linux device tree (DTB) load address, kernel image (xipImage) boot address, rootfs partition details, Secure RAM (SRAM0), and more.
meta-alif-iot	Provides recipes for IoT-specific libraries and applications.
meta-openembedded	A collection of sub-layers offering additional user-space tools and utilities beyond the openembedded-core layer.
meta-yocto	Includes sub-layers with generic machine configurations and settings to minimize rootfs image size.
openembedded-core	Contains core metadata (Python classes, modules) and configuration files essential to the Yocto Project. Manages: • Package and recipe dependencies across layers. • Task scheduling (e.g., do_fetch, do_unpack, do_patch, do_configure, do_compile) • Binary package generation for Linux and rootfs images. Also includes recipes for a basic embedded Linux distribution.
scripts	Contains a bash script (setup.sh) to configure the environment for building images with BitBake.
tools	Includes the BitBake tool.
ubuntu-docker	Holds scripts to fetch and run an Ubuntu 22.04/later Docker image on a Linux host within a Docker container.
windows-docker	Holds scripts to fetch and run an Ubuntu 22.04/later Docker image on a Windows 11 host within a Docker container.

2. Prerequisites

2.1 Host Requirements

2.1.1 Hardware Requirements

To get started, ensure your setup includes:

- A personal computer (PC) with an x86-64 processor, at least 8 GB of RAM, and a minimum of 50 GB of free disk space.
- An Alif Ensemble EAGLE Development Kit with a PCB revision of A0 or later.

2.1.2 Software Requirements

Your PC must run one of these operating systems:

- Windows 10 or Windows 11.
- A 64-bit Linux distribution, such as Ubuntu 22.04/later.

You'll also need:

- Alif Security Toolkit version 1.107 (public SETOOLS release).

3. Setup

3.1 Hardware and Software Setup

3.1.1 Board Setup

Set up your Beta Development Kit by following the steps in the *Ensemble EX Development Kit Quick Start Guide*.

- Connect a USB cable to the **SE-UART port** to power the board and establish a serial console.
- Confirm the pre-flashed "blinky" firmware executes, indicated by a flashing LED. This validates successful hardware bring-up.

3.1.2 Install the Alif Security Toolkit (SETOOLS)

Install the latest version of the Alif Security Toolkit (SETOOLS) as outlined in the *Alif Security Toolkit Quick Start Guide*. This guide assumes you've installed the tools to /home/$USER/app-release.

3.1.3 Connect the USB PRG Cable for Serial Consoles

The USB PRG J3 port on the Devkit provides two serial ports:

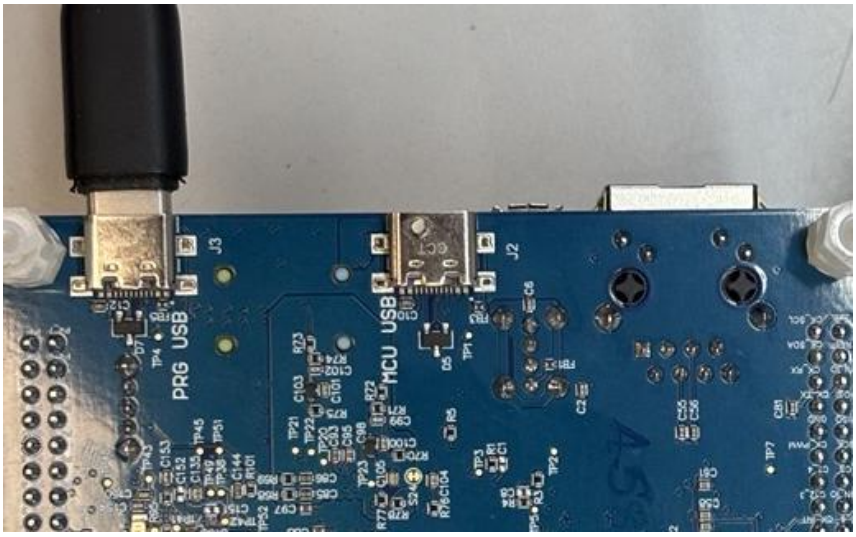


Figure 2 – USB PRG Cable.

- Please refer to the image below, Figure 3, which shows the SEUART “SE” setting in Switch SW4.

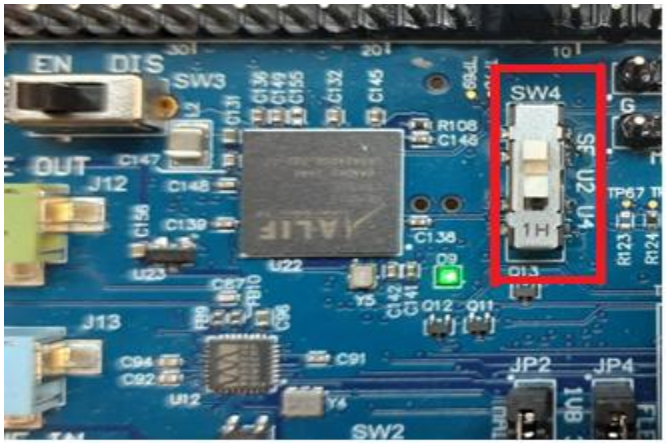


Figure 3 – SEUART SE setting in Switch.

- UART2 or UART4 port (selectable via SW4 Switch settings).

To configure UART2 or UART4, adjust the SW4 Switch:

- For UART2, please refer to SW4 labeled “U2,” as indicated by the yellow line in the figure below (See Figure 4).

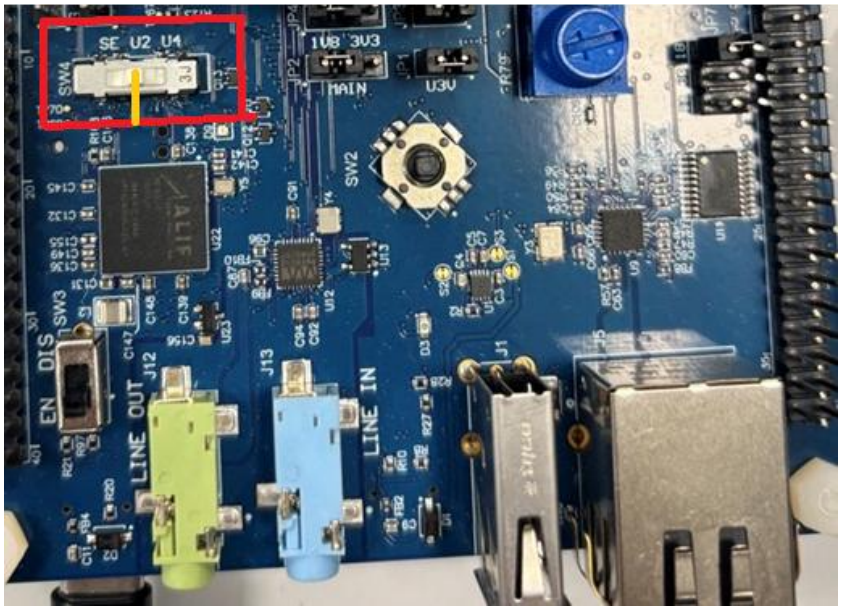


Figure 4 – UART2 setting in SW4 “U2”.

- For UART4, refer to SW4 labeled “U4,” which is indicated by the yellow line in Figure 5 below.

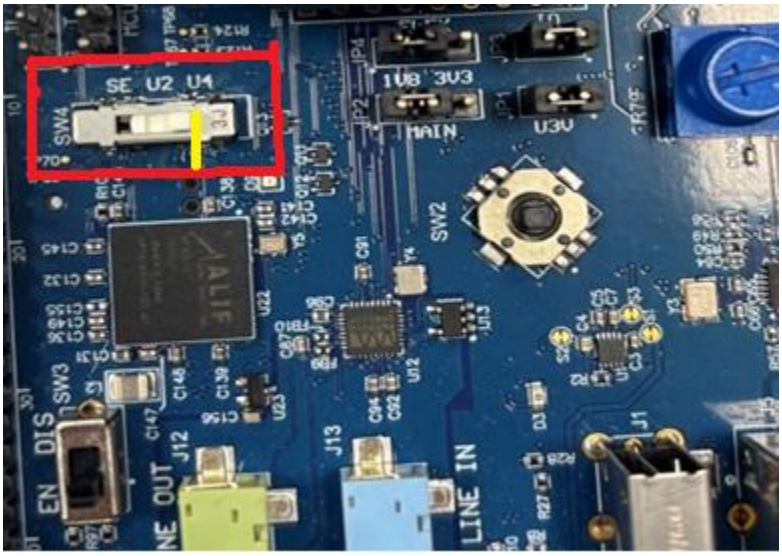


Figure 5 – UART4 “U4” setting in SW4.

Notes:

- UART2 is the serial console for the APSS in this guide. All APSS boot messages appear on UART2.
- UART4 is reserved for debugging user-space applications with GDBserver over a serial connection.

3.1.4 Download the Linux Beta Distribution v2.1.0 Files

1. Save the Linux v2.1.0 distribution ZIP file to */home/$USER*.
2. Extract it to a directory of your choice (default: *./APSS-v2.1.0*).

3.1.5 Extract the Release and Copy Pre-Built Files

1. **Extract the utilities:**

   Extract */home/$USER/APSS-v2.1.0/Utilities/Alif-Application-tools/app-release-exec-linux-SE_FW_1.106_DEV.tar* to */home/$USER* using this command:

```
$ tar xf /home/$USER/APSS-v2.1.0/Utilities/Alif-Application-tools/app-release-exec-linux-SE_FW_1.106_DEV.tar -C /home/$USER
```

3. **Copy pre-built binary images:**

   From */home/$USER/APSS-v2.1.0-Beta/prebuilt-images* to */home/$USER/app-release-exec-linux/build/images*:

   - alif-tiny-image-devkit-e8.cramfs-xip

- alif-tiny-image-devkit-e8-ospi1-en.cramfs-xip
- bl32.bin
- devkit-e8.dtb
- PC_Tool_HE.bin
- xipImage

4. **Copy configuration files:**

   From */home/$USER/APSS-v2.1.0-Beta/config-files* to */home/$USER/app-release-exec-linux/build/config*:

   - TINY_IMAGE.json
   - APSS_BASE_IMAGE.json
   - APSS_BASE_IMAGE2.json
   - APSS_BASE_IMAGE3.json

4. Programming Linux Images

4.1 Load Pre-Built Images for MRAM Flash Distribution

4.1.1 Program Linux Images into MRAM Flash

The Alif Security Toolkit includes a Python-based tool to combine Linux images and binaries from multiple cores into a single application image. This image is programmed into MRAM (non-volatile memory) and boots on the respective cores after a soft reset.

Important: Ensure the MRAM addresses used for programming match those specified when creating the images. For example:

- If xipImage is set to boot from 0x80020000, program it to 0x80020000.
- The same applies to the cramfs-xip root filesystem image.

After powering on, the images execute directly from MRAM.

4.1.1.1 Supported IMAGES

IMAGE	MRAM	STITCHED SRAM = 8MB	HYPERRAM = 64MB	OSPI FLASH	SD/MMC – (RFS)	Devices/IP	Size xipImage + RFS in MB
TINY_IMAGE	bl32.bin, devkit-e8, xipImage and cramfs.	Yes	No	No	No	UART, RTC, GPIO, MHU, HWSEM	2.3 + 1.8
BASE_IMAGE	bl32.bin, devkit-e8, xipImage and cramfs.	Yes	No	No	No	TINY_IMAGE + SD, USB, SPI, I2C	3.0 + 1.8
BASE_IMAGE2	bl32.bin, devkit-e8, xipImage.	No	Yes	Cramfs	No	BASE_IMAGE + Ethernet, PDM, Utimer, I3C, I2S, DMA+I2S, DAC12	3.9 + 2.9
BASE_IMAGE3	bl32.bin, devkit-e8.	Yes	Yes	xipImage and cramfs	No	BASE_IMAGE2	3.9 + 2.9
BASE_IMAGE4	bl32.bin, devkit-e8, and xipImage.	Yes	Yes	No	RFS – Ext3	BASE_IMAGE2	3.9 + 2.9

Figure 6 – Supported Images

4.1.1.2 Steps to Program Linux Images into MRAM Flash

Follow these steps:

1. Create an application image that includes Linux images.
2. Use SETOOLS to program the ATOC (containing the application images) into MRAM.

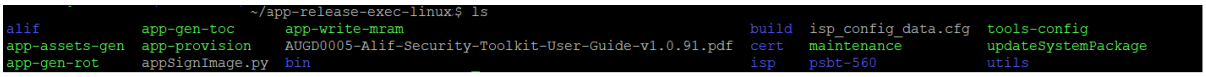
4.1.1.3 Create an Application Image

1. Log in to a Linux host or a Linux distribution on Windows.

5. Open a terminal as a regular user:

    - Log in with your user account.
    - Launch the terminal.
    - Navigate to the SETOOLS directory (e.g., *~/app-release-exec-linux*):

```
$ cd ~/app-release-exec-linux
$ ls
```



6. Generate the application image with the app-gen-toc script using the provided config file:

```
$ ./app-gen-toc -f build/config/APSS_TINY_IMAGE.json
```

This creates *AppTocPackage.bin* in the build directory.

4.1.1.4 Program the ATOC into MRAM

1. Program the application image into MRAM via ISP using app-write-mram with sudo:

```
$ sudo ./app-write-mram -p
```

Notes:

- The SE-UART is typically detected as /dev/ttyACM0. Use dmesg to confirm the device node.
- The -p option adds padding for 16-byte alignment.

4.1.1.5 BASE_IMAGE

```
The default image that will be built includes distro features such as: “apss-sd-share,” “apss-usb,” “apss-i2c,” “apss-spi,” “apss-mhu,” “apss-hwsem,” “apss-debug,” and “apss-crc.”
```

- **Booting from MRAM:** bl32.bin, devkit-e8.dtb, xipImage and cramfs.
- **RAM: STITCHED (SRAM0 + SRAM1) = 8MB**

1. Generate the application image with the app-gen-toc script using the provided config file:

```
$ ./app-gen-toc -f build/config/APSS_BASE_IMAGE.json
```

This creates *AppTocPackage.bin* in the build directory.

2. Program the application image into MRAM via ISP using app-write-mram with sudo:

```
$ sudo ./app-write-mram -p
```

Notes:

- The SE-UART is typically detected as /dev/ttyACM0. Use dmesg to confirm the device node.
- The -p option adds padding for 16-byte alignment.

4.1.1.6 BASE_IMAGE2

```
The BASE_IMAGE2 includes BASE_IMAGE and additional distro features: "apss-i2s", "apss-i3c", "apss-ethernet", "apss-utimer", "apss-pdm".
```

Booting from MRAM: bl32.bin, devkit-e8.dtb, xipImage.

OSPIFLASH: Cramfs

RAM: HYPERRAM = 64MB

1. Follow section 4.1.2 "Programming xipImage and cramfs-xip into the OSPI1 Flash using the PC Tool" to flash the cramfs "alif-tiny-image-devkit-e8.rootfs.cramfs.xip" onto the device OSPIFLASH.
2. Generate the application image with the app-gen-toc script using the provided config file:

```
$ ./app-gen-toc -f build/config/APSS_BASE_IMAGE2.json
```

This creates *AppTocPackage.bin* in the build directory.

3. Program the application image into MRAM via ISP using app-write-mram with sudo:

```
$ sudo ./app-write-mram -p
```

Notes:

- The SE-UART is typically detected as /dev/ttyACM0. Use dmesg to confirm the device node.
- The -p option adds padding for 16-byte alignment.

4.1.1.7 BASE_IMAGE3

Same as **BASE_IMAGE2** distro features.

Booting from MRAM: bl32.bin, devkit-e8.dtb.

OSPIFLASH: xipImage and Cramfs

RAM: HYPERRAM + STITCHED SRAM = (64 + 8) = 72MB.

1. Follow section 4.1.2 “Programming xipImage and cramfs-xip into the OSPI1 Flash using the PC Tool” to flash the xipImage and cramfs onto the device OSPIFLASH.
2. Generate the application image with the app-gen-toc script using the provided config file:

```
$ ./app-gen-toc -f build/config/APSS_BASE_IMAGE3.json
```

This creates *AppTocPackage.bin* in the build directory.

4. Program the application image into MRAM via ISP using app-write-mram with sudo:

```
$ sudo ./app-write-mram -p
```

Notes:

- The SE-UART is typically detected as /dev/ttyACM0. Use dmesg to confirm the device node.
- The -p option adds padding for 16-byte alignment.

4.1.1.8 BASE_IMAGE4

Same as **BASE_IMAGE2** distro features.

Booting from MRAM: bl32.bin, devkit-e8.dtb, xipImage.

SD Card: Root Filesystem (RFS)

RAM: HYPERRAM + STITCHED SRAM = (64 + 8) = 72MB.

1. Follow section 4.1.3 “Creating a Partition on an SD Card and Copying the rootfs (in ext4 Format)”.
2. Generate the application image with the app-gen-toc script using the provided config file:

```
$ ./app-gen-toc -f build/config/APSS_BASE_IMAGE4.json
```

This creates *AppTocPackage.bin* in the build directory.

5. Program the application image into MRAM via ISP using app-write-mram with sudo:

```
$ sudo ./app-write-mram -p
```

Notes:

- The SE-UART is typically detected as /dev/ttyACM0. Use dmesg to confirm the device node.
- The -p option adds padding for 16-byte alignment.

4.1.2 Programming xipImage and cramfs-xip into the OSPI1 Flash using the PC Tool

This section explains how to program xipImage and cramfs-xip (both 16-byte AES encrypted and unencrypted) into the OSPI1 NOR flash at address 0xC0000000 using a host PC tool. The process involves the RTSS_HE running an OSPI1 burner binary (PC_Tool_HE.bin) that receives images via UART 2 from the PC tool (flashtool) on a Linux host.

The RTSS_HE boots with the OSPI1 burner binary, which programs the OSPI1 NOR flash based on data sent from the host PC tool over UART 2. The host PC tool, executed on a Linux host or a Linux VM on Windows 11, allows you to specify the images and their flash addresses.

4.1.2.1 Programming Steps

Follow these steps to program the images into OSPI1 NOR flash:

1. **Program MRAM with the OSPI1 Burner Binary.**

Load the MRAM with an ATOC (Alif Table of Contents) containing the OSPI1 burner binary (PC_Tool_HE.bin):

- Use the configuration file: /home/$USER/APSS-v2.1.0/Utilities/Eagle_A0_PC_Tool/config-files/RTSS-PC-tool-burner.json.
- Refer to the section *Programming Linux Images into MRAM Flash* in the original documentation for detailed instructions on creating and programming the ATOC.

7. **Launch the PC Tool**

Open a new shell terminal on the Linux host or VM, navigate to the PC tool directory, and run flashtool with sudo privileges:

```
cd /home/$USER/APSS-v2.1.0/Utilities/Eagle_A0_PC_Tool/Flash-Tool/
sudo ./flashtool
```

8. **Configure and Program the Images**

   In the flashtool graphical interface:

   - Select the **BOARD** (e.g., DevKit-E8).
   - Choose the **COM** port connected to UART 2.
   - Specify the images to program:
   - Enter the programming address
   - Click **START FLASHING!** to begin programming.

- Click **CLOSE** when the process completes.

- **For Programming Un-encrypted xipImage and cramfs-xip Images**

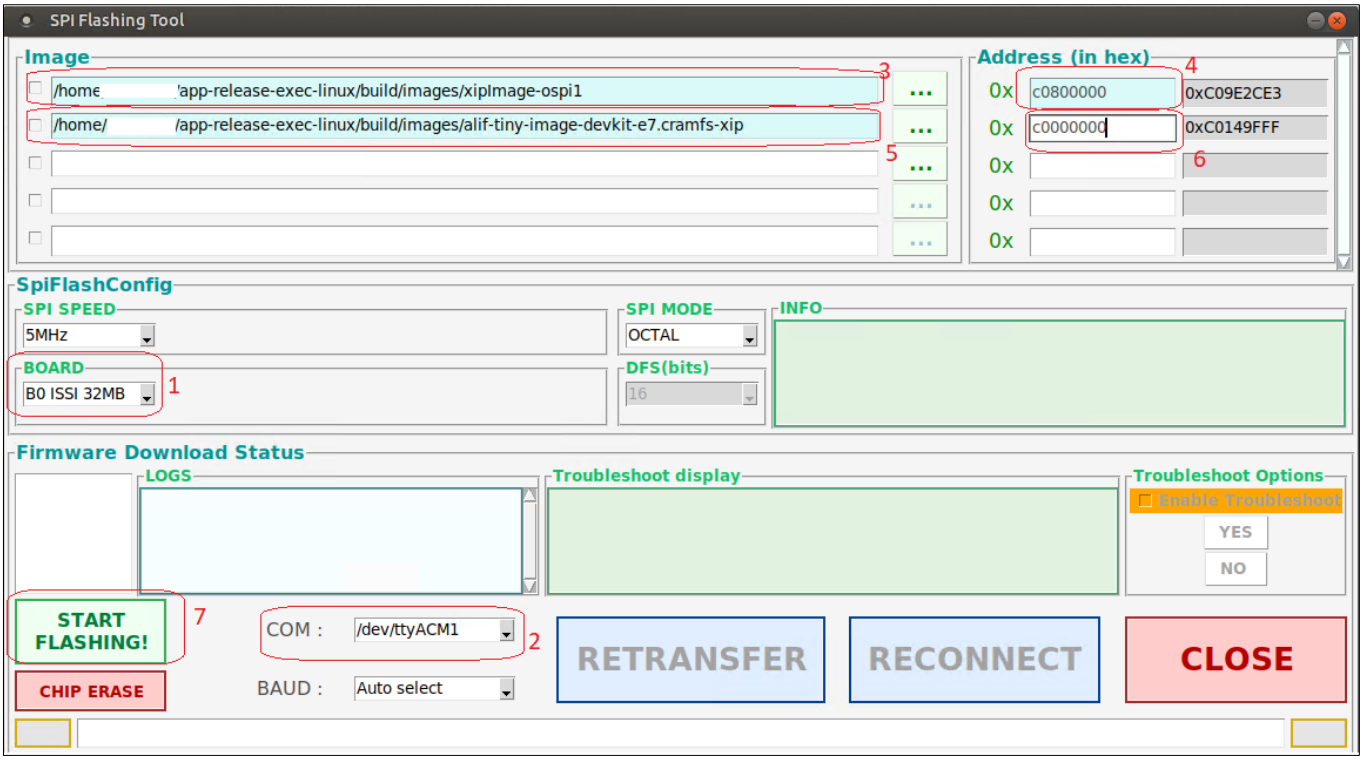
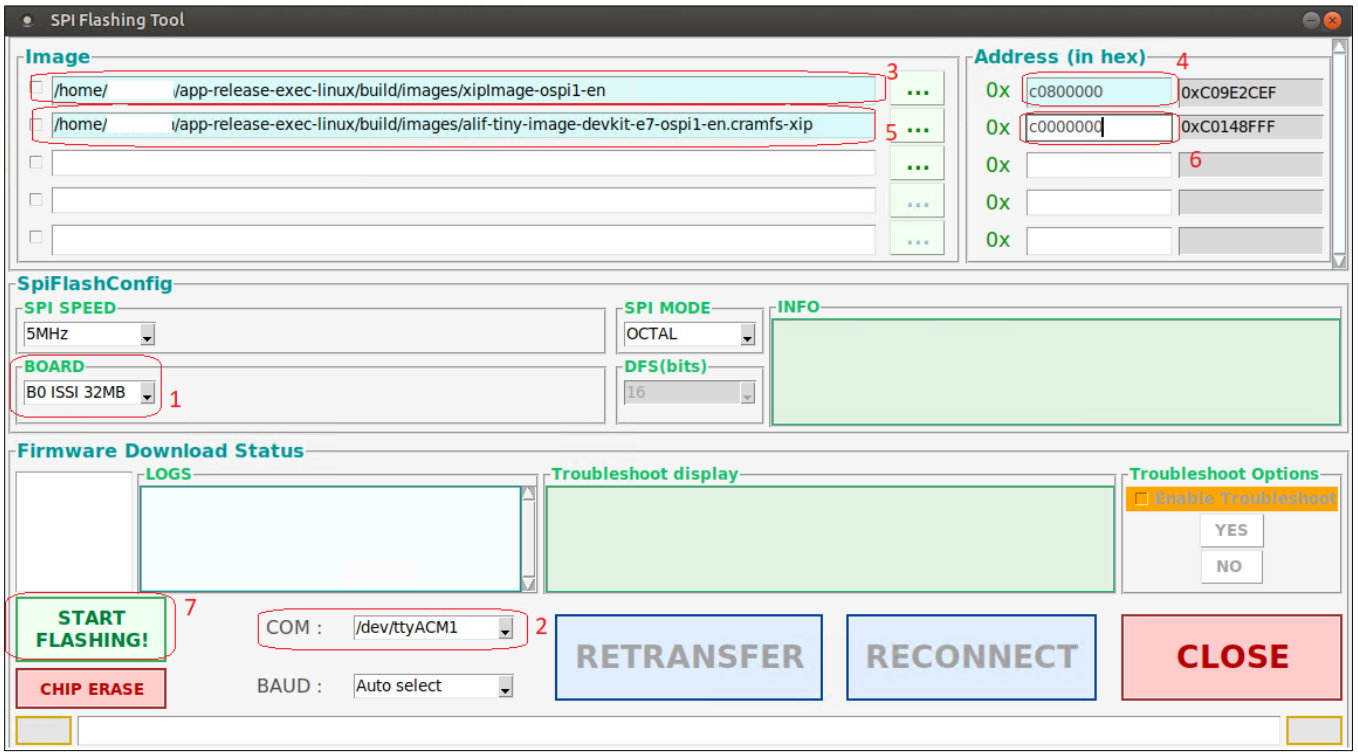


Figure 7

- **For Programming AES Encrypted xipImage and cramfs-xip Images**



4.1.3 Creating a Partition on an SD Card and Copying the rootfs (in ext4 Format)

Procedure

Follow the steps below to create an ext4 partition on an SD card and copy the root filesystem (alif-tiny-image-devkit-e8.ext4) onto it.

1. **Prepare the SD Card**

   a. Insert the SD Card

   Insert an SD card into an SD card reader and connect the reader to the host machine.

   b. **Identify the SD Card Device Node**

   The SD card typically appears as a device node such as /dev/sdX on a Linux host or a Linux distribution running on Windows 11. Verify the device node using the following commands:

   ```
   $ dmesg
   ```

   or

   ```
   $ sudo fdisk -l
   ```

   Example: The SD card is detected as */dev/sdc.*

   c. **Create an ext4 Partition**

   Use fdisk to create a new partition on the SD card. Replace /dev/sdc with the actual device node detected for your SD card.

   Command:

   ```
   sudo fdisk /dev/sdc
   Use p to print the current partition table to the terminal from within
   command mode.
   Command (m for help): p
   Use the d command to delete a partition if it there in SD CARD.
   Command (m for help): d
   Use the n command to create a new partition.
   Command (m for help): n
   Use the p command to create a 1st new primary 2GB ext4 partition.
   Command (m for help): p
   Partition number (1-128, default 1): <Press Enter>
   First sector (2048-50011815, default 2048): <Press Enter>
   ```

```
Last sector, +/-sectors or +/-size{K,M,G,T,P} (2048-500118158, default
500118158): <+2G>
Save the changes by running the w command:
Command (m for help): w
Type 'q' to come out of fdisk prompt
Command (m for help): q
```

```
$ sudo fdisk /dev/sdc

Welcome to fdisk (util-linux 2.31.1).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.


Command (m for help): p
Disk /dev/sdc: 29.8 GiB, 31976325120 bytes, 62453760 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x5cae097e

Device     Boot Start     End Sectors Size Id Type
/dev/sdc1        2048 4196351 4194304   2G 83 Linux

Command (m for help): d
Selected partition 1
Partition 1 has been deleted.
```

```
Command (m for help): n
Partition type
   p   primary (0 primary, 0 extended, 4 free)
   e   extended (container for logical partitions)
Select (default p): p
Partition number (1-4, default 1): 1
First sector (2048-62453759, default 2048):
Last sector, +sectors or +size{K,M,G,T,P} (2048-62453759, default 62453759): +2G

Created a new partition 1 of type 'Linux' and of size 2 GiB.
Partition #1 contains a ext4 signature.

Do you want to remove the signature? [Y]es/[N]o: y

The signature will be removed by a write command.

Command (m for help): w
The partition table has been altered.
Calling ioctl() to re-read partition table.
Syncing disks.

$ sudo mkfs.ext4 /dev/sdc1
mke2fs 1.44.1 (24-Mar-2018)
Creating filesystem with 524288 4k blocks and 131072 inodes
Filesystem UUID: 64bd55aa-80e3-4cfb-aec1-8b16639f4d11
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376, 294912
```

d. **Format the Partition as ext4**

Format the first partition on the SD card as ext4. Replace /dev/sdc1 with the actual partition node detected.

Command:

```
$ sudo mkfs.ext4 -L root /dev/sdc1
```

```
$ sudo mkfs.ext4  -L root  /dev/sdc1
mke2fs 1.44.1 (24-Mar-2018)
Creating filesystem with 524288 4k blocks and 131072 inodes
Filesystem UUID: 64bd55aa-80e3-4cfb-aec1-8b16639f4d11
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376, 294912

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
```

e. **Copy the rootfs to the SD Card**

Use the dd command to copy the ext4 root filesystem (alif-tiny-image-devkit-e8.ext4) to the SD card partition. Replace the input file path and output device node as needed.

Command:

```
$ sudo dd if=/path_to_alif-tiny-image-devkit-e8.ext4 of=/dev/sdXX
```

Example:

```
$ sudo dd if=/home/$USER/app-release-exec-linux/build/images/alif-tiny-image-devkit-e8.ext4 of=/dev/sdc1
```

f. **Finalize and Install the SD Card**

Safely remove the SD card from the reader and insert it into the SD card slot on the DevKit device.

4.1.4 Programming bl32.bin, devkit-e8.dtb, and xipImage into MRAM Using Alif Security Toolkit (SETOOLS)

Procedure

Follow the steps below to program the MRAM with the specified Linux images and enable the DevKit to mount the root filesystem (rootfs) from an SD card.

1. Program MRAM with ATOC

Use the Alif Security Toolkit (SETOOLS) to program the MRAM with an ATOC (Application Table of Contents) that includes the Linux images (bl32.bin, devkit-e8.dtb, and xipImage). This configuration will enable mounting the rootfs from the SD card.

- **Configuration File:** Utilize the JSON file located at:

/home/$USER/APSS-v2.1.0-Beta/config-files/APSS-tiny-linux-sd-boot.json

- **Instructions:** For detailed steps on programming the MRAM with the ATOC, refer to the section 4.1.1.

2. Power Cycle the DevKit Board

After programming the MRAM, power off and then power on the DevKit board to initiate the boot process.

3. **Boot and Login**

   The Linux kernel will boot, mount the rootfs from the SD card, and present a login prompt. Log in using the username root.

4.1.5 Programming Images for Verification of Hyper RAM Functionality

This section explains how to program Linux images to boot from OSPI1 NOR flash and verify Hyper RAM functionality at address 0xA0000000 on the Alif DevKit. After booting, you'll use the hyperram_test program to confirm Hyper RAM data allocation.

4.1.5.1 Prerequisites

- Complete the steps in section 6.4.13 to generate the required images (bl32.bin, devkit-e8.dtb, xipImage, alif-tiny-image-devkit-e8.cramfs-xip).

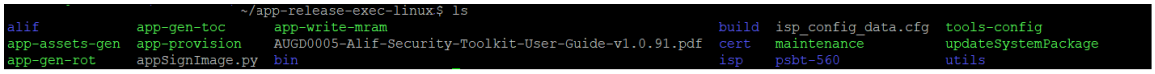
4.1.5.2 Programming and Verification Steps

Follow these steps to program the images and verify Hyper RAM functionality:

1. **Navigate to the Alif Security Toolkit Directory**

Open a terminal and change to the directory containing the Alif Security Toolkit tools (app-gen-toc, app-write-mram, updateSystemPackage). This example assumes the tools are at *~/app-release-exec-linux:*

```
cd ~/app-release-exec-linux
ls
```



1. **Create an Application Image (ATOC)**

Use the app-gen-toc script with the APSS-tiny-linux.json configuration file to generate an Alif Application Table of Contents (ATOC):

```
./app-gen-toc -f build/config/APSS-tiny-linux.json
```

2. **Program the MRAM with the ATOC**

Run app-write-mram with sudo privileges to program AppTocPackage.bin into the MRAM via the In-System Programmer (ISP):

```
sudo ./app-write-mram -p
```

- **Syntax Explanation**: The -p option ensures 16-byte alignment by adding padding bytes.
- **Note**: The SE-UART is assumed to be /dev/ttyACM0. Use dmesg (e.g., dmesg | grep ttyACM) to identify the correct device node if different.

3. **Boot Linux**

After programming, the DevKit boots Linux using xipImage and cramfs-xip from OSPI1 NOR flash:

- Log in with the username root (no password required by default).

4. **Run the Hyper RAM Test**

   Execute the hyperram_test program on the target to allocate and verify data in Hyper RAM:

- **Step 1**: Run the test to allocate 4 KB of dynamic memory and write 0xA5 across 32 MB (8192 iterations × 4 KB):

```
hyperram_test
```

- **Step 2**: Verify the data at Hyper RAM addresses (starting at 0xA0000000 or decimal 2684354560):

```
for iter in `seq 8192`; do echo $iter; echo -n "Data at address `expr
2684354560 + $iter * 4096` is "; devmem `expr 2684354560 + $iter * 4096`;
done | grep 0xA5A5A5A5
```

4.1.5.3 Sample Output

```
# hyperram_test
HyperRAM test that writes 0xA5 data of 4K bytes for 800 iterations.
Count 0: Wrote at virtual address 0x477010
Count 1: Wrote at virtual address 0x478020
Count 2: Wrote at virtual address 0x479030
Count 3: Wrote at virtual address 0x47a040
Count 4: Wrote at virtual address 0x47b050
Count 5: Wrote at virtual address 0x47c060
Count 6: Wrote at virtual address 0x47d070
Count 7: Wrote at virtual address 0x47e080
Count 8: Wrote at virtual address 0x47f090
Count 9: Wrote at virtual address 0x4800a0
Count 10: Wrote at virtual address 0x4810b0
Count 11: Wrote at virtual address 0x4820c0
Count 12: Wrote at virtual address 0x4830d0
```

```
# for iter in `seq 8192` ; do echo $iter ; echo -n "Data
at address `expr 2684354560 + $iter * 4096` is "; devmem `expr 268435456
0 + $iter * 4096` ; done | grep 0xA5A5A5A5
Data at address 2684358656 is 0xA5A5A5A5
Data at address 2684362752 is 0xA5A5A5A5
Data at address 2684366848 is 0xA5A5A5A5
Data at address 2684370944 is 0xA5A5A5A5
Data at address 2684375040 is 0xA5A5A5A5
Data at address 2684379136 is 0xA5A5A5A5
Data at address 2684383232 is 0xA5A5A5A5
Data at address 2684387328 is 0xA5A5A5A5
Data at address 2684391424 is 0xA5A5A5A5
Data at address 2684395520 is 0xA5A5A5A5
Data at address 2684399616 is 0xA5A5A5A5
Data at address 2684403712 is 0xA5A5A5A5
```

Notes

- **Hyper RAM Address**: 2684354560 is the decimal representation of 0xA0000000, the base address of Hyper RAM.

4.2 Program xipImage and cramfs-xip to OSPI1 Flash Using the PC Tool

In this method, the Real-Time Subsystem High-Efficiency core (RTSS_HE) runs the OSPI1 burner binary (*PC_Tool_HE.bin*) during boot-up. This binary receives images and the OSPI1 NOR flash location via UART2, then programs the flash. A host PC tool, running on a Linux host, facilitates this by collecting the images and flash address from the user and sending them to RTSS_HE over UART2.

Note: This approach also works for programming OSPI1 with Linux images up to 2 MB in size.

Follow these steps to program xipImage and cramfs-xip (both unencrypted and 16-byte AES-encrypted versions) into OSPI1 flash at address 0xC0000000 using the host PC tool located at /home/$USER/APSS-v2.1.0-Beta/Utilities/PC-Tool/flashtool:

1. **Program MRAM with the OSPI1 burner binary:**
    - Use the configuration file */home/$USER/APSS-v2.1.0-Beta/config-files/APSS-PC-tool-burner.json* to create an ATOC containing PC_Tool_HE.bin.
    - Refer to the *Program Linux Images into MRAM Flash* for detailed steps on programming MRAM with this ATOC.
2. **Run the flashtool:**
    - Open a new shell on your Linux host or a Linux VM on Windows 11.
    - Navigate to the PC Tool directory and execute flashtool with sudo:

```
$ cd /home/$USER/APSS-v2.1.0-Beta/Utilities/PC-Tool/
$ sudo ./flashtool
```

3. **Configure and flash the images:**
    - In the flashtool interface, select the BOARD, COM port, and images to program.
    - Enter the flash address (0xC0000000) as shown in Figures 4 and 5.
    - Click **START FLASHING!.** When complete, click **CLOSE**.

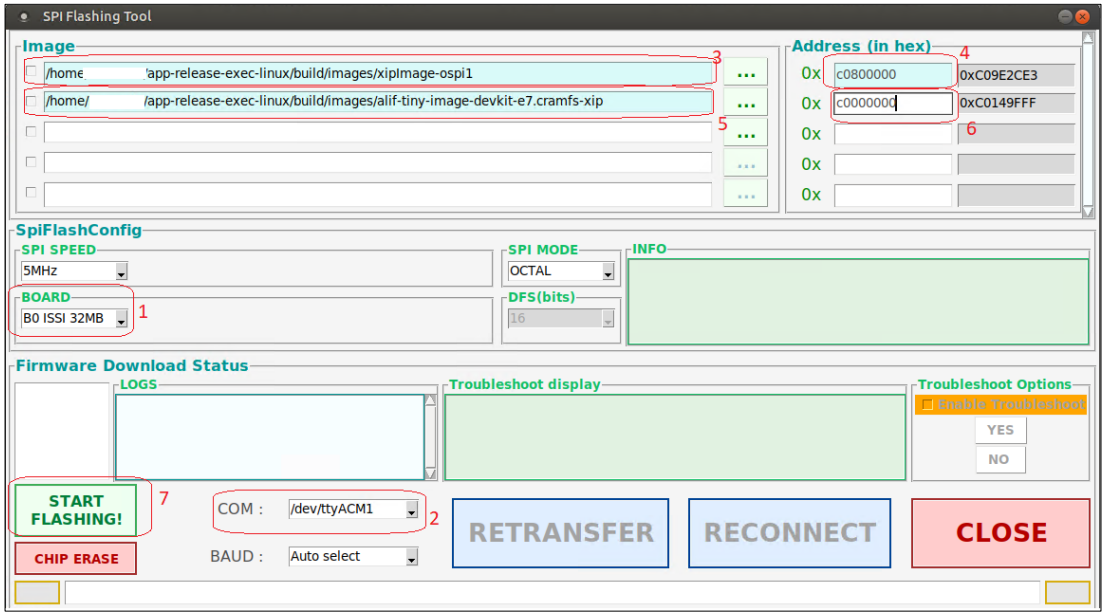


Figure 8: Programming Unencrypted xipImage and cramfs-xip Images

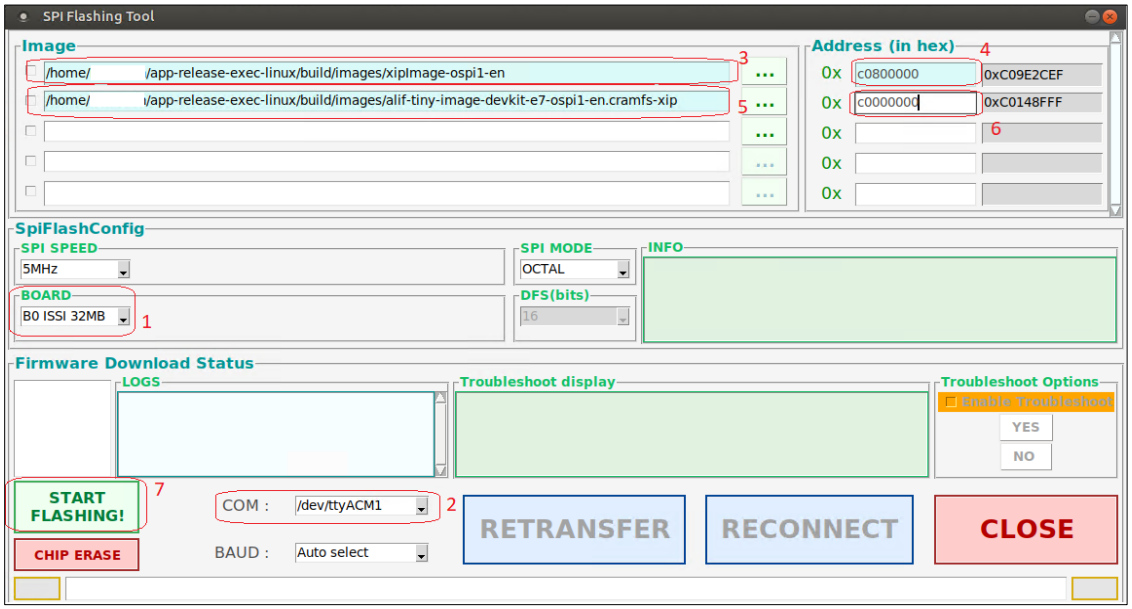


Figure 9: Programming AES-Encrypted xipImage and cramfs-xip Images

4. **Navigate to the Alif Security Toolkit directory:**

Go to the directory containing *app-gen-toc*, *app-write-mram,* and *updateSystemPackage* (e.g., *~/app-release-exec-linux*):

```
$ cd ~/app-release-exec-linux
```

```
$ ls
```

5. **Create an application image:**

Use the app-gen-toc script to generate an application image:

- For unencrypted images, use APSS-OSPI1.json:

```
$ ./app-gen-toc -f build/config/APSS-OSPI1.json
```

- For AES-encrypted images, use APSS-OSPI1-en.json:

```
$ ./app-gen-toc -f build/config/APSS-OSPI1-en.json
```

This creates AppTocPackage.bin in the build directory.

6. **Program the application image into MRAM:**

Run app-write-mram with sudo to program AppTocPackage.bin via ISP:

```
$ sudo ./app-write-mram -p
```

Notes:

1. The SE-UART is typically detected as /dev/ttyACM0. Use dmesg to confirm the device node.
2. The -p option adds padding for 16-byte alignment.

---

7. **Boot Linux from OSPI1:**

- The system boots with xipImage and cramfs-xip from OSPI1 NOR flash.
- Log in with the username root.

5. Booting Linux

5.1 Boot Linux on the APSS from MRAM

Follow these steps to boot Linux on the Application Processor Subsystem (APSS) from MRAM:

1. Open a terminal application like Minicom or PuTTY on your Linux host PC.
5. Configure the connection:
    - Select the UART2 USB port (e.g., /dev/ttyACM1).
    - Set the baud rate to 115,200.
6. Power cycle the DevKit by pressing the reset switch (SW2).
7. Observe the boot process:
    - If uninterrupted, "Linux Hello World!" prints continuously to the UART2 console.
    - If interrupted (by pressing any key), a login prompt appears. Enter root to access the BusyBox shell.

5.1.1 Boot Logs

The pre-built images boot using the new SRAM blocks. Below are the boot logs you'll see when booting from the Modem SRAM blocks:

```
NOTICE:  SP_MIN: v2.10.8(debug):7b985c2c3
NOTICE:  SP_MIN: Built : 04:36:27, Jan 29 2025
INFO:    ARM GICv2 driver initialized
INFO:    SP_MIN: Initializing runtime services
INFO:    SP_MIN: Preparing exit to normal world
INFO:    Entry point address = 0x80020000
INFO:    SPSR = 0x1d3
Booting Linux on physical CPU 0x0
Linux version 6.12.6-yocto-standard (oe-user@oe-host) (arm-poky-linux-
musleabi-gcc (GCC) 13.3.0, GNU ld (GNU Binutils) 2.42.0.20240723) #1 PREEMPT
Sun Mar  9 01:16:22 UTC 2025
CPU: ARMv8-a aarch32 Processor [411fd010] revision 0 (ARMv8), cr=50c5383d
CPU: PIPT / VIPT nonaliasing data cache, VIPT aliasing instruction cache
OF: fdt: Machine model: devkit-e8
Memory policy: Data cache writeback
Zone ranges:
```

```
  Normal   [mem 0x0000000002000000-0x000000000827ffff]
  HighMem  empty
Movable zone start for each node
Early memory node ranges
  node   0: [mem 0x0000000002000000-0x00000000023fffff]
  node   0: [mem 0x0000000008020000-0x000000000827ffff]
Initmem setup node 0 [mem 0x0000000002000000-0x000000000827ffff]
On node 0, zone Normal: 32 pages in unavailable ranges
On node 0, zone Normal: 384 pages in unavailable ranges
psci: probing for conduit method from DT.
psci: PSCIv1.1 detected in firmware.
psci: Using standard PSCI v0.2 function IDs
psci: MIGRATE_INFO_TYPE not supported.
psci: SMC Calling Convention v1.4
pcpu-alloc: s0 r0 d32768 u32768 alloc=1*32768
pcpu-alloc: [0] 0
Kernel command line: console=ttyS0,115200n8 root=mtd:physmap-flash.0
rootfstype=cramfs ro loglevel=9
Dentry cache hash table entries: 1024 (order: 0, 4096 bytes, linear)
Inode-cache hash table entries: 1024 (order: 0, 4096 bytes, linear)
Built 1 zonelists, mobility grouping off.  Total pages: 1632
mem auto-init: stack:all(zero), heap alloc:off, heap free:off
SLUB: HWalign=64, Order=0-3, MinObjects=0, CPUs=1, Nodes=1
rcu: Preemptible hierarchical RCU implementation.
rcu:    RCU event tracing is enabled.
rcu: RCU calculated value of scheduler-enlistment delay is 10 jiffies.
NR_IRQS: 16, nr_irqs: 16, preallocated irqs: 16
rcu: srcu_init: Setting srcu_struct sizes based on contention.
arch_timer: cp15 and mmio timer(s) running at 100.00MHz (phys/phys).
clocksource: arch_sys_counter: mask: 0x1fffffffffffff max_cycles:
0x171024e7e0, max_idle_ns: 440795205315 ns
sched_clock: 57 bits at 100MHz, resolution 10ns, wraps every 4398046511100ns
Switching to timer-based delay loop, resolution 10ns
Console: colour dummy device 80x30
```

```
Calibrating delay loop (skipped), value calculated using timer frequency..
200.00 BogoMIPS (lpj=1000000)
CPU: Testing write buffer coherency: ok
pid_max: default: 32768 minimum: 301
Mount-cache hash table entries: 1024 (order: 0, 4096 bytes, linear)
Mountpoint-cache hash table entries: 1024 (order: 0, 4096 bytes, linear)
Setting up static identity map for 0x800201a0 - 0x800201ec
rcu: Hierarchical SRCU implementation.
rcu:    Max phase no-delay instances is 1000.
Memory: 5624K/6528K available (1735K kernel code, 121K rwdata, 340K rodata,
105K init, 102K bss, 440K reserved, 0K cma-reserved, 0K highmem)
devtmpfs: initialized
VFP support v0.3: implementor 41 architecture 3 part 40 variant 1 rev 2
clocksource: jiffies: mask: 0xffffffff max_cycles: 0xffffffff, max_idle_ns:
19112604462750000 ns
futex hash table entries: 256 (order: -1, 3072 bytes, linear)
pinctrl core: initialized pinctrl subsystem
NET: Registered PF_NETLINK/PF_ROUTE protocol family
DMA: preallocated 256 KiB pool for atomic coherent allocations
ensemble-pinctrl 1a603000.pinctrl: Ensemble pinctrl initialized
OF: /client: could not find phandle 4
clocksource: Switched to clocksource arch_sys_counter
NET: Registered PF_UNIX/PF_LOCAL protocol family
workingset: timestamp_bits=30 max_order=11 bucket_order=0
io scheduler mq-deadline registered
io scheduler kyber registered
Serial: 8250/16550 driver, 8 ports, IRQ sharing disabled
printk: legacy console [ttyS0] disabled
4901a000.serial: ttyS0 at MMIO 0x4901a000 (irq = 46, base_baud = 6250000) is
a 16550A
printk: legacy console [ttyS0] enabled
4901b000.serial: ttyS1 at MMIO 0x4901b000 (irq = 47, base_baud = 6250000) is
a 16550A
4901c000.serial: ttyS2 at MMIO 0x4901c000 (irq = 48, base_baud = 6250000) is
a 16550A
```

```
physmap-flash physmap-flash.0: physmap platform flash device: [mem
0x80380000-0x8057ffff]
SMCCC: SOC_ID: ARCH_SOC_ID not implemented, skipping ....
alif_hwsem 4902e000.hwsem0: Alif hardware semaphore registered
alif_hwsem 4902e010.hwsem1: Alif hardware semaphore registered
alif_hwsem 4902e020.hwsem2: Alif hardware semaphore registered
alif_hwsem 4902e030.hwsem3: Alif hardware semaphore registered
alif_hwsem 4902e040.hwsem4: Alif hardware semaphore registered
alif_hwsem 4902e050.hwsem5: Alif hardware semaphore registered
alif_hwsem 4902e060.hwsem6: Alif hardware semaphore registered
alif_hwsem 4902e070.hwsem7: Alif hardware semaphore registered
alif_hwsem 4902e080.hwsem8: Alif hardware semaphore registered
alif_hwsem 4902e090.hwsem9: Alif hardware semaphore registered
alif_hwsem 4902e0a0.hwsem10: Alif hardware semaphore registered
alif_hwsem 4902e0b0.hwsem11: Alif hardware semaphore registered
alif_hwsem 4902e0c0.hwsem12: Alif hardware semaphore registered
alif_hwsem 4902e0d0.hwsem13: Alif hardware semaphore registered
alif_hwsem 4902e0e0.hwsem14: Alif hardware semaphore registered
alif_hwsem 4902e0f0.hwsem15: Alif hardware semaphore registered
clk: Disabling unused clocks
dw-apb-uart 4901a000.serial: forbid DMA for kernel console
cramfs: checking physical address 0x80380000 for linear cramfs image
cramfs: linear cramfs image on mtd:physmap-flash.0 appears to be 1292 KB in
size
VFS: Mounted root (cramfs filesystem) readonly on device 31:0.
Freeing unused kernel image (initmem) memory: 36K
This architecture does not have kernel memory protection.
Run /sbin/init as init process
  with arguments:
    /sbin/init
  with environment:
    HOME=/
    TERM=linux
mount: mounting sysfs on /sys failed: No such device
*** Press any key within 5 seconds to login into a shell prompt ***
```

```
dw-apb-uart 4901b000.serial: failed to request DMA
dw-apb-uart 4901c000.serial: failed to request DMA
```

Alif Iota - Tiny Linux Distribution 0.1 devkit-e8 /dev/ttyS0

```
devkit-e8 login: root


login[46]: root login on 'ttyS0'
```

5.1.2 Booting Linux with xipImage and cramfs-xip Images from OSPI1 NOR Flash

This section explains how to boot Linux on the Alif DevKit using xipImage and cramfs-xip images (both unencrypted and AES-encrypted) from OSPI1 NOR flash. The process involves preparing the images, programming the MRAM with an application table of contents (ATOC), and initiating the boot sequence.

Prerequisites

Before proceeding, ensure the following steps are completed based on your image type:

- **For Unencrypted Images**:
  - Build Linux images for OSPI1 NOR flash (Section *6.4.10*).
  - Program xipImage and cramfs-xip into OSPI1 flash using the PC tool (Section *4.1.2*).
- **For AES-Encrypted Images**:
  - Build encrypted Linux images for OSPI1 NOR flash (Section *6.4.12*).
  - Program encrypted xipImage and cramfs-xip into OSPI1 flash using the PC tool (Section *4.1.2*)

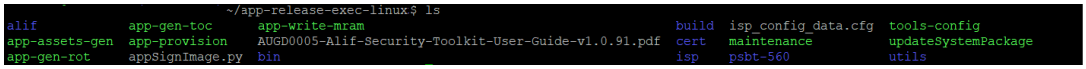
5.1.2.1 Booting Steps

Follow these steps to boot Linux from OSPI1 NOR flash:

1. **Navigate to the Alif Security Toolkit Directory**

Open a terminal and change to the directory containing the Alif Security Toolkit tools (app-gen-toc, app-write-mram, and updateSystemPackage). This example assumes the tools are at *~/app-release-exec-linux:*

```
cd ~/app-release-exec-linux
ls
```



2. **Create an Application Image (ATOC)**

Use the app-gen-toc script to generate an Alif Application Table of Contents (ATOC) image:

- **For Unencrypted Images**: Use the APSS-OSPI1.json configuration file:

./app-gen-toc -f build/config/APSS-OSPI1.json

- **For AES-Encrypted Images**: Use the APSS-OSPI1-en.json configuration file:

./app-gen-toc -f build/config/APSS-OSPI1-en.json

  - **Result**: Creates an AppTocPackage.bin file in the build/ directory.

3. **Program the MRAM with the ATOC**

Run app-write-mram with sudo to program the AppTocPackage.bin into the MRAM via the In-System Programmer (ISP):

```
sudo ./app-write-mram -p
```

- **Syntax Explanation:** The -p option ensures 16-byte alignment by adding padding bytes.

Note:

The SE-UART is assumed to be /dev/ttyACM0 in this example. Use the dmesg command to identify the correct device node on your host (e.g., dmesg | grep ttyACM).

---

4. **Boot Linux**

After programming, the DevKit boots Linux using xipImage and cramfs-xip from OSPI1 NOR flash:

  - Log in with the username root (no password required by default).

6. Building Linux Images

6.1 Build Flow of the APSS Linux Project

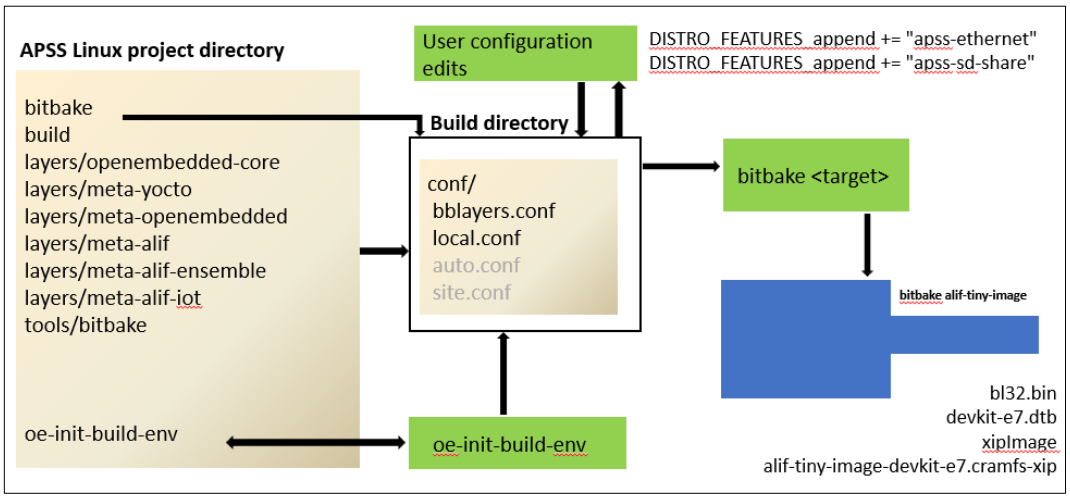


Figure 10 Build Flow of the APSS Linux Project

The components in this diagram are detailed in the **1.2**
Additionally, two configuration files in the build directory customize the Linux kernel and root filesystem (rootfs):

- **conf/bblayers.conf:** Adds layers to the build environment.
- **conf/local.conf:** Allows user-specific settings for building the Linux kernel and rootfs.

Running bitbake alif-tiny-image generates these images:

- bl32.bin
- devkit-e8.dtb
- xipImage
- alif-tiny-image-devkit-e8.cramfs-xip

The build flow shows how BitBake reads recipes from the layers directory and user configurations from the conf directory to execute predefined tasks, producing the Linux images.

6.2 BitBake Tasks

Here are the key tasks performed by BitBake:

- do_fetch: Downloads source code using the SRC_URI variable and the appropriate fetcher module.

- do_unpack: Extracts source code into the working directory (${WORKDIR}). The S variable determines the final location.
- do_patch: Finds and applies patches to the source code.
- do_configure: Configures build-time options for the software.
- do_compile: Compiles the source code.
- do_install: Copies files to the staging area (${D}) for packaging.
- do_package: Analyzes and splits the staging area (${D}) contents into package subsets.
- do_package_write_deb: Creates Debian packages (deb files) and stores them in ${DEPLOY_DIR_DEB}.
- do_rootfs: Builds the root filesystem structure for the image.
- do_image: Generates the final image after do_rootfs completes package installation and post-processing.

Note: For more details, see the *Yocto Project Reference Manual*.

6.3 Description of Linux Images

This section describes the generated Linux images:

- xipImage:

  The Linux kernel for the Alif DevKit, running in Execute-in-Place (XiP) mode from NOR flash or other CPU-addressable non-volatile storage. This conserves RAM by keeping the text section in flash memory and only copying the read-write sections, such as data and stack, to SRAM. However, the uncompressed XiP kernel requires more storage space, and its flash address, which is used for linking and storage, must align with the build configuration.

  Key kernel settings include:

  - CONFIG_XIP_KERNEL: Enables XiP mode.
  - CONFIG_XIP_PHYS_ADDR: Specifies the flash address for linking and storing xipImage.
  - CONFIG_CRAMFS: Supports mounting the XiP cramfs filesystem.
  - CONFIG_CRAMFS_MTD: Loads applications directly from linear memory (e.g., flash).

- bl32.bin:

  A stage 3-2 secure bootloader built from Arm Trusted Firmware-A (TF-A). It runs early in the A32 boot process and handles:

  - Early platform-specific setup (e.g., memory mapping).
  - MMU initialization and activation.

- GICv2 driver set up for interrupts.
- System-level generic timer activation.
- Configuring OSPI1 NOR flash for XiP execution.
- Transfers execution to the Linux kernel.

- devkit-e8.dtb:

  The device tree binary is a data structure that describes the hardware specific to DevKit(APSS), including CPUs, memory, buses, and peripherals, enabling the Linux kernel to manage them.

- alif-tiny-image-devkit-e8.cramfs-xip:

  A tiny, embedded Linux distribution that uses a cramfs-xip filesystem. If the boot process is not interrupted for 5 seconds, it will execute the "Linux Hello World" user-space application.

6.4 Build Linux Images for the APSS

6.4.1 Set Up Your Host PC for a Linux Distribution

For Windows 11 or Later

1. Install a virtual machine app like VirtualBox or VMware.
2. Run Ubuntu 22.04/later inside the virtual machine.
   - Check online guides for help:
     - Install VMware on Windows
     - Set Up Ubuntu on VMware
     - Run Ubuntu on VirtualBox
3. Open a terminal and run this command with admin rights:

```
sudo sh /home/$USER/APSS-v2.1.0/config-files/host_setup.sh
```

For Ubuntu 22.04

4. Open a terminal and run this command with admin rights:

```
sudo sh /home/$USER/APSS-v2.1.0/config-files/host_setup.sh
```

6.4.2 Create a GitHub Personal Access Token

1. Go to your GitHub account.

Select **Settings** > **Developer Settings** > **Personal Access Tokens** > **Tokens (classic)**.

2. Click Generate **new token (classic)**.
3. Copy the token to use when cloning repositories.

6.4.3 Build Linux Images with Bitbake

Use a new terminal as a regular user on a Linux host or a Windows 11 virtual machine.

1. Clone the repository and switch to the scarthgap_yocto_5.0 branch:

```
git clone https://github.com/AlifSemi/alif_linux-apss-build-setup
-b scarthgap_yocto_5.0
```

Note:

See "Create a GitHub Personal Access Token" for cloning help.

---

2. Install required packages:

```
sudo alif_linux-apss-build-setup/scripts/host_setup.sh
```

3. Fetch Yocto and BSP layers:

```
cd alif_linux-apss-build-setup
sh scripts/fetch-layers.sh
```

Note:

Run sh scripts/fetch-layers.sh update for the latest updates.

---

4. Set up the build environment:

```
source scripts/setup.sh
```

5. By default, build is configured for APSS E8 with SMP support enabled.
6. To enable APSS E6 UniCore support, edit the conf/auto.conf file. Set the “MACHINE” variable to devkit-e6 instead of devkit-e8.

```
 vi conf/auto.conf
 MACHINE=“devkit-e8” to MACHINE=”devkit-e6”
```

7. To enable APSS E8 UniCore support, edit conf/local.conf file and add line SMP = “0”. This setting disables SMP and configures the build for UniCore.

```
echo ‘SMP = “0”’ >> conf/local.conf
```

8. Build the kernel and root filesystem images:

```
bitbake alif-tiny-image linux-alif
```

   - Find the images in *tmp/deploy/images/devkit-e8*:
     - bl32.bin
     - devkit-e8.dtb
     - xipImage-devkit-e8.bin
     - alif-tiny-image-devkit-e8.cramfs-xip

9. Boot setup: The images boot from MRAM at these addresses:
   - bl32.bin: 0x80002000
   - devkit-e8.dtb: 0x80010000
   - xipImage: 0x80020000
   - cramfs-xip: 0x80380000

6.4.4 Build an SDK Toolchain Installer

Use a new terminal as a regular user. Alternatively, use the pre-built installer at: */home/$USER/APSS-v2.1.0-Beta/SDK/apss-tiny-musl-x86_64-alif-tiny-image-cortexa32hf-neon-devkit-e8-toolchain-0.1.sh*

1. Clone the repository:

```
git clone https://github.com/alifsemi/alif_linux-apss-build-setup -b
scarthgap_yocto_5.0
```

2. Install packages:

```
sudo alif_linux-apss-build-setup/scripts/host_setup.sh
```

3. Fetch layers:

```
cd alif_linux-apss-build-setup
sh scripts/fetch-layers.sh
```

Note:

Run sh scripts/fetch-layers.sh update for updates.

---

8. Set up the environment:

```
source scripts/setup.sh
```

9. Build the SDK:

```
bitbake -c populate_sdk alif-tiny-image
```

10. The installer is at:

```
tmp/deploy/sdk/apss-tiny-musl-x86_64-alif-tiny-image-cortexa32hf-neon-devkit-e8-toolchain-0.1.sh
```

6.4.5 Understand APSS Memory Segments

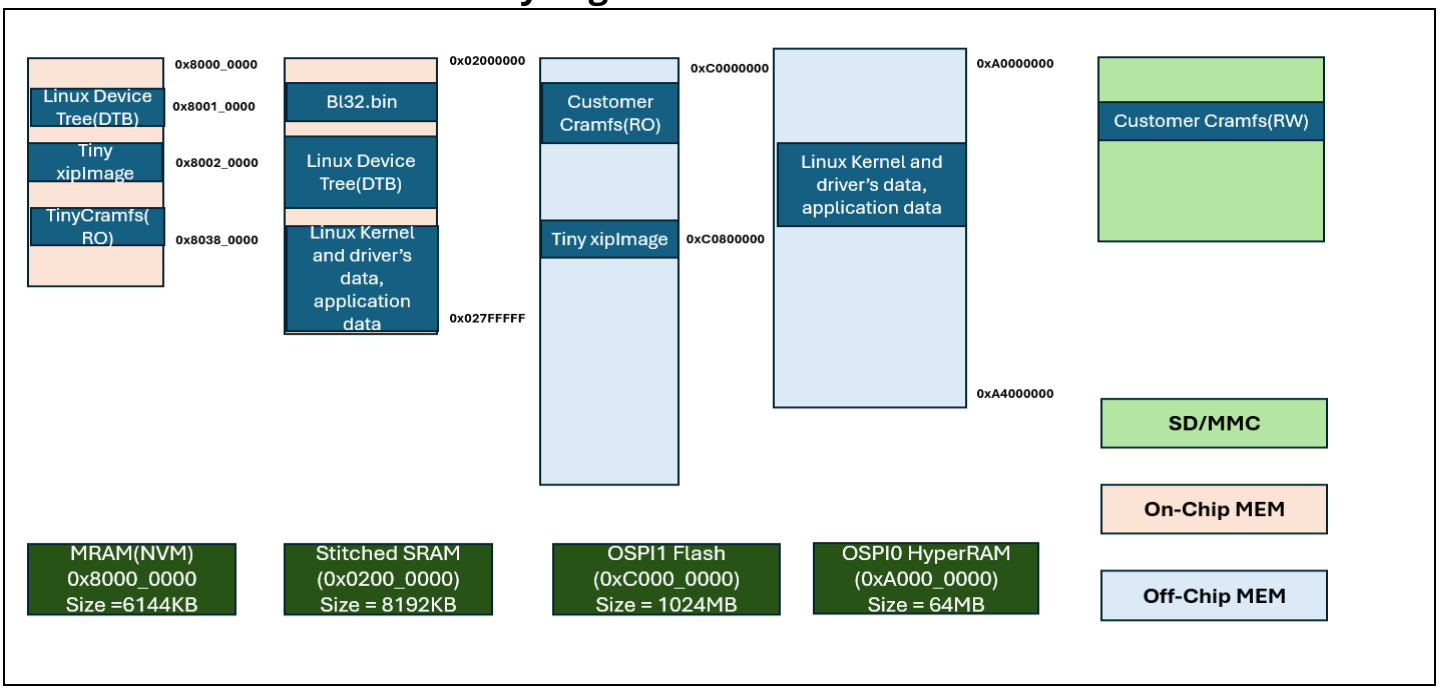


Figure 11 APSS Memory Segments

- **MRAM:** Stores APSS images (non-volatile). Runs bl32.bin at 0x80002000 in XiP mode, loads devkit-e8.dtb to Stitched SRAM, and executes xipImage and cramfs-xip from MRAM.
- **Stitch SRAM(SRAM0+SRAM1):** Provides 8 MB of combined data space for the bootloader, kernel, and drivers. Reserves (136KB) for core communication and ARM-TF.
- **OSPI0 HyperRAM:** Adds data space for apps needing more than SRAM offers.
- **OSPI1 NOR Flash:** Holds larger custom images like xipImage and cramfs-xip.

Note:

Default boot uses MRAM. For a read/write filesystem, use an SDMMC card with ext4.

6.4.6 Customize Your Build

You can create custom Linux builds with Docker, Alif recipes, and config files.

6.4.6.1 Configuration Files

Unedited files generate the default MRAM image. Edit them for custom drivers or memory setups

6.4.6.2 Build Instructions

Includes HyperRAM limits and SRAM options: Combine Stitch SRAM(SRAM0+SRAM1) (**default**) or split for M55 use.

6.4.6.3 RAM Requirements

A table lists RAM needs:

- Minimum driver set.
- Extra RAM for features like SD, USB, or Ethernet.

6.4.6.4 Build with Ethernet Support

1. **Edit local.conf:**

```
DISTRO_FEATURES:remove = " apss-usb"
DISTRO_FEATURES:append = " apss-ethernet"
```

Note:

You can swap out 'apss-usb' with 'apss-sd-share' or other options to use less RAM.

---

2. **Build:**

```
bitbake alif-tiny-image
```

3. Use images from *tmp/deploy/images/devkit-e8*.

6.4.7 Pre-built Image Memory Setup

- **xipImage:**
  - Uses Stitch SRAM (SRAM0+SRAM) default.
  - Boots from MRAM.
- **xipImage-ospi1:**
  - Uses Stitch SRAM(SRAM0+SRAM1) + HyperRAM or HyperRAM only
  - Boots from OSPI1.

6.4.8 Configuration Parameters for Boot Addresses of Linux Kernel, DTB, Rootfs and More

The default boot addresses in MRAM are:

- **bl32.bin**: 0x80002000
- **devkit-e8.dtb**: 0x80010000
- **xipImage**: 0x80020000
- **cramfs-xip**: 0x80380000

Below are the variables to customize these addresses:

Variable	Purpose	Format/Values	Default	Notes
BL32_IN_XIP_MEM	Chooses if bl32.bin runs from MRAM (XiP mode) or STITCHED SRAM	1 = MRAM, 0 = STITCHED SRAM(0x02000000)	1	1 uses BL32_XIP_BASE address: 0 uses fixed STITCHED SRAM1 address
BL32_XIP_BASE	Sets MRAM boot address for bl32.bin in XiP mode	Hex (4KB page-aligned, ends in 000)	0x80002000	Only applies if BL32_IN_XIP_MEM = 1; e.g., 0x80001000 (valid), 0x80001200 (invalid)
XIP_KERNEL_LOAD_ADDR	Defines MRAM boot address for xipImage (Linux kernel)	Hex (32-bit aligned, last 5 bits = 0)	0x80020000	Also informs bl32.bin; e.g., 0x80100020
KERNEL_XIP_RAM	Sets starting RAM address for xipImage execution	Hex	0x02000000	

Variable	Purpose	Format/Values	Default	Notes
KERNEL_DTB_ADDR	Sets MRAM load address for devkit-e8.dtb (device tree)	Hex	0x80010000	
KERNEL_MTD_START_ADDR	Sets MRAM load address for cramfs-xip (root filesystem)	Hex	0x80380000	
KERNEL_MTD_LEN	Sets size (bytes) of cramfs-xip partition from KERNEL_MTD_START_ADDR	Hex (min. 1MB = 0x100000)	0x200000 (2MB)	Adjust to 4MB (0x400000) for 2–3MB files, 8MB (0x800000) for 4–6MB to avoid boot errors
UART	Selects UART port for boot/serial console	2 = UART2, 4 = UART4	2	
HYPRAM_ONLY	Enables HyperRAM (0xA0000000) for extra data space	1 = Enabled, 0 = Disabled	0	Uses OSPI0 NOR flash
HYPERAM_STITCH	Enables HyperRAM (0xA0000000) and STITCH SRAM (0x02000000)	1 = Enabled, 0 = Disabled	0	Uses OSPI0 and Stitch SRAM
FLASH_EN	Enables XiP mode on OSPI1 NOR flash (0xC0000000) for xipImage/cramfs-xip	1 = Enabled, 0 = Disabled	0	For extra execution space

Variable	Purpose	Format/Values	Default	Notes
AES_EN	Enables decryption of AES-encrypted files in OSPI1 NOR flash	1 = Enabled, 0 = Disabled	0	
AES_ENC_KEY	16-byte AES key to decrypt files in OSPI1 NOR flash	16-byte string	“” (empty)	Must match encryption key

Additional Notes

Example Configuration: To boot from custom MRAM addresses (e.g., bl32.bin at 0x80008000, xipImage at 0x80100000, devkit-e8.dtb at 0x80080000, cramfs-xip at 0x80300000 with 2MB size), add to conf/local.conf:

```
BL32_XIP_BASE = "0x80008000"
XIP_KERNEL_LOAD_ADDR = "0x80100000"
KERNEL_DTB_ADDR = "0x80080000"
KERNEL_MTD_START_ADDR = "0x80300000"
KERNEL_MTD_LEN = "0x200000"
```

Build with bitbake linux-alif alif-tiny-image, find files in build/tmp/deploy/images/devkit-e8, and follow *Program Linux Images into MRAM Flash* for booting.

6.4.9 Configure Linux Images with DISTRO_FEATURES

The APSS Linux project, built on Yocto, allows you to create custom Linux images (e.g., xipImage, devkit-e8.dtb, and root filesystem in cramfs-xip or ext4 format) by specifying features in the DISTRO_FEATURES variable. These features determine the functionality included in the kernel and root filesystem, such as support for peripherals like MHU, Ethernet, USB, or SPI. Follow the steps below to set up and build these images on a Linux host or a Linux distro on Windows 11, using a new terminal as a normal user.

6.4.9.1 Setup and Build Process

1. **Clone the Repository**

Clone the APSS Linux build setup and switch to the scarthgap_yocto_5.0 branch:

```
$ git clone https://github.com/AlifSemi/alif_linux-apss-build-setup
-b scarthgap_yocto_5.0
```

Note: Generate a personal access token for GitHub cloning if needed

2. **Install Host Dependencies**

Run the setup script to install required packages on your host machine:

```
$ sudo alif_linux-apss-build-setup/scripts/host_setup.sh
```

3. **Fetch Yocto Layers**

Navigate to the cloned directory and fetch the necessary Yocto and Ensemble BSP layers:

```
$ cd alif_linux-apss-build-setup
$ sh scripts/fetch-layers.sh
```

Note: Use *sh scripts/fetch-layers.sh* update to get the latest layer updates.

11. **Set Up Build Environment**

Initialize the build environment:

```
$ source scripts/setup.sh
```

12. **Configure and Build Images**

Edit conf/local.conf in the build directory to add desired features to DISTRO_FEATURES, then build the images with bitbake. Below are examples for different features:

- **BASE_IMAGE**

```
The default image that will be built includes distro features such as: “apss-sd-share,” “apss-usb,” “apss-i2c,” “apss-spi,” “apss-mhu,” “apss-hwsem,” “apss-debug,” and “apss-crc.”
```

Booting from MRAM: bl32.bin, devkit-e8.dtb, xipImage and cramfs.

RAM: STITCHED (SRAM0 + SRAM1) = 8MB

- **BASE_IMAGE2**

```
The BASE_IMAGE2 includes BASE_IMAGE and additional distro features: “apss-i2s”, “apss-i3c”, “apss-ethernet”, “apss-utimer”, “apss-pdm”.
```

Booting from MRAM: bl32.bin, devkit-e8.dtb, xipImage.

OSPIFLASH: Cramfs

RAM: HYPERRAM = 64MB

- **BASE_IMAGE3**

Same as BASE_IMAGE2 distro features.

Booting from MRAM: bl32.bin, devkit-e8.dtb.

OSPIFLASH: xipImage and Cramfs

RAM: HYPERRAM + STITCHED SRAM = (64 + 8) = 72MB.

- **BASE_IMAGE4**

Same as BASE_IMAGE2 distro features.

Booting from MRAM: bl32.bin, devkit-e8.dtb, xipImage.

SD Card: Root Filesystem (RFS)

RAM: HYPERRAM + STITCHED SRAM = (64 + 8) = 72MB.

- **BASE_IMAGE5**

This feature is similar to BASE_IMAGE2, but it excludes PDM, DAC12, ETHERNET, UTIMER, I2S, and I3C.

Booting from MRAM: bl32.bin, devkit-e8.dtb, xipImage, cramfs.

RAM: HYPERRAM + STITCHED SRAM = (64 + 8) = 72MB.

IMAGE	MRAM	STITCHED SRAM = 8MB	HYPERRAM = 64MB	OSPI FLASH	SD/MMC – (RFS)	Devices/IP	Size xipImage + RFS in MB
TINY_IMAGE	bl32.bin, devkit-e8, xipImage and cramfs.	Yes	No	No	No	UART, RTC, GPIO, MHU, HWSEM	2.3 + 1.8
BASE_IMAGE	bl32.bin, devkit-e8, xipImage and cramfs.	Yes	No	No	No	TINY_IMAGE + SD, USB, SPI, I2C	3.0 + 1.8
BASE_IMAGE2	bl32.bin, devkit-e8, xipImage.	No	Yes	Cramfs	No	BASE_IMAGE + Ethernet, PDM, Utimer, I3C, I2S, DMA+I2S, DAC12	3.9 + 2.9

BASE_IMAGE3	bl32.bin, devkit-e8.	Yes	Yes	xipImage and cramfs	No	BASE_IMAGE2	3.9 + 2.9
BASE_IMAGE4	bl32.bin, devkit-e8, and xipImage.	Yes	Yes	No	RFS – Ext3	BASE_IMAGE2	3.9 + 2.9

Figure 11

Note: To build TINY_IMAGE

```
$ echo 'BASE_IMAGE = "0"' >> conf/local.conf
```

- **Message Handling Unit (MHU)**

```
$ echo 'DISTRO_FEATURES:append = " apss-mhu"' >> conf/local.conf
$ bitbake alif-tiny-image
```

- **Hardware Semaphore (HWSEM)**

```
$ echo 'DISTRO_FEATURES:append = " apss-hwsem"' >> conf/local.conf
$ bitbake alif-tiny-image
```

- **Ethernet**

```
$ echo 'DISTRO_FEATURES:append = " apss-ethernet"' >> conf/local.conf
$ bitbake alif-tiny-image
```

- **Debugging with Ethernet (gdbserver)**

Adds gdbserver for TCP/IP debugging:

```
$ echo 'DISTRO_FEATURES:append = " apss-debug apss-ethernet"' >>
conf/local.conf
$ bitbake alif-tiny-image
```

- **USB**

```
$ echo 'DISTRO_FEATURES:append = " apss-usb"' >> conf/local.conf
$ bitbake linux-alif alif-tiny-image
```

- **SPI**

```
$ echo 'DISTRO_FEATURES:append = " apss-spi"' >> conf/local.conf
```

Then build:

```
$ bitbake alif-tiny-image
```

- **I2C**

```
$ echo 'DISTRO_FEATURES:append = " apss-i2c"' >> conf/local.conf
```

Then build:

```
$ bitbake alif-tiny-image
```

- **CRC Support**

```
$ echo 'DISTRO_FEATURES:append = " apss-crc"' >> conf/local.conf
```

$ bitbake alif-tiny-image

- **CDC-200 Support**

```
$ echo 'DISTRO_FEATURES:append = " apss-cdc200"' >> conf/local.conf
$ bitbake alif-tiny-image
```

- **ADC (Analog-to-Digital Converter)**

  For ADC12:

```
$ echo 'BASE_IMAGE5 = "1"' >> conf/local.conf
$ echo 'ADC_MODE = "adc12"' >> conf/local.conf
$ bitbake alif-tiny-image
```

To switch to ADC24 (after validating ADC12, as they share pin mux):

Edit conf/local.conf to replace adc12 with adc24, then rebuild:

```
$ vim conf/local.conf
$ bitbake -c cleanall linux-alif
$ bitbake linux-alif
$ bitbake alif-tiny-image
```

- **DAC12 (Digital-to-Analog Converter)**

  For DAC12, both instances DAC12_0 and DAC12_1 will be enabled and probed during the kernel image boot.

```
$ echo 'DISTRO_FEATURES:append = " apss-dac12"' >> conf/local.conf
$ bitbake alif-tiny-image
```

- **High-Speed Comparator (CMP0-CMP3)**

All the comparator instances (CMP0 to CMP3) will be enabled and probed during the kernel image boot.

```
$ echo 'DISTRO_FEATURES:append = " apss-cmp"' >> conf/local.conf
$ bitbake alif-tiny-image
```

- **I3C Support**

```
$ echo 'DISTRO_FEATURES:append = " apss-i3c"' >> conf/local.conf
$ bitbake alif-tiny-image
```

To remove any distro features, use the command below.

```
$ echo 'DISTRO_FEATURES:remove = " apss-i3c"' >> conf/local.conf
$ bitbake alif-tiny-image
```

Output

- Generated images *(xipImage, devkit-e8.dtb, cramfs-xip, or ext4)* are stored in: tmp/deploy/images/devkit-e8
- Use the latest built images for development.

Notes

1. **Feature List**: See the Linux Kernel Feature section for all supported DISTRO_FEATURES in the APSS Linux project.
2. **Image Size**: Adding features increases xipImage and cramfs-xip sizes. Adjust XIP_KERNEL_LOAD_ADDR, KERNEL_MTD_START_ADDR, and KERNEL_MTD_LEN in conf/local.conf to fit the memory layout.
3. **Testing**: The alif-tiny-image is a small, bootable Linux distro. Log in as root to test features like HWSEM or SPI.

6.4.10 Build Linux Images for OSPI1 NOR Flash

This section describes how to build the stage 3-2 bootloader (bl32.bin) and configure OSPI1 NOR flash in Execute-in-Place (XiP) mode to boot the Linux kernel (xipImage) and root filesystem (cramfs-xip) from the OSPI1 NOR flash memory starting at address 0xC0000000.

Run the following commands in a new shell terminal as a normal user, either on a Linux host or within a Linux distribution running on Windows 11, to build the required images (devkit-e8.dtb, xipImage, and root filesystem in cramfs-xip or ext4 format).

Procedure

1. **Clone the Repository**

Clone the alif_linux-apss-build-setup repository and check out the scarthgap_yocto_5.0 branch.

```
git clone https://github.com/AlifSemi/alif_linux-apss-build-setup
-b scarthgap_yocto_5.0
```

2. **Install Host Dependencies**

Run the host_setup.sh script to install required packages on the host machine before executing bitbake commands.

```
sudo alif_linux-apss-build-setup/scripts/host_setup.sh
```

3. **Fetch Yocto Layers**

Navigate to the repository directory and fetch the Yocto and Ensemble BSP layers using the fetch-layers.sh script.

```
cd alif_linux-apss-build-setup
sh scripts/fetch-layers.sh
```

Note: To update to the latest layer changes, run *sh scripts/fetch-layers.sh* update.

4. **Set Up Build Environment**

Prepare the environment for building Linux images with Yocto layers.

```
source scripts/setup.sh
```

5. **Reset Configuration (If Needed)**

If conf/local.conf has been modified, remove it to ensure pristine images are built without contamination, then reinitialize the environment.

```
rm conf/local.conf
source scripts/setup.sh
```

13. **Configure and Build Images**

Edit conf/local.conf in the build directory to enable OSPI1 NOR flash booting, then build the images using bitbake. Choose one of the following options:

- **Option 1: Custom Build**

  Set specific boot parameters to load xipImage from 0xC0800000 and mount cramfs-xip from 0xC0000000.

```
echo 'FLASH_EN = "1"' >> conf/local.conf
echo 'KERNEL_MTD_START_ADDR = "0xC0000000"' >> conf/local.conf
echo 'KERNEL_MTD_LEN = "0x400000"' >> conf/local.conf
```

```
echo 'XIP_KERNEL_LOAD_ADDR = "0xC0800000"' >> conf/local.conf
bitbake alif-tiny-image linux-alif
```

- **Option 2: Default OSPI1 Build**

  Use a simpler configuration for OSPI1 booting.

```
echo 'OSPI_BOOT = "1"' >> conf/local.conf
bitbake alif-tiny-image linux-alif
```

14. **Save Generated Images**

    Copy the built images (bl32.bin, devkit-e8.dtb, xipImage, and alif-tiny-image-devkit-e8.cramfs-xip) to a custom directory to avoid overwriting, renaming where necessary.

```
cp tmp/deploy/images/devkit-e8/bl32.bin /home/$USER/app-release-exec-linux/build/images/bl32-ospi1.bin
cp tmp/deploy/images/devkit-e8/devkit-e8.dtb /home/$USER/app-release-exec-linux/build/images/devkit-e8.dtb
cp tmp/deploy/images/devkit-e8/xipImage /home/$USER/app-release-exec-linux/build/images/xipImage-ospi1
cp tmp/deploy/images/devkit-e8/alif-tiny-image-devkit-e8.cramfs-xip /home/$USER/app-release-exec-linux/build/images/alif-tiny-image-devkit-e8.cramfs-xip
```

Note:

The generated devkit-e8.dtb and alif-tiny-image-devkit-e8.cramfs-xip are identical to default images, so renaming isn't required for these.

Limitations

- Booting xipImage directly from the OSPI1 NOR flash address 0xC0000000 is not supported. Use an address starting at 0xC0800000 or higher instead.

Additional Notes

1. **Output Location**: Each bitbake command generates xipImage, devkit-e8.dtb, and cramfs-xip in the tmp/deploy/images/devkit-e8 directory. Use the most recent images for development.
2. **Programming Instructions**: See section *Programming bl32.bin, devkit-e8.dtb, and xipImage into MRAM Using Alif Security Toolkit* (SETOOLS)

Procedure

Follow the steps below to program the MRAM with the specified Linux images and enable the DevKit to mount the root filesystem (rootfs) from an SD card.

Program **MRAM with ATOC**

15. **Use the Alif Security** Toolkit (SETOOLS) to program the MRAM with an ATOC (Application Table of Contents) that includes the Linux images (bl32.bin, devkit-e8.dtb, and xipImage). This configuration will enable mounting the rootfs from the SD card.

Configuration File: Utilize the JSON file located at:

16. /home/$USER/APSS-v2.1.0-Beta/config-files/APSS-tiny-linux-sd-boot.json

Instructions: For detailed steps on programming the MRAM with the ATOC, refer to the section 4.1.1.

Power Cycle the DevKit Board

After programming the MRAM, power off and then power on the DevKit board to initiate the boot process.

- **Boot and** Login
- The Linux kernel will boot, mount the rootfs from the SD card, and present a login prompt. Log in using the username root.

Programming Images for Verification of Hyper RAM Functionality

- **This** section explains how to program Linux images to boot from OSPI1 NOR flash and verify *Hyper* RAM functionality at address 0xA0000000 on the Alif DevKit. After booting, you'll use the hyperram_test program to confirm Hyper RAM data allocation.

17. Prerequisites

18. **Complete the steps in** section 6.4.13 to generate the required images (bl32.bin, devkit-e8.dtb, xipImage, alif-tiny-image-devkit-e8.cramfs-xip).

Programming and Verification Steps

19. Follow these steps to program the images and verify Hyper RAM functionality:

Navigate to the **Alif Security Toolkit Directory**

20. **Open a terminal** and change to the directory containing the Alif Security Toolkit tools (app-gen-toc, app-write-mram, updateSystemPackage). This example assumes the tools are at *~/app-release-exec-linux:*
21. cd ~/app-release-exec-linux
22. ls
23. 

    Create an Application Image (ATOC)

    Use the app-gen-toc script with the APSS-tiny-linux.json configuration file to generate an Alif Application Table of Contents (ATOC):

6.4.11 ./app-gen-toc -f build/config/APSS-tiny-linux.json

6.4.12 Program the MRAM with the ATOC

Run app-write-mram with sudo privileges to program AppTocPackage.bin into the MRAM via the In-System Programmer (ISP):

sudo ./app-write-mram -p

Syntax Explanation: The -p option ensures 16-byte alignment by adding padding bytes.

Note: The SE-UART is assumed to be /dev/ttyACM0. Use dmesg (e.g., dmesg | grep ttyACM) to identify the correct device node if different.

6.4.12.1 Boot Linux

- After programming, the DevKit boots *Linux* using xipImage and cramfs-xip from OSPI1 NOR flash:
- Log in with the username root (no password required by default).

6.4.12.2 Run the Hyper RAM Test

Execute the hyperram_test program on the target to allocate and verify data in Hyper RAM:

24. **Step 1: Run the test to** allocate 4 KB of dynamic memory and write 0xA5 across 32 MB (8192 iterations × 4 KB):
25. hyperram_test

Step 2: Verify the data at Hyper RAM addresses (starting at 0xA0000000 or decimal 2684354560):

26. for iter in `seq 8192`; do *echo* $iter; echo -n "Data at address `expr 2684354560 + $iter * 4096` is "; devmem `expr 2684354560 + $iter * 4096`; done | grep 0xA5A5A5A5

Sample Output

```
# hyperram_test
HyperRAM test that writes 0xA5 data of 4K bytes for 800 iterations.
Count 0: Wrote at virtual address 0x477010
Count 1: Wrote at virtual address 0x478020
Count 2: Wrote at virtual address 0x479030
Count 3: Wrote at virtual address 0x47a040
Count 4: Wrote at virtual address 0x47b050
Count 5: Wrote at virtual address 0x47c060
Count 6: Wrote at virtual address 0x47d070
Count 7: Wrote at virtual address 0x47e080
Count 8: Wrote at virtual address 0x47f090
Count 9: Wrote at virtual address 0x4800a0
Count 10: Wrote at virtual address 0x4810b0
Count 11: Wrote at virtual address 0x4820c0
Count 12: Wrote at virtual address 0x4830d0
```

```
# for iter in `seq 8192` ; do echo $iter ; echo -n "Data
at address `expr 2684354560 + $iter * 4096` is "; devmem `expr 268435456
0 + $iter * 4096` ; done | grep 0xA5A5A5A5
Data at address 2684358656 is 0xA5A5A5A5
Data at address 2684362752 is 0xA5A5A5A5
Data at address 2684366848 is 0xA5A5A5A5
Data at address 2684370944 is 0xA5A5A5A5
Data at address 2684375040 is 0xA5A5A5A5
Data at address 2684379136 is 0xA5A5A5A5
Data at address 2684383232 is 0xA5A5A5A5
Data at address 2684387328 is 0xA5A5A5A5
Data at address 2684391424 is 0xA5A5A5A5
Data at address 2684395520 is 0xA5A5A5A5
Data at address 2684399616 is 0xA5A5A5A5
Data at address 2684403712 is 0xA5A5A5A5
```

Notes

27. **Hyper RAM Address: 2684354560 is** the decimal representation of 0xA0000000, the base address of Hyper RAM.

28.

3. Program xipImage and cramfs-xip to OSPI1 Flash Using the PC Tool for details on programming xipImage and cramfs-xip into OSPI1 NOR flash.

4. **Image Size Considerations**: If additional DISTRO_FEATURES are included (as in section 11.9), the sizes of xipImage and cramfs-xip increase. Adjust XIP_KERNEL_LOAD_ADDR, KERNEL_MTD_START_ADDR, and KERNEL_MTD_LEN to ensure the images fit the allocated memory space.

6.4.13 Building 16-Bytes AES Encrypted Linux Images to Boot from OSPI1 NOR Flash

This section guides you through building a stage 3-2 bootloader (bl32.bin) and 16-byte AES encrypted Linux images (xipImage and cramfs-xip) to boot from OSPI1 NOR flash on the Alif DevKit. These steps

generate encrypted devkit-e8.dtb, xipImage, and cramfs-xip (in ext4 format) images, configured to boot from address 0xC0000000 using the Yocto build system.

6.4.13.1 Build Steps

Follow these steps to build the encrypted Linux images:

1. **Clone the Repository**

   Clone the alif_linux-apss-build-setup repository and switch to the scarthgap_yocto_5.0 branch:

```
git clone https://github.com/AlifSemi/alif_linux-apss-build-setup -b
scarthgap_yocto_5.0
```

2. **Install Required Packages**

   Run the host_setup.sh script with sudo privileges to install dependencies on the host machine:

```
sudo alif_linux-apss-build-setup/scripts/host_setup.sh
```

3. Fetch Yocto Layers.

Navigate to the repository directory and fetch the Yocto and Ensemble BSP layers:

```
cd alif_linux-apss-build-setup
sh scripts/fetch-layers.sh
```

Note:

To update to the latest changes, run sh scripts/fetch-layers.sh update.

---

29. **Set Up the Build Environment**

Source the setup.sh script to prepare the Yocto build environment:

```
source scripts/setup.sh
```

30. **Reset Configuration (Optional)**

If conf/local.conf has been modified, remove it to ensure a clean build, then re-run the setup script:

```
rm conf/local.conf
source scripts/setup.sh
```

31. **Configure for OSPI1 NOR Flash with AES Encryption**

Edit conf/local.conf in the build directory to enable AES encryption and set boot addresses:

```
echo 'BASE_IMAGE3 = "1"' >> conf/local.conf
```

```
echo 'AES_EN = "1"' >> conf/local.conf
echo 'AES_ENC_KEY = "0123456789ABCDEF"' >> conf/local.conf
```

- AES_EN = "1": Enables AES encryption.
- AES_ENC_KEY: Specifies a 16-byte AES key (e.g., "0123456789ABCDEF").

32. **Build the Images**

Run Bitbake to generate the encrypted bootloader and Linux images:

```
bitbake alif-tiny-image linux-alif
```

33. **Save Generated Images**

Copy the built images from *tmp/deploy/images/devkit-e8/* to a custom directory (*e.g., /home/$USER/app-release-exec-linux/build/images*) with unique names to avoid overwriting:

bash

```
cp tmp/deploy/images/devkit-e8/bl32.bin /home/$USER/app-release-exec-linux/build/images/bl32-ospi1-en.bin
cp tmp/deploy/images/devkit-e8/devkit-e8.dtb /home/$USER/app-release-exec-linux/build/images/devkit-e8.dtb
cp tmp/deploy/images/devkit-e8/xipImage /home/$USER/app-release-exec-linux/build/images/xipImage-ospi1-en
cp tmp/deploy/images/devkit-e8/alif-tiny-image-devkit-e8.cramfs-xip /home/$USER/app-release-exec-linux/build/images/alif-tiny-image-devkit-e8-ospi1-en.cramfs-xip
```

Note: The devkit-e8.dtb file is identical to the default image, so renaming is optional.

6.4.13.2 Limitation

- **Booting Issue**: Booting xipImage directly from 0xC0000000 on OSPI1 NOR flash does not work. Use an address starting at 0xC0800000 instead (as configured above).

Notes

- **Image Location**: Each Bitbake command generates xipImage, devkit-e8.dtb, and cramfs-xip in tmp/deploy/images/devkit-e8/. Use the latest images for development.
- **Programming**: Refer to section 6 of the original documentation for instructions on programming xipImage and cramfs-xip into OSPI1 NOR flash.

- **AES Encryption**: When AES_EN = "1" and AES_ENC_KEY is set, the bootloader (bl32.bin) registers the AES key with the AES decoder, decrypting xipImage and cramfs-xip on-the-fly from OSPI1 flash. Ensure XIP_KERNEL_LOAD_ADDR and KERNEL_MTD_START_ADDR match the flash layout.
- **Image Size**: Enabling AES encryption and additional DISTRO_FEATURES increases image sizes. Adjust XIP_KERNEL_LOAD_ADDR, KERNEL_MTD_START_ADDR, and KERNEL_MTD_LEN to fit the address space (see *Configuration Parameters* in the original documentation for details).

6.4.14 Building Linux Images with Hyper RAM Functionality

This section guides you through building Linux images, including a stage 3-2 bootloader (bl32.bin), to enable Hyper RAM functionality at address 0xA0000000 on the Alif DevKit. These images configure Hyper RAM as data space for user-space applications and include devkit-e8.dtb, xipImage, and cramfs-xip (in ext4 format).

Optional: Pre-built images (bl32.bin, devkit-e8.dtb, xipImage, alif-tiny-image-devkit-e8.cramfs-xip) can be used directly to verify Hyper RAM functionality without building.

6.4.14.1 Build Steps

Follow these steps to build the Linux images with Hyper RAM support:

1. **Clone the Repository**

Clone the alif_linux-apss-build-setup repository and switch to the scarthgap_yocto_5.0 branch:

```
git clone https://github.com/AlifSemi/alif_linux-apss-build-setup -b
scarthgap_yocto_5.0
```

2. **Install Required Packages**

Run the host_setup.sh script with sudo privileges to install dependencies on the host machine:

```
sudo alif_linux-apss-build-setup/scripts/host_setup.sh
```

3. **Fetch Yocto Layers**

Navigate to the repository directory and fetch the Yocto and Ensemble BSP layers:

```
cd alif_linux-apss-build-setup
sh scripts/fetch-layers.sh
```

Note:

To update to the latest changes, run sh scripts/fetch-layers.sh update.

---

4. **Set Up the Build Environment**

Source the setup.sh script to prepare the Yocto build environment:

```
source scripts/setup.sh
```

5. **Reset Configuration (Optional)**

If conf/local.conf has been modified, remove it to ensure a clean build, then re-run the setup script:

```
rm conf/local.conf
source scripts/setup.sh
```

6. **Build the Images**

Run Bitbake to generate the bootloader, device tree, kernel image, and root filesystem:

```
bitbake linux-alif alif-tiny-image
```

7. **Save Generated Images**

Copy the built images from *tmp/deploy/images/devkit-e8/* to a custom directory (e.g., */home/$USER/app-release-exec-linux/build/images*):

```
cp tmp/deploy/images/devkit-e8/bl32.bin /home/$USER/app-release-exec-
linux/build/images/bl32.bin
cp tmp/deploy/images/devkit-e8/devkit-e8.dtb /home/$USER/app-release-exec-
linux/build/images/devkit-e8.dtb
cp tmp/deploy/images/devkit-e8/xipImage /home/$USER/app-release-exec-
linux/build/images/xipImage
cp tmp/deploy/images/devkit-e8/alif-tiny-image-devkit-e8.cramfs-xip
/home/$USER/app-release-exec-linux/build/images/alif-tiny-image-devkit-
e8.cramfs-xip
```

Notes

- **Image Location**: Bitbake generates xipImage, devkit-e8.dtb, and cramfs-xip in tmp/deploy/images/devkit-e8/. Use the latest images for development.
- **Hyper RAM Configuration**: The bl32.bin bootloader configures Hyper RAM at 0xA0000000 as data space for user applications.
- **Verification**: To test Hyper RAM functionality, program the images and run the hyperram_test program. Refer to the section *Programming Images for Verification of Hyper RAM Functionality* in the original documentation for details.

- These images are identical to default builds, so renaming (e.g., appending -hyperram) is optional.

6.4.15 Building Linux Images to Mount rootfs from an SD Card

This chapter describes how to build and configure Linux images (bl32.bin, devkit-e8.dtb, xipImage, and an ext4 rootfs) to boot the APSS and mount the root filesystem from an SD card.

6.4.15.1 Building Linux Kernel, Device Tree, and rootfs to Mount from SD Card

This section provides instructions to build the stage 3-2 bootloader (bl32.bin), Linux device tree blob (devkit-e8.dtb), Linux kernel (xipImage), and root filesystem (rootfs) in ext4 format for mounting from an SD card. Execute the following commands in a new shell terminal as a normal user on a Linux host or within a Linux distribution running on Windows 11.

Procedure

1. **Clone the Repository**

Clone the alif_linux-apss-build-setup repository and checkout the scarthgap_yocto_5.0 branch.

Command:

```
$ git clone https://github.com/AlifSemi/alif_linux-apss-build-setup -b
scarthgap_yocto_5.0
```

2. **Install Required Packages**

Run the host_setup.sh script with elevated privileges to install necessary packages for building Linux images on the host machine.

Command:

```
$ sudo alif_linux-apss-build-setup/scripts/host_setup.sh
```

3. **Fetch Yocto and BSP Layers**

Navigate to the repository directory and fetch the Yocto layers and Ensemble BSP layers using the fetch-layers.sh script.

Commands:

```
$ cd alif_linux-apss-build-setup
$ sh scripts/fetch-layers.sh
```

Note: To update to the latest changes, run:

```
$ sh scripts/fetch-layers.sh update
```

4. Set Up Build Environment

Source the setup.sh script to prepare the environment for building Linux images with Yocto layers.

Command:

```
$ source scripts/setup.sh
```

5. **Reset Configuration (If Modified)**

If the conf/local.conf file has been altered, remove it to ensure pristine images are built and re-run the setup script.

Commands:

```
$ rm conf/local.conf
$ source scripts/setup.sh
```

6. **Update UART Console Configuration**

Modify the sd_boot.cfg file to change the console from ttyS1 to ttyS0 for Linux boot messages on the UART2 console.

File Location:

```
/home/$USER/ensemble-apss-v2.1.0-202401111408/apss-linux-project/layers/meta-alif/recipes-kernel/linux/files/apss-tiny/sd_boot.cfg
```

Original Line:

```
CONFIG_CMDLINE="console=ttyS1,115200n8 root=/dev/mmcblk0p1 rootfstype=ext4 rootwait rw loglevel=9"
```

Modified Line:

```
CONFIG_CMDLINE="console=ttyS0,115200n8 root=/dev/mmcblk0p1 rootfstype=ext4 rootwait rw loglevel=9"
```

7. **Append DISTRO_FEATURES**

Modify the conf/local.conf file in the build directory to include the apss-sd-boot feature.

Command:

```
$ echo 'DISTRO_FEATURES_append += "apss-sd-boot"' >> conf/local.conf
```

8. **Build Linux Images**

Execute the bitbake command to generate the Linux kernel, device tree, and rootfs in ext4 format.

Command:

```
$ bitbake linux-alif alif-tiny-image
```

Note: Ignore the following warning about cramfs-xip exceeding the MTD partition size, as ext4 is used for SD mounting:

```
NOTE: Executing Tasks
```

```
NOTE: Setscene tasks completed
WARNING: alif-tiny-image-1.0-r1 do_image_cramfs_xip: The MTD partition size
is less than alif-tiny-image-devkit-e8-20240111125338.rootfs.cramfs-xip size.
Set KERNEL_MTD_LEN with value morethan 0x200000 in conf/local.conf to mount
cramfs-xip successfully.
NOTE: Tasks Summary: Attempted 1282 tasks of which 1194 didn't need to be
rerun and all succeeded.
```

9. **Save Generated Images**

Copy the generated bl32.bin, devkit-e8.dtb, xipImage, and alif-tiny-image-devkit-e8.ext4 files to a designated directory to avoid overwriting.

Commands:

```
$ cp tmp/deploy/images/devkit-e8/bl32.bin /home/$USER/app-release-exec-
linux/build/images/bl32.bin
$ cp tmp/deploy/images/devkit-e8/devkit-e8.dtb /home/$USER/app-release-exec-
linux/build/images/devkit-e8.dtb
$ cp tmp/deploy/images/devkit-e8/xipImage /home/$USER/app-release-exec-
linux/build/images/xipImage-sd-boot
$ cp tmp/deploy/images/devkit-e8/alif-tiny-image-devkit-e8.ext4
/home/$USER/app-release-exec-linux/build/images/alif-tiny-image-devkit-
e8.ext4
```

Note: Each bitbake command generates xipImage, devkit-e8.dtb, and the ext4 rootfs in the *tmp/deploy/images/devkit-e8* directory. Use the latest built images for development.

7. Development and Debugging

7.1 Software Development Tools of the APSS Linux Project

The table below outlines the Linux kernel, software development tools, and runtime components used in the APSS Linux Project. These tools and components are essential for:

- Building the Linux kernel, system libraries, applications, and a root filesystem (rootfs) in cramfs-xip format for an embedded Linux application or a tiny Linux distribution.
- Running on the APSS, including the Linux kernel (xipImage), device tree binary (devkit-e8.dtb), trusted firmware (bl32.bin), and rootfs with the embedded application or tiny distribution.

Software	Version	Description
Yocto Project	5.0 (Scarthgap)	An open-source collaboration project that enables developers to create custom Linux-based systems for any hardware architecture. The APSS Linux Project uses it to build the Linux kernel (xipImage), applications, and rootfs.
Linux Kernel	6.12	A free, open-source, monolithic, modular, multitasking, Unix-like operating system kernel. Serves as the OS kernel for the APSS Linux Project and is used to build xipImage.
GCC (GNU Compiler Collection)	13.3.x	An optimizing compiler from the GNU Project, supporting multiple languages, architectures, and operating systems. Used to compile the Linux kernel and standard system libraries.
Binutils (GNU Binary Utilities)	2.42.0	A set of tools for creating and managing binary programs, object files, libraries, profile data, and assembly source code.
musl libc	1.2.4	A lightweight C standard library for Linux-based systems, offering advantages in size, correctness, static linking, and clean code over alternatives like uClibc.
Trusted Firmware-A (TF-A)	LTS-v2.10.8	A reference implementation of secure world software for Armv8-A and Armv7-A architectures, including an Exception Level 3 (EL3) Secure Monitor. Used to build the stage 3-2 secure bootloader (bl32.bin), which runs early in the A32 boot process, passing control to the stage 3-3 binary (xipImage).

7.2 Application Execution Flow on the APSS

The diagram below illustrates the execution flow of an embedded Linux application on the Application Processor Subsystem (APSS). In this process:

- The stage 3-2 bootloader (bl32.bin) runs directly on the APSS, launching the Linux kernel (xipImage) in Execute-in-Place (XiP) mode.
- The Linux kernel initializes RAM caches and hardware devices, mounts the root filesystem (rootfs), and starts /sbin/init as the first user-space process.
- Applications in the rootfs use the musl standard C library, which provides macros, type definitions, and functions for tasks like string handling, math computations, input/output, memory management, and other OS services. These applications call high-level library functions, which trigger system calls. The Linux kernel handles these via a system call interface, servicing the resulting software interrupts.

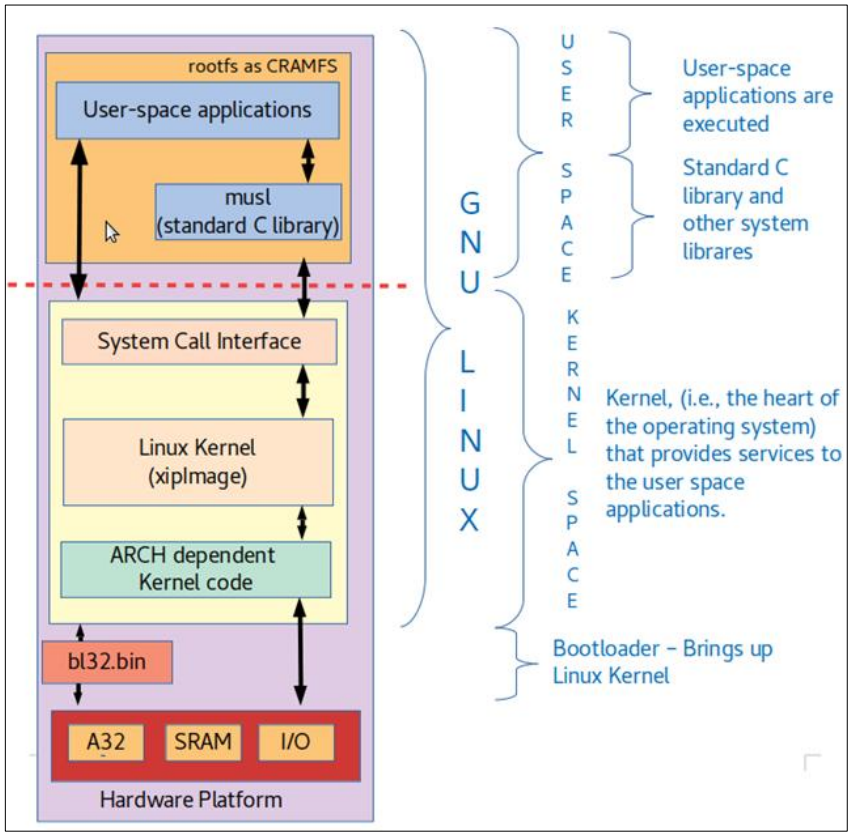


Figure 2: Linux Bootup and Application Execution Flow on the APSS

7.3 Debugging User Applications Using GDB Server Over Serial Cable

This chapter explains how to debug Linux user-space applications on the Alif DevKit's APSS (Application Processor Subsystem) using the GNU Debugger (GDB) over a serial cable connection.

7.3.1 Cross-Compiling and Debugging a Linux User-Space Application in cramfs-xip (rootfs) Using GDB via Serial Cable

This section describes how to use GDB to debug user-space applications embedded in the cramfs-xip root filesystem on the APSS. The process uses GDBserver on the DevKit (target) and a GDB client on a host machine (Linux or Windows 11 with a Linux distro) connected via UART4.

- **GDBserver (DevKit)**: Runs on the target, listening on a serial device (*e.g., /dev/ttyS1*), and executes the application.
- **GDB (Host)**: Runs on the host, connecting to GDBserver via a USB serial port (*e.g., /dev/ttyUSBx),* using an unstripped executable with debug symbols.

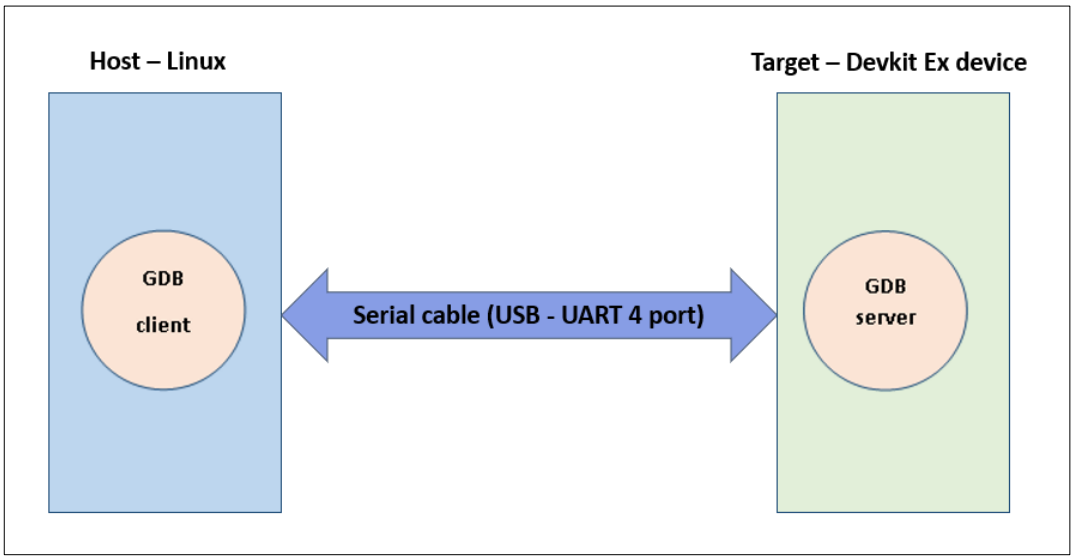


Figure 3: Remote Debugging using GDB Client-Server via a Serial Connection

7.3.1.1 Prerequisites

Ensure the following are in place before debugging:

1. Build SDK Toolchain Installer Using Bitbake :
    - This provides a cross-compilation toolchain for building applications.

- Optional, if a pre-built SDK toolchain installer is provided or available.

2. Build User-Space Embedded Application.
   - Creates the application to be debugged.
   - Compile the application with debug symbols for GDB compatibility.
3. Build Images with Serial Support:
   - Generates Linux images with UART4 support and embeds the application in the cramfs-xip root filesystem.
   - Ensures the DevKit can communicate with the host over the serial cable.
4. Program Images to MRAM
   - Loads the bootloader (bl32.bin), device tree (devkit-e8.dtb), kernel (xipImage), and root filesystem (cramfs-xip) into MRAM.
   - Uses the Alif Security Toolkit (SETOOLS) for programming.
5. Boot the APSS with Tiny Linux Distribution
   - Starts the DevKit with a minimal Linux environment supporting serial debugging.
   - Requires the programmed images to be operational.

7.3.1.2 SDK Toolchain Installer Using Bitbake

Build a cross-compilation toolchain for the DevKit:

1. Clone the Repository

```
git clone https://github.com/AlifSemi/alif_linux-apss-build-setup -b
scarthgap_yocto_5.0
```

2. Install Host Packages

```
sudo alif_linux-apss-build-setup/scripts/host_setup.sh
```

3. Fetch Layers

```
cd alif_linux-apss-build-setup
sh scripts/fetch-layers.sh
```

Note:

Run sh scripts/fetch-layers.sh update for the latest updates.

---

4. Set Up the Build Environment

```
source scripts/setup.sh
```

5. **Reset Configuration (Optional)**

If conf/local.conf is modified:

```
rm conf/local.conf
source scripts/setup.sh
```

6. **Build the SDK**

```
bitbake -c populate_sdk alif-tiny-image
```

7. **Locate the Installer**

Find the toolchain installer at:

tmp/deploy/sdk/apss-tiny-musl-x86_64-alif-tiny-image-cortexa32hf-neon-devkit-e8-toolchain-0.1.sh

7.3.1.3 Build User-Space Embedded Application

Compile a sample application (helloworld) for debugging:

1. **Open a New Shell**

Open a terminal on the host.

2. **Install the Toolchain**

Install the SDK to the default location (/opt/apss-tiny/0.1):

```
sh apss-tiny-musl-x86_64-alif-tiny-image-cortexa32hf-neon-devkit-e8-toolchain-0.1.sh
```

```
$ sh apss-tiny-musl-x86_64-alif-tiny-image-cortexa32hf-neon-devkit-e7-toolchain-0.1.sh
Alif Iota - Tiny Linux Distribution SDK installer version 0.1
==========================================================
Enter target directory for SDK (default: /opt/apss-tiny/0.1):
The directory "/opt/apss-tiny/0.1" already contains a SDK for this architecture.
If you continue, existing files will be overwritten! Proceed [y/N]? y
Extracting SDK.....................................done
Setting it up...done
SDK has been successfully set up and is ready to be used.
Each time you wish to use the SDK in a new shell session, you need to source the environment setup script e.g.
 $ . /opt/apss-tiny/0.1/environment-setup-cortexa32hf-neon-poky-linux-musleabi
```

3. **Source the Environment**

```
source /opt/apss-tiny/0.1/environment-setup-cortexa32hf-neon-poky-linux-musleabi
```

```
$ sh apss-tiny-musl-x86_64-alif-tiny-image-cortexa32hf-neon-devkit-e7-toolchain-0.1.sh
Alif Iota - Tiny Linux Distribution SDK installer version 0.1
=============================================================
Enter target directory for SDK (default: /opt/apss-tiny/0.1):
The directory "/opt/apss-tiny/0.1" already contains a SDK for this architecture.
If you continue, existing files will be overwritten! Proceed [y/N]? y
Extracting SDK.....................................done
Setting it up...done
SDK has been successfully set up and is ready to be used.
Each time you wish to use the SDK in a new shell session, you need to source the environment setup script e.g.
 $ . /opt/apss-tiny/0.1/environment-setup-cortexa32hf-neon-poky-linux-musleabi
 $ source /opt/apss-tiny/0.1/environment-setup-cortexa32hf-neon-poky-linux-musleabi
```

4. Create a Project Directory

```
mkdir -p /home/$USER/projects
cd /home/$USER/projects
```

5. Create helloworld.c

Save the following code as helloworld.c:

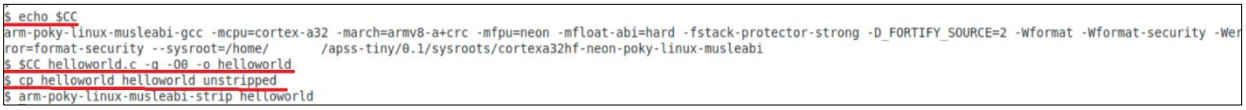
```
#include <stdio.h>
#include <unistd.h>
int main() {
    for(;;) {
        printf("Linux Hello World Example 1\n");
        sleep(1);
    }
}
```

```
$ cat helloworld.c
#include <stdio.h>
#include <unistd.h>

int main() {

    for(;;) {
    printf("Linux Hello World Example 1\n");
    sleep(1);
    }
        return 0;
}
$
```

6. Compile the Application with Debug Flags

Check the cross-compiler and compile the embedded application with debug support:

```
echo $CC
arm-poky-linux-musleabi-gcc -mcpu=cortex-a32 -march=armv8-a+crc -mfpu=neon -
mfloat-abi=hard -fstack-protector-strong -D_FORTIFY_SOURCE=2 -Wformat -
Wformat-security -Werror=format-security--sysroot=/var/lib
enkinss/DevKit_E8_APSS_Linux_Project/build-scripts/opt/apss-
tiny/0.1/sysroots/cortexa32hf-neon-poky-linux-musleabi helloworld.c -g -O0 -o
helloworld
```



7. **Backup and Strip the Binary**

```
cp helloworld helloworld_unstripped
arm-poky-linux-musleabi-strip helloworld
```

7.3.1.4 Build Images with Serial Support for Debugging over Serial Cable and Include User-space Embedded Application into the rootfs (cramfs-xip) Image

This section explains how to build Linux images with serial support for debugging over a serial cable and embed a user-space application (e.g., helloworld) into the cramfs-xip root filesystem.

Run these commands as a normal user in a new shell terminal on a Linux host or a Linux distribution on Windows 11.

Build Steps

Follow these steps to create the images:

1. **Clone the Repository**

Clone the alif_linux-apss-build-setup repository and checkout the scarthgap_yocto_5.0 branch:

```
git clone https://github.com/AlifSemi/alif_linux-apss-build-setup
-b scarthgap_yocto_5.0
```

2. **Install Required Packages**

Run the host_setup.sh script with sudo to install dependencies:

```
sudo alif_linux-apss-build-setup/scripts/host_setup.sh
```

3. **Fetch Yocto Layers**

Navigate to the repository directory and fetch the Yocto and Ensemble BSP layers:

```
cd alif_linux-apss-build-setup
sh scripts/fetch-layers.sh
```

Note:

Run sh scripts/fetch-layers.sh update to get the latest changes.

---

4. **Set Up the Build Environment**

Source the setup.sh script to prepare the Yocto build environment:

```
source scripts/setup.sh
```

5. **Reset Configuration (Optional)**

If the conf/local.conf file has been modified, remove it to ensure a clean build, and then re-run the setup script:

```
rm conf/local.conf
source scripts/setup.sh
```

6. **Enable Serial Debugging**

To enable serial debugging support, add "apss-debug" to the DISTRO_FEATURES in the conf/local.conf file:

```
echo 'DISTRO_FEATURES_append += "apss-debug"' >> conf/local.conf
```

7. **Build Firmware and Kernel**

Forcefully compile and build the trusted firmware and Linux kernel:

```
bitbake -c compile -f trusted-firmware-a linux-alif
bitbake trusted-firmware-a linux-alif
```

8. **Copy the Application to rootfs**

Generate the root filesystem and copy the pre-built helloworld application into it:

```
bitbake -c rootfs -f alif-tiny-image
cp /home/$USER/projects/helloworld tmp/work/devkit_e8-poky-linux-
musleabi/alif-tiny-image/2.0-r1/rootfs/
```

9. **Build the Root Filesystem**

Build the cramfs-xip root filesystem containing the helloworld application:

```
bitbake alif-tiny-image
```

10. **Save Generated Images**

Copy the built images to a custom directory (*e.g., /home/$USER/app-release-exec-linux/build/images*) with unique names to avoid overwriting:

```
cp tmp/deploy/images/devkit-e8/bl32.bin /home/$USER/app-release-exec-
linux/build/images/bl32.bin
cp tmp/deploy/images/devkit-e8/devkit-e8.dtb /home/$USER/app-release-exec-
linux/build/images/devkit-e8-gdb.dtb
cp tmp/deploy/images/devkit-e8/xipImage /home/$USER/app-release-exec-
linux/build/images/xipImage
cp tmp/deploy/images/devkit-e8/alif-tiny-image-devkit-e8.cramfs-xip
/home/$USER/app-release-exec-linux/build/images/alif-tiny-image-devkit-e8-
gdb.cramfs-xip
```

Notes

Image Naming: The generated bl32.bin and xipImage images are functionally identical to the default images, so using default names (e.g., bl32.bin, xipImage) is adequate unless tracking specific builds.

7.3.1.5 Programming bl32.bin, devkit-e8.dtb, xipImage, and rootfs in cramfs-xip Format into MRAM Using Alif Security Toolkit (SETOOLS)

Procedure

1. **Programming MRAM with ATOC Containing Linux Images**

Use the Alif Security Toolkit (SETOOLS) to program the MRAM with the ATOC that includes Linux images supporting TCP and a GDB server. Follow these steps:

- Utilize the configuration file located at */home/$USER/APSS-v2.1.0-Beta/config-files/APSS-tiny-linux-gdb-serial.json* to generate the ATOC.
- For detailed instructions on programming the MRAM with the ATOC, refer to the section titled 4.1.1

7.3.2 GDB Client-Server Debug over Ethernet

7.3.2.1 At Devkit Device

Procedure

1. **Configure Network Interface**

Execute the following commands to assign an IP address and activate the eth0 interface:

```
$ ifconfig eth0 <IP_ADDR> netmask <NETMASK> up
$ route add default gw <GATEWAY> eth0
```

Note:

Replace *<IP_ADDR>, <NETMASK>,* and *<GATEWAY>* with the appropriate values for your network configuration.

Example:

```
$ ifconfig eth0 10.10.4.13 netmask 255.255.255.0 up
$ route add default gw 10.10.4.1 eth0
```

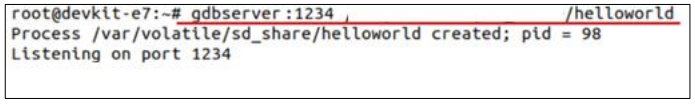
2. **Launch gdbserver**

Start the gdbserver on the Devkit device with a specified port number (prefixed with a colon, e.g.,

:1234) and the user-space embedded application to debug (e.g., helloworld).

Command:

gdbserver :1234 /helloworld



3. **Wait for GDB Client Connection**

   The gdbserver will pause and wait for a connection from the GDB client running on the host machine.

7.3.2.2 At Linux Host

Procedure

1. **Open Terminal and Navigate to Project Directory**

Open a shell terminal window and change to your project directory. For this example, the directory is */home/$USER/projects*.

2. Verify SDK Installation

   Ensure the SDK is installed as outlined in previous sections. In this example, the SDK is installed at */opt/apss-tiny/0.1.*

3. **Source the Environment Setup Script**

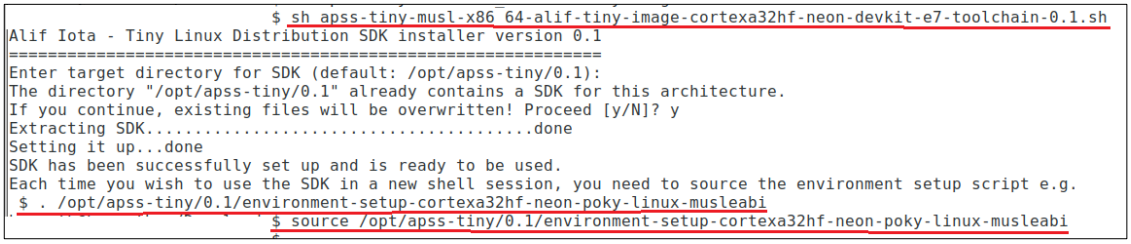
Source the environment-setup-cortexa32hf-neon-poky-linux-musleabi script to configure the environment.

Command:

```
$ source ./environment-setup-cortexa32hf-neon-poky-linux-musleabi
```

Example:

```
$ source /opt/apss-tiny/0.1/environment-setup-cortexa32hf-neon-poky-linux-musleabi
```



4. **Launch Cross-GDB**

Run the cross-GDB tool (arm-poky-linux-musleabi-gdb) on the Linux host or a Linux distribution running on Windows 11.

Example:

```
$ arm-poky-linux-musleabi-gdb
```

5. **Configure GDB and Connect to Remote Target**

Inside the GDB prompt, configure the sysroot, target description (tdesc), and connect to the Devkit device over Ethernet using the target remote command.

Commands:

```
gdb) set tdesc filename /PATH/TO/sysroot/x86_64-pokysdk-
linux/usr/share/gdb/features/arm-vfpv3.xml
(gdb) set sysroot /PATH/TO/sysroot/cortexa32hf-neon-poky-linux-musleabi
(gdb) target remote <Devkit_IP_address>:<Port number>
```

Notes:

- Replace /PATH/TO/sysroot/ with the actual sysroot path.
- Replace <Devkit_IP_address> with the Devkit's IP address (e.g., 10.10.4.13).
- Replace <Port number> with the port used by gdbserver (e.g., 1234).

Example:

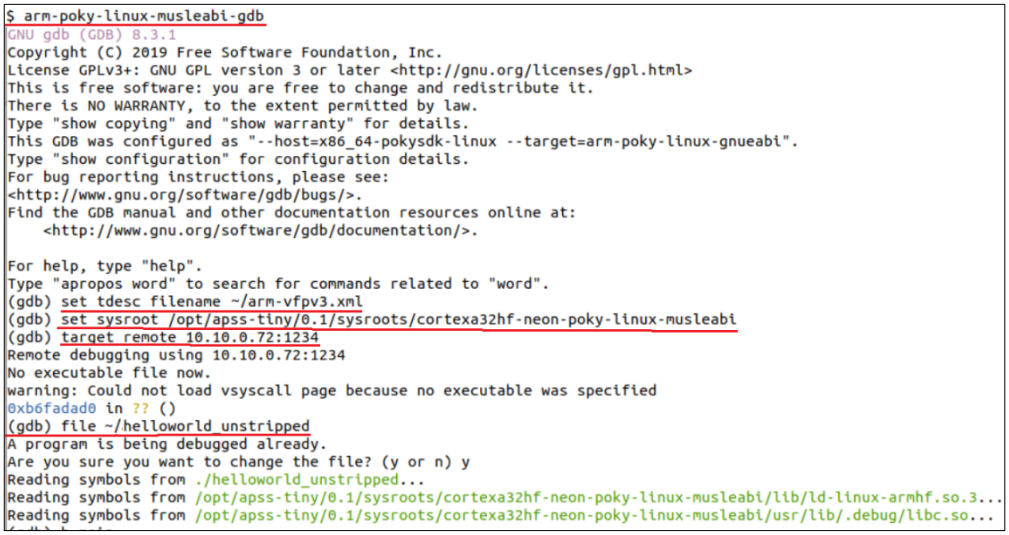
```
(gdb) set tdesc filename /opt/apss-tiny/0.1/sysroots/x86_64-pokysdk-
linux/usr/share/gdb/features/arm-vfpv3.xml
(gdb) set sysroot /opt/apss-tiny/0.1/sysroots/cortexa32hf-neon-poky-linux-
musleabi
(gdb) target remote 10.10.4.13:1234
```

6. **Load the Application**

Load the unstripped version of the Linux user-space application (e.g., helloworld_unstripped) using the file command.

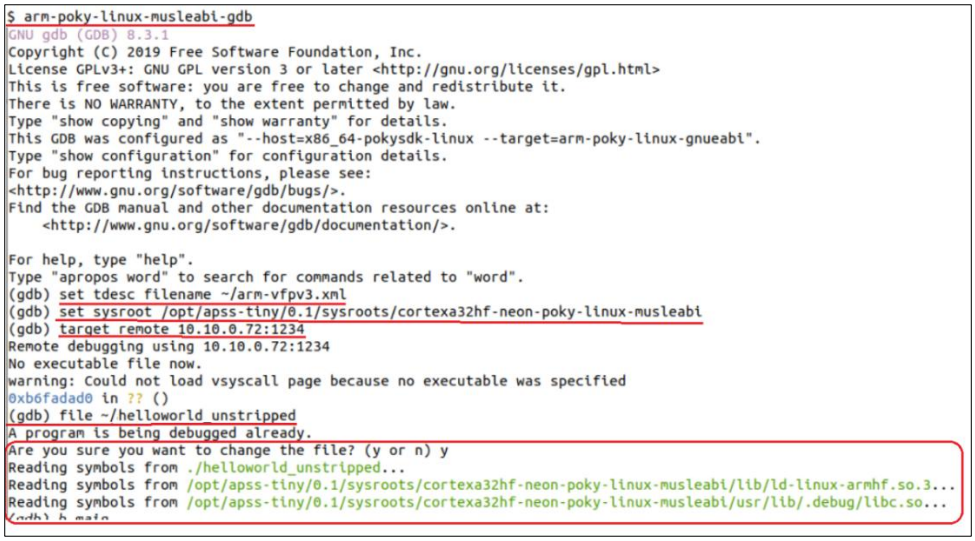
Command:

```
(gdb) file ~/projects/helloworld_unstripped
```



8. **Verify Connection**

   After a successful connection, confirmation messages will appear in the GDB prompt.



9. **Set Breakpoint and Start Debugging**

   Set a breakpoint at the main function and continue execution.

   Commands:

```
(gdb) b main
(gdb) c
```

```
(gdb) file ~/helloworld_unstripped
A program is being debugged already.
Are you sure you want to change the file? (y or n) y
Reading symbols from ./helloworld_unstripped...
Reading symbols from /opt/apss-tiny/0.1/sysroots/cortexa32hf-neon-poky-linux-musleabi/lib/ld-linux-armhf.so.3...
Reading symbols from /opt/apss-tiny/0.1/sysroots/cortexa32hf-neon-poky-linux-musleabi/usr/lib/.debug/libc.so...
(gdb) b main
Breakpoint 1 at 0x4005f0: file helloworld.c, line 7.
```

```
(gdb) b main
Breakpoint 1 at 0x4005f0: file helloworld.c, line 7.
(gdb) c
Continuing.

Breakpoint 1, main () at helloworld.c:7
7       helloworld.c: No such file or directory.
(gdb) c
Continuing.

Breakpoint 1, main () at helloworld.c:7
7       in helloworld.c
(gdb) i r
r0             0x0                 0
r1             0xbefffdcc          3204447692
r2             0x0                 0
r3             0xa6c552e5          2797949669
r4             0x1                 1
r5             0xbefffe44          3204447812
r6             0x4005e8            4195816
r7             0xbefffe4c          3204447820
r8             0xb6fff1a4          3070226852
r9             0x20                32
r10            0x1                 1
r11            0xbefffe0c          3204447756
r12            0xbefffd90          3204447632
sp             0xbefffe08          0xbefffe08
lr             0xb6fb7858          3069933656
pc             0x4005f0            0x4005f0 <main+8>
cpsr           0x0                 0
(gdb) disas
```

```
(gdb) disas
Dump of assembler code for function main:
   0x004005e8 <+0>:     push    {r11, lr}
   0x004005ec <+4>:     add     r11, sp, #4
=> 0x004005f0 <+8>:     ldr     r3, [pc, #20]   ; 0x40060c <main+36>
   0x004005f4 <+12>:    add     r3, pc, r3
   0x004005f8 <+16>:    mov     r0, r3
   0x004005fc <+20>:    bl      0x4003b8 <puts@plt>
   0x00400600 <+24>:    mov     r0, #1
   0x00400604 <+28>:    bl      0x4003d0 <sleep@plt>
   0x00400608 <+32>:    b       0x4005f0 <main+8>
   0x0040060c <+36>:    andeq   r0, r0, r0, lsr #32
End of assembler dump.
(gdb)
```

10. **Debugging Commands**

    Use the following GDB commands within the prompt for debugging:

    - b <function> – Set a breakpoint at the specified function.
    - n – Step over a single line.

- s – Step into a function.
- b *<address> – Set a breakpoint at the specified address.
- list – Display the source code being debugged.
- i r – View register information.
- i b – List all breakpoints.
- del – Delete breakpoints.
- q – Quit GDB.

11. **Additional Resources**

    For more details on GDB commands, refer to the *GDB Quick Reference*.

12. **View Output**

    The output of the embedded application (e.g., helloworld) will be displayed on the console running gdbserver on the Devkit device.

```
root@devkit-e7:~# gdbserver :1234 /var/volatile/sd_share/helloworld
Process /var/volatile/sd_share/helloworld created; pid = 98
Listening on port 1234
Remote debugging from host 10.10.4.141, port 51782
Linux Hello World Example 1
```

8. Kernel Drivers and Features

8.1 List of Linux Kernel Drivers Supported

This chapter lists the Linux kernel drivers supported in this release for the Alif DevKit, along with their features, configurations, and verification methods.

Supported Drivers

- HWSEM (Hardware Semaphore)
- MHU (Message Handling Unit)
- I2C Temperature Sensor (ENV3 Click)
- SPI Slave
- I3C (Improved Inter Integrated Circuit)
- Ethernet
- USB Host Mode (DWC3 Dual Role Controller)
- USB Device Mode (DWC3 Dual Role Controller)
- USB Device Detection
- SD/MMC /eMMC
- CRC (Cyclic Redundancy Check)
- I2S (Integrated Inter-IC Sound Bus)
- DAC12 (Digital-to-Analog Converter 12)
- ADC12/ADC24 (Analog-to-Digital Converter 12/24)
- SPI Temperature Sensor (ENV3 Click)
- COMPARATOR (CMP)
- PDM
- UTIMER
- I2S

8.1.1 HWSEM

HWSEM manages access to shared resources across multiple cores, supporting 16 semaphores on the DevKit to prevent race conditions.

8.1.1.1 Features

- Lock/unlock semaphores,
- read MASTER_ID and lock count.

8.1.1.2 Kernel Configurations

```
CONFIG_HWSPINLOCK=y
CONFIG_HWSEM_ALIF=y
```

8.1.1.3 Verification

Tested with:

- */opt/linux_dd_test/hwsem/basic_lock_unlock_test_all:*
  Locks/unlocks all 16 semaphores twice.
- /opt/linux_dd_test/hwsem/basic_lock_unlock_test_hwsem0:
  Locks/unlocks HWSEM0 thrice.
- /opt/linux_dd_test/hwsem/pthreads_lock_unlock:
  Two POSIX threads lock/unlock HWSEM0.

Note: Enable apss-hwsem in DISTRO_FEATURES to build these tests.

8.1.2 MHU (Message Handling Unit)

The MHU facilitates communication between cores in the DevKit's Heterogeneous Multicore Processing (HMP) platform using the Linux kernel's Mailbox and RPMSG drivers. It employs paired Sender and Receiver frames for full-duplex communication. MHUv2 nodes are divided into separate TX and RX channels.

8.1.2.1 Data Transfer

- MHU 0 (secure): 8 bytes (4 bytes/channel)
- MHU 1 (non-secure): 8 bytes (4 bytes/channel)

8.1.2.2 Subsystems

- APSS communicates with RTSS-HP, RTSS-HE, and SE via MHU 0 and 1.

8.1.2.3 Kernel Configurations

```
CONFIG_NET=y
CONFIG_UNIX=y
CONFIG_MAILBOX=y
CONFIG_ARM_MHUv2=y
CONFIG_RPMSG=y
CONFIG_RPMSG_CHAR=y
CONFIG_RPMSG_ARM=y
```

```
CONFIG_RPMSG_CTRL=y
CONFIG_RPMSG_NS=y
```

8.1.2.4 MHU Linux Kernel Driver Verification

The MHU Linux kernel driver is tested using the user space example located at /opt/linux_dd_test/mhu/main. This example sends service requests to the Secure Enclave (SE) via service APIs. The service APIs transmit requests as service data structures to the SE through MHU calls. The SE processes these requests and returns responses to the Application Processor Subsystem (APSS) via MHU calls.

Service Data Structure

The service data structure varies depending on the requested service type, initiated by the user space example. All structures include a common service header with the following fields:

- Requested Service ID
- Flags
- Response Error Code

Example:

```
typedef struct
{
   uint16_t send_service_id;  // Requested Service ID
   uint16_t send_flags;       // Request flags
   uint16_t resp_error_code;  // Transport layer error code
   uint16_t none_padding;     // Padding
} service_header_t;
```

8.1.2.5 Request and Response Process

The user-space example assigns a service request to the send_service_id field in the service header. It sends an MHU message to the Secure Element (SE) via the TXDB4 channel, using the service header address as MHU data. A fixed 4KB memory block (0x027FE000–0x027FEFFF) stores the service data structures. Upon receiving the MHU message via the RXDB4 channel, the SE identifies the service request by reading the MHU data. The SE then sends a response back to the user-space example via an MHU call, using the same service header address as MHU data. The user-space application reads and processes the response accordingly. Dual endpoint support (TXDB4/RXDB4) has been added for bidirectional communication.

Supported Service Requests

The user space example (/opt/linux_dd_test/mhu/main) sends the following service requests to the SE and displays the results:

- Heartbeat SE Service: Pings the SE to verify readiness for requests.
- Random Number (RND) SE Service: Requests random numbers from the SE.
- Life Cycle State (LCS) SE Service: Retrieves the device's life cycle state.
- Table of Contents (TOC) Version SE Service: Obtains the TOC version.
- Table of Contents (TOC) Number SE Service: Retrieves the TOC count.
- UART Write SE Service: Sends a message for display on the SE UART.

8.1.2.6 Example Output

```
root@devkit-e8:~# /opt/linux_dd_test/mhu/main
MHU0 TEST BETWEEN A32 and SE cores for RND
=========================================
Size of struct all_services_svc_t = 1824
Size of union all_services_svc_t = 1040
Memory mapped at address 0xb6e78000
Memory mapped at address 1000 27fe000 0
The heartbeat test is done
The RND values are: 0x78 0x8a 0xf1 0x0f 0x41 0x39 0xce 0x14
The LCS response is: 0x1
The TOC version is 0
The TOC number is 7
The UART write is successful
Closed opened files
```

8.1.3 I2C ENV3 Click Temperature Sensor

The ENV3-CLICK chemical sensor with an I2C interface measures device temperature. Readings are transmitted via a DesignWare I2C interface.

8.1.3.1 Features

- Uses DWC I2C Master Driver
- Measures temperatures from -40 °C to +85 °C
- Data read/write via DesignWare I2C Controller cat
- Thermometer accuracy: ±2.0 °C

8.1.3.2 ENV3-CLICK Temperature Sensor Verification

After booting Linux on the APSS, execute this command on the target to read the real-time temperature:

```
root@devkit-e8:~# cat /sys/class/i2c-dev/i2c-0/0-
0076/iio:device0/in_temp_input
```

Example output: 26.3125 °C

8.1.3.3 Summary of the Test

1. ENV3-CLICK sensor connects as a client to DWC I2C (instances 0–3), using i2c_0.

7. Registered interrupt for DWC I2C is 121. Interrupt log:

```
root@devkit-e8:~# cat /proc/interrupts
CPU0 33: 96 GIC-0 121 Level 49010000.i2c, 49011000.i2c, 49012000.i2c,
49013000.i2c
```

8. The device displays the real-time temperature, with values that change over time as new readings are taken.

9. Set to 16-bit resolution mode to detect minor temperature changes.

10. Operates in Forced Mode for each temperature reading.

8.1.3.4 Linux Kernel Configurations for ENV3-CLICK Support

```
CONFIG_I2C=y
CONFIG_I2C_BOARDINFO=y
CONFIG_I2C_COMPAT=y
CONFIG_I2C_HELPER_AUTO=y
CONFIG_I2C_DESIGNWARE_PLATFORM=y
CONFIG_I2C_DESIGNWARE_SLAVE=y
CONFIG_I2C_DESIGNWARE_CORE=y
CONFIG_IIO=y
CONFIG_HWSEM_ALIF=n
CONFIG_MAILBOX=n
CONFIG_ARM_MHUv2=n
CONFIG_RPMSG=n
CONFIG_BME680=y
CONFIG_BME680_I2C=y
CONFIG_REGMAP_I2C=y
CONFIG_CONFIGFS_FS=y
```

```
CONFIG_SYSFS=y
CONFIG_EXPERT=y
CONFIG_SYSFS_SYSCALL=y
CONFIG_SLAB=y
CONFIG_BLK_DEV_NULL_BLK=y
```

8.1.4 SPI Slave

The DWC_SSI Controller can be configured as a slave device. On the Alif DevKit, SPI slave and master instances can be connected externally. Messages can be transferred between them using the appropriate /dev entries for each SPI instance..

8.1.4.1 Kernel Configuration

Configure the kernel with the following options:

```
CONFIG_SPI=y
CONFIG_SPI_MASTER=y
CONFIG_SPI_DESIGNWARE=y
CONFIG_SPI_DW_MMIO=y
CONFIG_CONFIGFS_FS=y
CONFIG_BME680=y
CONFIG_BME680_SPI=y
CONFIG_IIO=y
CONFIG_IIO_CONFIGFS=y
CONFIG_SLAB=y
CONFIG_SPI_SPIDEV=y
CONFIG_SPI_SLAVE=y
```

8.1.4.2 Verifying SPI Controller Communication with a Userspace Application

Enable and verify SPI communication from a userspace application by creating and configuring a device entry for the SPI controller on the Alif DevKit.

Procedure

1. **Add a Child Node in the Device Tree.**

   Update the device tree node for the SPI controller to add a child node that specifies the SPI slave or master functionality.

   - Edit the device tree source (DTS) file for the SPI controller.

- Please ensure to add a child node, following the example provided below:
- The application utilizes the SPI3 instance, which already has BME680 configured as a child device. Please remove it and add an entry for spidev0 for easier validation.

Example 1 - SPI Master Child Node:

```
&spi3 {
    pinctrl-names = "default";
    pinctrl-0 = <&pinctrl_spi3>;
    #address-cells = <1>;
    #size-cells = <0>;
    status = SPI3_STATUS;
    spidev0: spidev@0 {
        compatible = "snps,dwc-ssi-1.01a";
        reg = <0>;
        spi-max-frequency = <1000000>;
    }
};
```

- compatible: Specifies driver compatibility ("snps,dwc-ssi-1.01a").
- reg: Sets chip select line (<0> for first device).
- spi-max-frequency: Defines maximum SPI clock frequency (1 MHz or 1000000 Hz).

Example 2 - SPI Slave Child Node:

```
&spi1 {
    pinctrl-names = "default";
    pinctrl-0 = <&pinctrl_spi1>;
    #address-cells = <0>;
    #size-cells = <0>;
    spi-slave;
    status = SPI1_STATUS;
    slave {
        compatible = "snps,dwc-ssi-1.01a";
        spi-max-frequency = <1000000>;
    };
};
```

Note: To configure the SPI controller as a slave, update the clock setting in the DTS from:

clocks = <&psclks ENSEMBLE_SPI1_SS_IN_VAL_MST_CLK>

to:

clocks = <&psclks ENSEMBLE_SPI1_SS_IN_VAL_SLV_CLK>

This change enables slave mode operation.

2. Pin Control changes in DTS

Add the below pincontrol changes in the devkit-e8.dts file.

SPI1_C

```
PIN_P14_7__SPI1_SS0_C 0x23
PIN_P14_6__SPI1_SCLK_C 0x23
PIN_P14_4__SPI1_MISO_C 0x23
PIN_P14_5__SPI1_MOSI_C 0x23
```

SPI3_A

```
PIN_P12_7__SPI3_SS0_A 0x0
PIN_P12_6__SPI3_SCLK_A 0x0
PIN_P12_4__SPI3_MISO_A 0x23
PIN_P12_5__SPI3_MOSI_A 0x0
```

3. Locate Device Entries in Sysfs

Verify SPI device entries after booting with the updated device tree.

Steps:

- Boot the system with the modified device tree.
- Check for SPI device entries in the /sys directory.

Example:

```
/sys/bus/spi/devices/spi0.0/
```

Note: These entries represent SPI devices and can be manually bound to the spidev driver.

4. Bind the Device to the spidev Driver.

To enable communication in userspace, bind the SPI device to the spidev driver..

Steps:

Execute the following commands in a terminal:

```
echo spidev > /sys/bus/spi/devices/spi0.0/driver_override
echo spi0.0 > /sys/bus/spi/drivers/spidev/bind
```

- First command: Sets the driver override to spidev.

- Second command: Binds the spi0.0 device to the spidev driver.

When using SPI1 and SPI3, and only enabling these SPI instances, we will observe spi0.0 (for SPI1) and spi1.0 (for SPI3). Bind both instances to the spidev driver.

```
echo spidev > /sys/bus/spi/devices/spi1.0/driver_override
echo spi1.0 > /sys/bus/spi/drivers/spidev/bind
```

Result

The SPI controller is accessible from a userspace application via the spidev driver.

8.1.4.3 Pin Assignments for SPI3_A and SPI1_C

SPI1_C (Slave) Pins

Pin Name	Pin Number
PIN_P14_7__SPI1_SS0_C	j10-10
PIN_P14_6__SPI1_SCLK_C	j10-8
PIN_P14_4__SPI1_MISO_C	j10-7
PIN_P14_5__SPI1_MOSI_C	j10-9

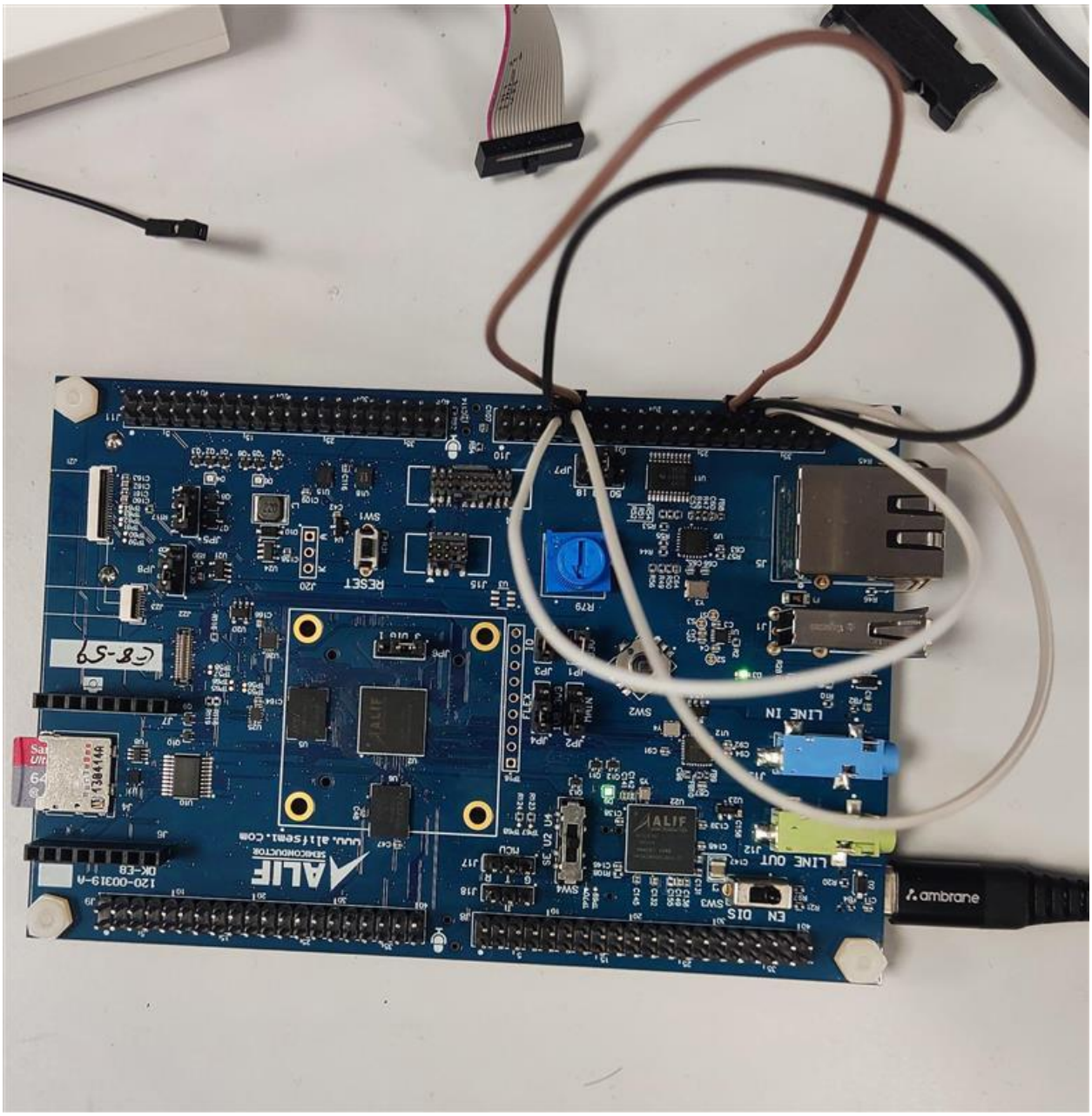
SPI3_A (Master) Pins

Pin Name	Pin Number
PIN_P12_7__SPI3_SS0_A	j10-28
PIN_P12_6__SPI3_SCLK_A	j10-30
PIN_P12_4__SPI3_MISO_A	j10-32
PIN_P12_5__SPI3_MOSI_A	j10-34

External Connections

SPI1_C (Slave) Pin		SPI3_A (Master) Pin
PIN_P14_7__SPI1_SS0_C	->	PIN_P12_7__SPI3_SS0_A
PIN_P14_6__SPI1_SCLK_C	->	PIN_P12_6__SPI3_SCLK_A
PIN_P14_4__SPI1_MISO_C	->	PIN_P12_4__SPI3_MISO_A
PIN_P14_5__SPI1_MOSI_C	->	PIN_P12_5__SPI3_MOSI_A

Sample Connection



Note:

- Pin multiplexers for SPI0_B and the SD card share resources and cannot operate simultaneously. Enabling SPI0 disables SD card functionality.
- When building with Yocto for SD card support, SPI0 instance is disabled by default.

8.1.4.4 Sample Application Example

Demonstrate SPI communication using sample master and slave applications.

Run the slave application first with & so that it will run in background and then run the master application.

Scenario: Two SPI device entries exist, and a block of data is transferred between controllers.

```
Alif Iota - Tiny Linux Distribution 0.1 devkit-e7 /dev/ttyS0

devkit-e7 login: root
login[36]: root login on 'ttyS0'
root@devkit-e7:~# /dev/spidev
spidev0.0  spidev1.0
root@devkit-e7:~# /dev/spidev
spidev0.0  spidev1.0
root@devkit-e7:~# spidev_Slave &
root@devkit-e7:~# Before performing transfer

root@devkit-e7:~# spidev_Master -D /derandom: fast init done
root@devkit-e7:~# spidev_Master -D /dev/spidev1.0 -v
spi mode: 0x0
bits per word: 8
max speed: 500000 Hz (500 KHz)
Received data: FF FF FF FF FF FF 40 00 00 00 00 95 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF F0 0D
TX | FF FF FF FF FF FF 40 00 00 00 00 95 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF F0 0D  |......@.....................|
RX | BA AD CA FE AB AC AD AE AF BA BB BC BD BE BF CA CB CC CD CE CF DA DB DC DD DE DF EA EB EC ED EE  |.............................|
root@devkit-e7:~#
```

8.1.5 I3C

The DesignWare Cores MIPI I3C Controller (DWC_mipi_i3c) is a digital core that implements the MIPI I3C Specification for master functionality. This section describes the I3C controller in the Alif DevKit's APSS Linux environment, covering its features, the BMI323 IMU unit, kernel configuration, and device tree setup.

The DWC_mipi_i3c provides an interface between the system and the I3C bus, enabling communication with I3C devices and, with limitations per the MIPI I3C Specification, legacy I2C devices. This Linux release supports only the I3C master driver.

8.1.5.1 Features Supported

- **Master Functionality**: Supports Main Master mode only.
- **Data Rates**:
  - Fast Speed (FS) mode.

  - Fast Speed Plus (FS+) mode.
  - Single Data Rate (SDR) up to 10 Mbps.
  - High Data Rate—Double Data Rate (HDR-DDR) up to 20 Mbps.
- **Legacy I2C Support**: Compatible with legacy I2C devices on the I3C bus.
- **Buffers**:
  - Command buffer: 8 entries (16 locations, 4 bytes each).
  - Response buffer: 8 entries (8 locations, 4 bytes each).
  - Transmit and receive data buffers: 64 entries each (64 locations, 4 bytes each).
- **Transfer Capacity**: Up to 216 bytes for write/read with a single command.
- **Error Checking**: CRC and parity generation/validation.
- **Command Types**: Broadcast and directed Common Command Code (CCC) transfers.
- **Addressing**: Device address table for multiple slaves.

8.1.5.2 BMI323 IMU Unit

The Alif Ensemble board includes a BMI323 Inertial Measurement Unit (IMU) connected to the I3C 0_D instance. The BMI323 is a low-power, highly integrated sensor with the following components:

- **Accelerometer:** 16-bit, triaxial, with configurable ranges of ±2g, ±4g, ±8g, ±16g.
- **Gyroscope:** 16-bit, triaxial, with configurable ranges of ±125°/s, ±250°/s, ±500°/s, ±1000°/s, ±2000°/s.
- **Temperature Sensor:** 16-bit, operating from -40°C to +85°C.
- **Features:** Includes intelligent on-chip motion-triggered interrupts.

8.1.5.3 Kernel Configuration

Enable the following Linux kernel options to support the I3C controller and BMI323 IMU:

```
CONFIG_I3C=y
CONFIG_I2C=y
CONFIG_DW_I3C_MASTER=y
CONFIG_IIO=y
CONFIG_IIO_CONFIGFS=y
CONFIG_BMI323=y
CONFIG_BMI323_I3C=y
CONFIG_REGMAP=y
CONFIG_REGMAP_I2C=y
```

```
CONFIG_REGMAP_MMIO=y
CONFIG_REGMAP_I3C=y
CONFIG_REGULATOR=y
CONFIG_REGULATOR_ENSEMBLE=y
CONFIG_REGULATOR_FIXED_VOLTAGE=y
```

8.1.5.4 Device Tree Configuration

The device tree configures the I3C 0_D instance and BMI323 IMU on the Alif Ensemble board.

Below is a sample configuration (adding BMI323 child node to the I3C node):

```
&i3c0 {
        pinctrl-names = "default";
        pinctrl-0 = <&pinctrl_i3c0>;
        status = I3C0_STATUS;
        bmi323_i3c@69 {
                reg = <0x69 0x03b8 0x0>;
                vdd-supply = <&reg_1p8v>;
                vddio-supply = <&reg_1p8v>;
                assigned-address = <0xa>;
                status = "okay";
        };
};
```

8.1.5.5 Test Output

```
root@devkit-e7:~# cat /sys/bus/iio/devices/iio:device0/in_temp_raw
735
```

8.1.5.6 Temperature Calculation from Raw Value

```
Temperature value. T (°C):= in_temp_raw/512 + 23
```

8.1.5.7 Known Issues

8. Pressing the reset button on the Ensemble board resets the core, but the BMI323 sensor fails to reset because its VDD and VDDIO lines remain fixed at 1.8V, rather than cycling from 0V to 1.8V as required for a proper power-on reset. Additionally, the sensor's software reset is disabled to ensure I3C compatibility, leading to incorrect status data that may disrupt system performance. A software solution is currently in developme

9.

10. nt to address this issue.

8.1.6 UTIMER

The Universal Timer (UTIMER) is a 32-bit high-resolution signal timing generator and pulse counter, clocked at a 400 MHz clock with 2.5 ns accuracy for generating PWM output or measuring input signal timing characteristics. The device includes up to 16 independent channels, with channels 0–11 (UTIMER0 to UTIMER11) configured as standard UTIMER modules. Each channel supports:

- Reaction to events from 2 channel events.
- Programmatic start/stop/clear of one, some, or all channels simultaneously.
- 128 maskable interrupts (8 per channel).
- 2 output drivers based on compare events per channel.
- Counting in pulse/direction or increment/decrement modes (only Basic Mode is supported in this implementation).
- 32-bit wide counters and compare registers.
- 8 interrupt events, some shareable among channels for complex state machine implementation.

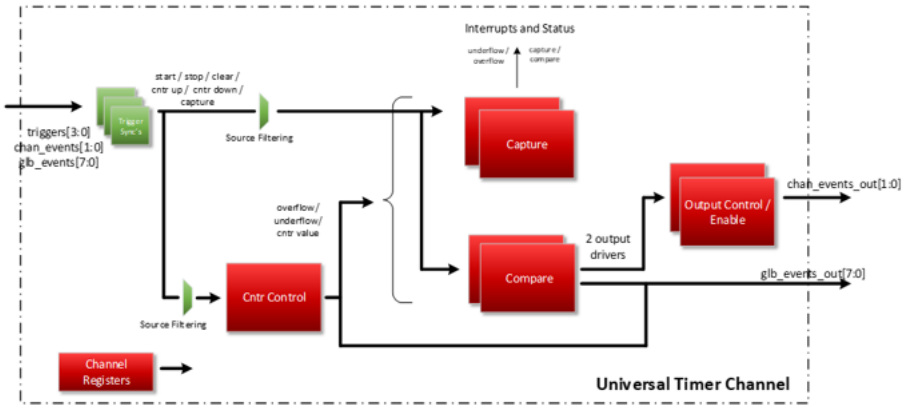


Figure 4 : Utimer Block Diagram

8.1.6.1 Modes Supported

Although the UTIMER hardware supports multiple modes, this implementation supports only the **Basic Mode**. In Basic Mode, the counter runs freely once started until stopped. When interrupts are enabled, each counter overflow or underflow event generates an interrupt.

8.1.6.2 Kernel Configuration

To enable UTIMER-Basic support in the Linux kernel, configure the following options:

```
CONFIG_COUNTER=y
```

```
CONFIG_ALIF_UTIMER_CNT=y
CONFIG_CONFIGFS_FS=y
CONFIG_SYSFS=y
```

These settings enable the UTIMER driver, counter framework, and sysfs interface.

8.1.6.3 Build Integration

To include UTIMER support in the APSS Linux project:

1. Add "apss-utimer" to the DISTRO_FEATURES variable in conf/local.conf.
2. Build the project to generate:
   - xipImage with the UTIMER driver enabled.
   - cramfs-xip containing the UTIMER test program at */opt/linux_dd_test/utimer/utimer_test*.

8.1.6.4 Device Tree Configuration

The UTIMER module requires a device tree entry to define its hardware properties.

```
utimer: utimer@48000000 {
        compatible = "alif,alif-utimer";
        clocks = <&psclks ENSEMBLE_SYST_ACLK>;
        clock-names = "axi_clk" ;
        reg = <0x48000000 0x100000>;
        interrupt-parent = <&gic>;
        interrupts =    <GIC_SPI 366 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 367 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 368 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 369 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 370 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 371 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 372 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 373 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 374 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 375 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 376 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 377 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 378 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 379 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 380 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 381 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 382 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 383 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 384 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 385 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 386 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 387 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 388 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 389 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 390 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 391 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 392 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 393 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 394 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 395 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 396 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 397 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 398 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 399 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 400 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 401 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 402 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 403 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 404 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 405 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 406 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 407 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 408 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 409 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 410 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 411 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 412 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 413 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 414 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 415 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 416 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 417 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 418 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 419 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 420 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 421 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 422 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 423 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 424 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 425 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 426 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 427 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 428 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 429 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 430 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 431 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 432 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 433 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 434 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 435 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 436 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 437 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 438 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 439 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 440 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 441 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 442 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 443 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 444 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 445 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 446 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 447 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 448 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 449 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 450 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 451 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 452 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 453 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 454 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 455 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 456 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 457 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 458 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 459 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 460 IRQ_TYPE_EDGE_RISING>,
                        <GIC_SPI 461 IRQ_TYPE_EDGE_RISING>;
};
```

Figure 52 : Utimer Device Tree

8.1.6.5 Verification

The Alif UTIMER Linux device driver can be tested using the provided test program to demonstrate up and down counter functionality in Basic Mode.

Test Program

- **Location***: /opt/linux_dd_test/utimer/utimer_test*
  (available in *cramfs-xip* when *apss-utimer* is enabled).
- **Purpose**: Tests up and down counter functionality, returning counter status and elapsed time for the selected channel.
- **Usage**:

```
./utimer_test -[a-p]
```

Options:

```
-a: Test Channel 0
-b: Test Channel 1
-c: Test Channel 2
-d: Test Channel 3
-e: Test Channel 4
-f: Test Channel 5
-g: Test Channel 6
-h: Test Channel 7
-i: Test Channel 8
-j: Test Channel 9
-k: Test Channel 10
-l: Test Channel 11
```

8.1.7.5 Test Output

The following example shows testing Channel 1 using the UTIMER test program on the devkit-e8 system:

```
root@devkit-e8: /opt/linux_dd_test/utimer # ./utimer_test -b


=== UTIMER Basic Information ===
1. Utimer Name
Utimer name= 48000000.utimer
2. Utimer Channel Count
Utimer num_counts= 16
3. Utimer Signal Count
Utimer num_signals= 44
```

```
=== Channel 1 Information ===
1. Channel Name
Channel name= Channel 2
2. Channel Function
Channel function= increase
3. Channel Count Mode
Channel count_mode= normal
4. Channel Direction
Channel direction= forward
5. Counter Signal Action
Counter Signal Action= falling edge

=== Testing UP COUNTER on channel 1 ===
/sys/bus/counter/devices/counter0/count1/ceiling: 200000000
/sys/bus/counter/devices/counter0/count1/enable: 1
/sys/bus/counter/devices/counter0/count1/direction_rw: forward
/sys/bus/counter/devices/counter0/count1/count: 0
/sys/bus/counter/devices/counter0/count1/running: 1

/sys/bus/counter/devices/counter0/count1/counter_status
utimer 48000000.utimer: UTIMER channel 1: Overflow (ms)
499
/sys/bus/counter/devices/counter0/count1/count: 13818502
/sys/bus/counter/devices/counter0/count1/count: 15788812
/sys/bus/counter/devices/counter0/count1/count: 19789889
/sys/bus/counter/devices/counter0/count1/count: 23782677
/sys/bus/counter/devices/counter0/count1/count: 27782290
/sys/bus/counter/devices/counter0/count1/running: 0
/sys/bus/counter/devices/counter0/count1/enable: 0
/sys/bus/counter/devices/counter0/count1/count: 0
/sys/bus/counter/devices/counter0/count1/count: 0
/sys/bus/counter/devices/counter0/count1/count: 0
/sys/bus/counter/devices/counter0/count1/count: 0
/sys/bus/counter/devices/counter0/count1/count: 0
```

```
=== Testing DOWN COUNTER on channel 1 ===
/sys/bus/counter/devices/counter0/count1/ceiling: 400000000
/sys/bus/counter/devices/counter0/count1/enable: 1
/sys/bus/counter/devices/counter0/count1/direction_rw: backward
/sys/bus/counter/devices/counter0/count1/count: 400000000
/sys/bus/counter/devices/counter0/count1/running: 1

/sys/bus/counter/devices/counter0/count1/counter_status
utimer 48000000.utimer: UTIMER channel 1: Underflow (ms)
1000
/sys/bus/counter/devices/counter0/count1/count: 386131635
/sys/bus/counter/devices/counter0/count1/count: 384192203
/sys/bus/counter/devices/counter0/count1/count: 380190923
/sys/bus/counter/devices/counter0/count1/count: 376198569
/sys/bus/counter/devices/counter0/count1/count: 372199170
/sys/bus/counter/devices/counter0/count1/running: 0
/sys/bus/counter/devices/counter0/count1/enable: 0
/sys/bus/counter/devices/counter0/count1/count: 400000000
/sys/bus/counter/devices/counter0/count1/count: 400000000
/sys/bus/counter/devices/counter0/count1/count: 400000000
/sys/bus/counter/devices/counter0/count1/count: 400000000
/sys/bus/counter/devices/counter0/count1/count: 400000000
```

The output shows a delay of 499 ms for the up counter (overflow) and 1000 ms for the down counter (underflow). The initial count and ceiling value (maximum value of the counter) for up and down counters are defined in the test code and can be modified. The up counter measures the time to reach the ceiling and overflow, while the down counter measures the time to count down from the ceiling to zero.

8.1.7 Ethernet

The Alif DevKit supports the DesignWare Cores Ethernet Quality of Service (QoS) driver within a minimal Linux distribution. This section describes the Ethernet driver features, kernel configuration, and device tree setup required to enable Ethernet functionality on the DevKit.

8.1.7.1 Features

The Ethernet driver provides the following capabilities:

- Speed Support: Operates at 10 Mbps and 100 Mbps.
- RGMII PHY Configurations: Supports Reduced Gigabit Media Independent Interface (RGMII) PHY configurations.
- Ethtool Queries: Handles common ethtool commands for network interface status and configuration.
- NAPI Support: Implements New API (NAPI) for efficient packet processing.
- Checksum Offload: Offers full or partial checksum offloading to reduce CPU load.
- Multi-Queue Support: Enables multiple transmit and receive queues for improved performance.

8.1.7.2 Kernel Configuration

To build the AXI Ethernet driver for the DesignWare QoS controller, enable the following Linux kernel configuration options:

Linux DesignWare QoS Driver Support

plaintext

```
CONFIG_PACKET=y
CONFIG_TLS=y
CONFIG_INET=y
CONFIG_NETFILTER_ADVANCED=y
CONFIG_NETDEVICES=y
CONFIG_ETHERNET=y
CONFIG_NET_VENDOR_STMICRO=y
CONFIG_STMMAC_ETH=y
CONFIG_STMMAC_SELFTESTS=y
CONFIG_STMMAC_PLATFORM=y
CONFIG_DWMAC_DWC_QOS_ETH=y
CONFIG_DWMAC_GENERIC=y
CONFIG_MDIO_DEVICE=y
CONFIG_PHYLIB=y
CONFIG_REALTEK_PHY=y
```

8.1.7.3 Device Tree Configuration

The device tree defines the Ethernet controller's hardware configuration on the DevKit. Below is an example device tree entry for the Ethernet interface (eth0):

```
eth0@0x48100000 {
    clock-names = "phy_ref_clk", "apb_pclk";
    clocks = <&syst_pclk>, <&syst_pclk>;
    compatible = "snps,dwc-qos-ethernet-4.10";
    interrupt-parent = <&gic>;
    interrupts = <GIC_SPI 137 IRQ_TYPE_LEVEL_HIGH>;
    reg = <0x48100000 0x2000>;
    phy-handle = <&phy2>;
    max-speed = <100>;
    phy-mode = "rmii";
    mac-address = [92 60 04 02 c3 e6];
    pinctrl-names = "default";
    pinctrl-0 = <&pinctrl_eth0>;
    status = ETH_STATUS;
    mdio {
        #address-cells = <0x1>;
        #size-cells = <0x0>;
        phy2: phy@0 {
            compatible = "ethernet-phy-ieee802.3-c22";
            device_type = "ethernet-phy";
            reg = <0x1>;
        };
    };
};
```

8.1.8 USB Host Mode (DWC3 Dual Role Controller)

Standard USB operates on a Host/Device model. The Host controls the bus, while the Device follows commands. In standard USB, devices assume a fixed role: computers typically act as Hosts, and peripherals (e.g., printers) function as Devices. Embedded systems with dual-role capability can switch

between Host and Device modes based on requirements. The USB Controller supports both modes, and the DevKit includes the DWC3 Dual Role Controller.

8.1.8.1 Features

- Supports up to 127 devices
- Supports up to 1024 interrupters
- Supports up to 15 USB 2.0 ports and 15 SuperSpeed ports
- Supports up to 4 SuperSpeed, 4 High-Speed, and 1 Full-Speed/Low-Speed bus instances
- Compatible with xHCI 1.1
- Supports standard or open-source xHCI and class drivers
- Supports up to 16 bidirectional endpoints (including control endpoint ep0)
- Offers flexible endpoint configuration
- Enables simultaneous IN and OUT transfers
- Supports scatter-list functionality
- Supports up to 256 TRBs per endpoint
- Handles all transfer types: Control, Bulk, Interrupt, Isochronous
- Supports SuperSpeed Bulk Streams
- Includes Link Power Management

The DWC3 Dual Role Controller uses the xHCI Host Controller Driver (HCD) for USB 3.0 to detect, enumerate, and transfer data with connected USB 3.0 and USB 2.0 devices in Host mode. The extensible Host Controller Interface (xHCI) is the standard for USB 3.0 SuperSpeed host hardware. Figure 15 illustrates the USB Host and Device components. The USB interface supports a USB-B micro cable or direct connection via a USB-A port. The target operates as a USB Host using xHCI, with the driver comprising the DWC3 Dual Role Controller and xHCI Host Controller Driver.

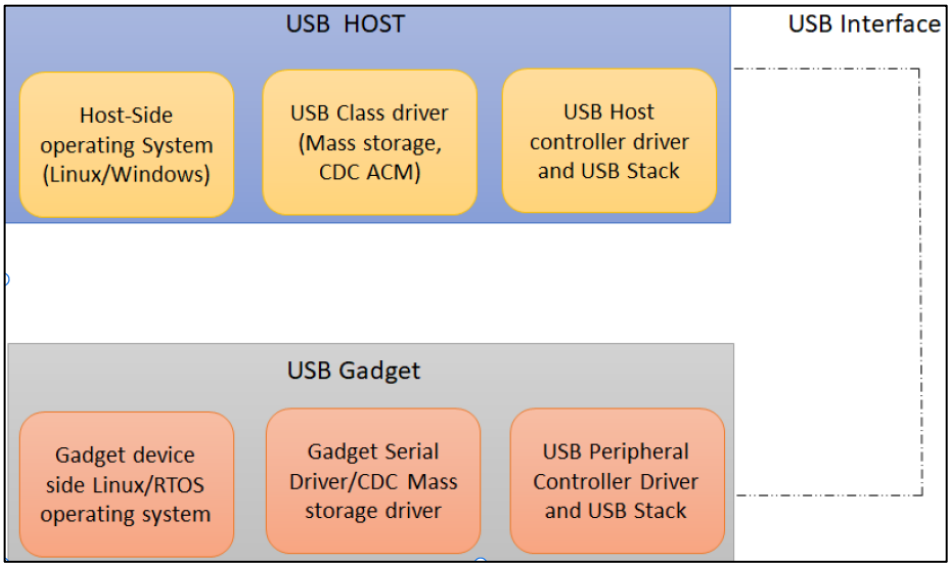


Figure 23 DWC3 Dual-Role USB Controller

8.1.8.2 Kernel Configurations

```
CONFIG_SCSI=y
CONFIG_SCSI_PROC_FS=y
CONFIG_USB=y
CONFIG_USB_SUPPORT=y
CONFIG_BLK_DEV_SD=y
CONFIG_USB_ANNOUNCE_NEW_DEVICES=y
CONFIG_USB_XHCI_HCD=y
CONFIG_USB_XHCI_PLATFORM=y
CONFIG_USB_DWC3=y
CONFIG_USB_DWC3_HOST=y
CONFIG_USB_STORAGE=y
CONFIG_USB_ACM=y
```

8.1.8.3 Device Tree Configuration

```
hsusb: dwc3-drd {
    usbdual_hs: usb-dual@48200000 {
        compatible = "snps,dwc3";
        reg = <0x48200000 0x100000>;
        interrupts = <GIC_SPI 28 IRQ_TYPE_LEVEL_HIGH>;
        dr_mode = "host";
        snps,hsphy_interface = "utmi";
```

```
        phy_type = "utmi_wide";
        maximum-speed = "high-speed";
        status = "okay";
    };
};
```

Configuration Notes:

- dr_mode = "host": Configures the controller in Host mode.
- maximum-speed = "high-speed": Sets the controller as a USB 2.0 High-Speed Host.

8.1.9 USB Device Mode (DWC3 Dual Role Controller)

Certain embedded devices feature USB Device Controllers configurable in Host or Device mode. A USB-B micro cable connects the device to a USB Host, such as a PC. In Device mode, the embedded target requires a "Gadget driver" to define its device type (e.g., CDC ACM, CDC ECM, CDC MS). These are composite devices with multiple functions. To configure the USB Device as a serial gadget, the g_serial gadget driver must be enabled in the kernel.

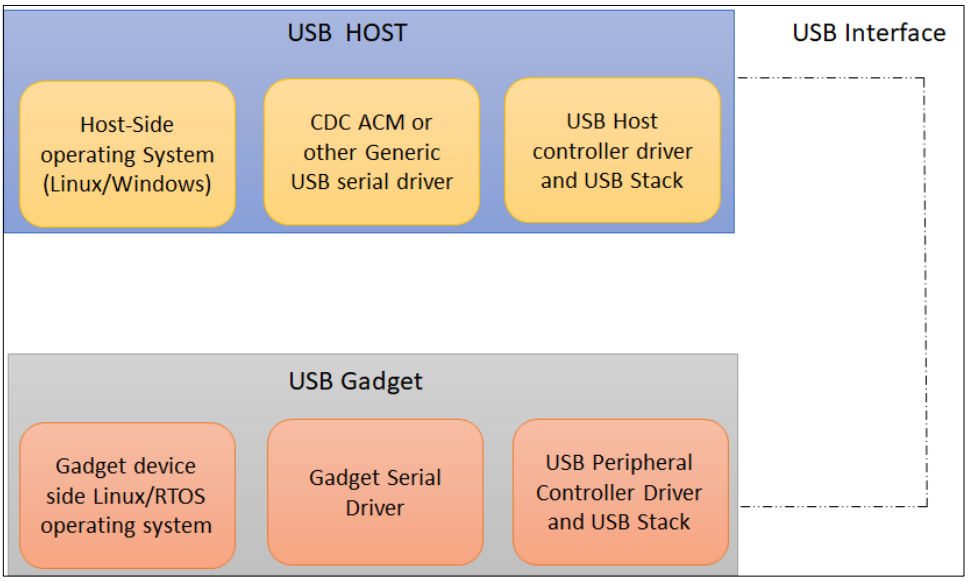


Figure 6 DWC3 Dual-Role USB Controller

Figure above illustrates the components for a USB Host and USB Device operating as a USB Serial Gadget, including related class components in the USB Host. The USB interface uses a USB-B or USB-C cable for physical connection.

8.1.9.1 DWC3 Dual Role USB Controller Features

- Up to 16 bidirectional endpoints, dynamic TxFIFOs.
- DWC3 Dual-Role USB Controller Features:
- Supports dual-role capability (Host or Device)
- Unified programming model for SuperSpeed (SS), High-Speed (HS), Full-Speed (FS), and Low-Speed (LS)
- Includes internal DMA Controller
- Supports LPM protocol in USB 2.0

8.1.9.2 DWC3 Device Mode Features:

- Supports up to 16 bidirectional endpoints, including control endpoint 0
- Software handles non-timing-critical tasks (e.g., control transactions)
- Offers flexible endpoint configuration
- Supports dynamic TxFIFO mapping for more Tx-endpoints than physical FIFOs
- Enables simultaneous IN and OUT transfers

  The DevKit supports the DWC3 Dual Role Controller, configurable as either Host or Device. In Device mode, it operates as a USB 2.0 High-Speed serial gadget. When connected via a USB-B micro cable to a Linux Host (USB-A end), it appears as a cdc-acm device: /dev/ttyACM0 on the Host and /dev/ttyGS0 on the target. As USB follows a master-slave protocol, all transactions are initiated by the Host.

8.1.9.3 Kernel Configurations

```
CONFIG_USB_OHCI_LITTLE_ENDIAN=y
CONFIG_USB_SUPPORT=y
CONFIG_USB_COMMON=y
CONFIG_USB_ARCH_HAS_HCD=y
CONFIG_USB_DWC3=y
CONFIG_USB_DWC3_GADGET=y
CONFIG_USB_DWC3_ENSEMBLE=y
CONFIG_USB_GADGET=y
CONFIG_USB_GADGET_VBUS_DRAW=2
CONFIG_USB_GADGET_STORAGE_NUM_BUFFERS=2
CONFIG_USB_LIBCOMPOSITE=y
CONFIG_USB_F_ACM=y
```

```
CONFIG_USB_U_SERIAL=y
CONFIG_USB_F_SERIAL=y
CONFIG_USB_F_OBEX=y
CONFIG_USB_G_SERIAL=y
```

8.1.9.4 Device Tree Configuration

```
                hsusb: dwc3-drd {
                         usbdual_hs: usb-dual@48200000 {
                         compatible = "snps","snps,dwc3";
                         reg = <0x48200000 0x100000>;
                         interrupts = <GIC_SPI 26 IRQ_TYPE_LEVEL_HIGH>;
                        dr_mode = "peripheral";
                        maximum-speed = "high-speed";
                         status = "okay";
                         };
                 };
```

8.1.9.5 Detection

When connected to a Linux Host via USB Micro-B, the DevKit is detected as a ttyACM0 device on the Host:

```
usb 1-1: New USB device found, idVendor=0525, idProduct=a4a7, bcdDevice=5.04
usb 1-1: New USB device strings: Mfr=1, Product=2, SerialNumber=0
usb 1-1: Product: Gadget Serial v2.4
usb 1-1: Manufacturer: Linux 5.4.25-00024-g9283a6810958-dirty with dwc3-
gadget
cdc_acm 1-1:2.0: ttyACM0: USB ACM device
```

8.1.10 USB Device Detection

The Alif DevKit's USB controller operates as a high-speed USB 2.0 host. When a slave device is connected—via a USB-A port on the target device and a USB-B micro port on a Linux gadget (e.g., embedded targets) or another bus-powered device—the connected USB device is detected based on its class and the availability of corresponding drivers on the host side. This section covers USB device detection, kernel configuration, and example logs for common device types.

The USB host detects devices as follows:

CDC ACM Devices: Recognized as serial devices (e.g., /dev/ttyACM0), such as a USB serial gadget.

Mass Storage Devices: Detected as storage devices (e.g., /dev/sda*), such as a USB pen drive with mount points.

Requirements: The USB host must have the appropriate class drivers enabled in the kernel configuration for the connected device to function.

8.1.10.1 Kernel Configuration

To enable USB device detection and support, configure the Linux kernel with the following options:

```
CONFIG_USB_OHCI_LITTLE_ENDIAN=y
CONFIG_USB_SUPPORT=y
CONFIG_USB_COMMON=y
CONFIG_USB_ARCH_HAS_HCD=y
CONFIG_USB_DWC3=y
CONFIG_USB_DWC3_GADGET=y
CONFIG_USB_DWC3_OF_SIMPLE=y
CONFIG_USB_DWC3_ENSEMBLE=y
CONFIG_USB_GADGET=y
CONFIG_USB_GADGET_VBUS_DRAW=2
CONFIG_USB_GADGET_STORAGE_NUM_BUFFERS=2
CONFIG_USB_LIBCOMPOSITE=y
CONFIG_USB_F_ACM=y
CONFIG_USB_U_SERIAL=y
CONFIG_USB_F_SERIAL=y
CONFIG_USB_F_OBEX=y
CONFIG_USB_G_SERIAL=y
CONFIG_SLAB=y
CONFIG_BLK_DEV_NULL=y
```

8.1.10.2 USB Device Detection Examples

The USB host enumerates devices and loads the appropriate class driver based on the connected device.

Below are examples of detection logs.

CDC ACM Device Detection

A target device configured as a USB gadget is connected to the host via a USB-B micro cable.

Log Output:

```
usb 1-1: New USB device found, idVendor=0525, idProduct=a4a7, bcdDevice= 5.04
usb 1-1: New USB device strings: Mfr=1, Product=2, SerialNumber=0
usb 1-1: Product: Gadget Serial v2.4
usb 1-1: Manufacturer: Linux 5.4.25-00024-g9283a6810958-dirty with dwc3-
gadget
cdc_acm 1-1:2.0: ttyACM0: USB ACM device
```

Mass Storage Device Detection

A USB pen drive is plugged into the USB-A port of the target device (host).

Log Output:

```
usb 1-1: new high-speed USB device number 7 using xhci-hcd
usb 1-1: New USB device found, idVendor=0781, idProduct=5567, bcdDevice= 1.00
usb 1-1: New USB device strings: Mfr=1, Product=2, SerialNumber=3
usb 1-1: Product: Cruzer Blade
usb 1-1: Manufacturer: SanDisk
usb 1-1: SerialNumber: 4C530001170825116350
usb-storage 1-1:1.0: USB Mass Storage device detected
scsi host0: usb-storage 1-1:1.0
scsi 0:0:0:0: Direct-Access     SanDisk  Cruzer Blade     1.00 PQ: 0 ANSI: 6
sd 0:0:0:0: [sda] 30464000 512-byte logical blocks: (15.6 GB/14.5 GiB)
sd 0:0:0:0: [sda] Write Protect is off
sd 0:0:0:0: [sda] Mode Sense: 43 00 00 00
sd 0:0:0:0: [sda] Write cache: disabled, read cache: enabled, doesn't support
DPO or FUA
sda: sda1
sd 0:0:0:0: [sda] Attached SCSI removable disk
```

8.1.11 SD/eMMC SDHCI (DWC MSHC)

The DesignWare Cores DWC_mshc is a high-performance, configurable Secure Digital (SD) host controller used in the Alif DevKit's APSS Linux environment. This section details its features, kernel configuration, and device tree setup for SD/eMMC support.

The DWC_mshc is a programmable mobile storage host controller with AXI/AHB bus interfaces, designed for communication with SD memory cards in mobile and portable devices. It offers robust SD card functionality tailored to embedded systems.

8.1.11.1 Features Supported

- **Card Detection and Capacity**:
  - Detects and mounts Class 4 SD cards up to 32 GB and eMMC up to 64GB.
  - Supports Class 10 SD cards up to 8 GB.
  - *Limitation*: Transcend SD cards are currently not detected.
- **Filesystem Support**: Boots ext4, ext3, and ext2 root filesystems.
- **Operations**: Supports read, write, and erase operations on mounted cards.
- **FAT32 Compatibility**: Recognizes FAT32-formatted SD cards.
- **Frequency**: SDHCI controller operates at a maximum frequency of 50 MHz.
- **ADMA**: Includes Advanced Direct Memory Access (ADMA) support for efficient data transfer.

8.1.11.2 Kernel Configuration

Enable the following Linux kernel options to support the SDHCI controller:

SD Card Mount Support

```
CONFIG_REGULATOR=y
CONFIG_REGULATOR_FIXED_VOLTAGE=y
CONFIG_MMC=y
CONFIG_PWRSEQ_EMMC=y
CONFIG_PWRSEQ_SIMPLE=y
CONFIG_MMC_BLOCK=y
CONFIG_MMC_BLOCK_MINORS=8
CONFIG_MMC_SDHCI=y
CONFIG_MMC_SDHCI_PLTFM=y
CONFIG_MMC_SDHCI_OF_DWCMSHC=y
CONFIG_EXT3_FS=y
```

SD Card Boot Support

```
CONFIG_CMDLINE="console=ttyS0,115200n8 root=/dev/mmcblk0p1 rootfstype=ext4
rootwait rw loglevel=9"
CONFIG_CMDLINE_FORCE=y
CONFIG_REGULATOR=n
```

```
CONFIG_MMC=y
CONFIG_PWRSEQ_EMMC=y
CONFIG_PWRSEQ_SIMPLE=y
CONFIG_MMC_BLOCK=y
CONFIG_MMC_BLOCK_MINORS=8
CONFIG_MMC_SDHCI=y
CONFIG_MMC_SDHCI_PLTFM=y
CONFIG_MMC_SDHCI_OF_DWCMSHC=y
CONFIG_EXT3_FS=y
```

8.1.11.3 Device Tree Configuration

The device tree defines the SDHCI controller's hardware setup. Below is a configuration snippet:

```
sdmmc:sdhci:@48102000 {
    compatible = "snps,dwcmshc-sdhci";
    reg = <0x48102000 0x1000>;
    interrupts = <GIC_SPI 27 IRQ_TYPE_LEVEL_HIGH>, <GIC_SPI 28
IRQ_TYPE_LEVEL_HIGH>;
    clocks = <&psclks ENSEMBLE_SYST_HCLK>, <&psclk ENSEMBLE_100M_CLK>;
    clock-names = "core", "bus";
    bus-width = <4>;
    max-frequency = <25000000>;
    no-hispeed;
    status = SDHCI_STATUS;
};
```

eMMC Card Validations:

Insert the eMMC 64-GB chip into the eMMC adapter.

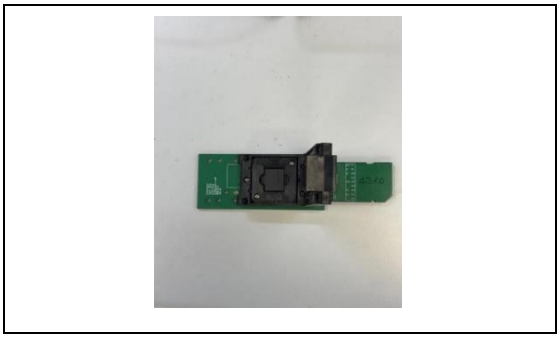


Figure 24 MMC Adapter.

11. **Prepare the SD Card**

Insert an SD card into the SD Card reader and connect it to the host machine.

g. **Identify the SD Card Device Node**

The SD card usually appears as a device node, such as /dev/sdX, on a Linux host or a Linux distribution running on Windows 11. Verify the device node by using the following commands.:

```
$ dmesg
```

or

```
$ sudo fdisk -l
```

Example: The SD card is detected as */dev/sdc.*

h. **Create an ext4 Partition**

Use fdisk to create a new partition on your SD card. Be sure to replace /dev/sdc with the correct device node for your SD card.

Command:

```
$ sudo fdisk /dev/sdc
 Welcome to fdisk (util-linux 2.37.2).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.
Command (m for help): p
Disk /dev/sdc: 58.25 GiB, 62545461248 bytes, 122159104 sectors
Disk model: SD  Transcend
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x2728257f
 Device     Boot Start      End  Sectors Size Id Type
/dev/sdc1        2048 10487807 10485760   5G 83 Linux

Command (m for help): d
Selected partition 1
Partition 1 has been deleted.
```

```
Command (m for help): n
Partition type
   p   primary (0 primary, 0 extended, 4 free)
   e   extended (container for logical partitions)
Select (default p): p
Partition number (1-4, default 1):
First sector (2048-122159103, default 2048):
Last sector, +/-sectors or +/-size{K,M,G,T,P} (2048-122159103, default
122159103):
Created a new partition 1 of type 'Linux' and of size 58.2 GiB.
Partition #1 contains a vfat signature.
Do you want to remove the signature? [Y]es/[N]o: n

Command (m for help): t

Selected partition 1
Hex code or alias (type L to list all): b
Changed type of partition 'Linux' to 'W95 FAT32'.

Command (m for help): w
The partition table has been altered.
Calling ioctl() to re-read partition table.
Syncing disks.
$ sudo mkfs.vfat /dev/sdc1
mkfs.fat 4.2 (2021-01-31)
```

2. Connect the eMMC card adapter to the micro-SD to SD adapter and then insert it into the micro-SD slot of the Eagle devkit. The indicator light on the micro-SD to SD adapter, labeled D1 LED, will glow green, confirming that the Vcc power supply is properly functioning for the eMMC adapter.

Figure 26 eMMC

3. Boot the Image build with apss-sd-share:-

```
root@devkit-e8:~# cd /var/volatile/
root@devkit-e8:/var/volatile# mkdir sd
root@devkit-e8:/var/volatile# mount /dev/mmcblk0p1 sd/
FAT-fs (mmcblk0p1): Volume was not properly unmounted. Some data may be
corrupt. Please run fsck.
root@devkit-e8:/var/volatile# mount
mtd:physmap-flash.0 on / type cramfs (ro,relatime)
proc on /proc type proc (rw,relatime)
sysfs on /sys type sysfs (rw,relatime)
devtmpfs on /dev type devtmpfs
(rw,relatime,size=3716k,nr_inodes=929,mode=755)
devpts on /dev/pts type devpts (rw,relatime,mode=600,ptmxmode=000)
tmpfs on /run type tmpfs (rw,nosuid,nodev,mode=755)
tmpfs on /var/volatile type tmpfs (rw,relatime)
/dev/mmcblk0p1 on /var/volatile/sd type vfat
(rw,relatime,fmask=0022,dmask=0022,codepage=437,iocharset=iso8859-
1,shortname=mi)
root@devkit-e8:/var/volatile# cd sd
root@devkit-e8:/var/volatile/sd# dd if=/dev/zero of=testfile bs=1024
count=1024
1024+0 records in
```

```
1024+0 records out
1048576 bytes (1.0MB) copied, 0.287792 seconds, 3.5MB/s
root@devkit-e8:/var/volatile/sd# dd if=/dev/zero of=testfile bs=1024
count=10240
10240+0 records in
10240+0 records out
10485760 bytes (10.0MB) copied, 1.080789 seconds, 9.3MB/s
root@devkit-e8:/var/volatile/sd# dd if=/dev/zero of=testfile bs=1024
count=102400
102400+0 records in
102400+0 records out
104857600 bytes (100.0MB) copied, 11.326372 seconds, 8.8MB/s
root@devkit-e8:/var/volatile/sd#
```

a. Boot the image from the eMMC partition on the Eagle devkit. Mount the eMMC card partition to read and write to it.

```
#sudo fdisk /dev/sdc
Welcome to fdisk (util-linux 2.34). Changes will remain in memory only,
until you decide to write them. Be careful before using the write
command.
Command (m for help): n
Partition type
p primary (0 primary, 0 extended, 4 free)
e extended (container for logical partitions)
Select (default p):
Using default response p.
Partition number (1-4, default 1):
First sector (2048-122159103, default 2048):
Last sector, +/-sectors or +/-size{K,M,G,T,P} (2048-122159103, default
122159103): +4G
Created a new partition 1 of type 'Linux' and of size 4 GiB.
Partition #1 contains a vfat signature.
Do you want to remove the signature? [Y]es/[N]o: Y
The signature will be removed by a write command.
Command (m for help): w
The partition table has been altered.
```

```
Calling ioctl() to re-read partition table.
Syncing disks.
#sudo fdisk -l /dev/sdc
Disk /dev/sdc: 58.26 GiB, 62545461248 bytes, 122159104 sectors
Disk model: SD Transcend
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x2728257f
Device       Boot Start      End        Sectors     Size     Id     Type
/dev/sdc1    2048           8390655  8388608  4G       83    Linux
#sudo mkfs.ext4 -L root /dev/sdc1
mke2fs 1.45.5 (07-Jan-2020) Creating filesystem with 1048576 4k blocks
and 262144 inodes
Filesystem UUID: 40e827db-d411-4829-9df2-c0b29b6713ff
Superblock backups stored on blocks:
32768, 98304, 163840, 229376, 294912, 819200, 884736
Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done
#sudo dd if=/opt/app_release_107/app-release-exec-linux-
SE_FW_1.107.00_DEV/app-release-exec-linux/build/images/alif-tiny-image-
devkit-e8.rootfs.ext4 of=/dev/sdc1
131072+0 records in
131072+0 records out
67108864 bytes (67 MB, 64 MiB) copied, 9.05336 s, 7.4 MB/s
root@devkit-e8:~# cd /var/volatile/
root@devkit-e8:/var/volatile# mkdir sd
root@devkit-e8:/var/volatile# mount /dev/mmcblk0p1 sd/
FAT-fs (mmcblk0p1): Volume was not properly unmounted. Some data may be
corrupt. Please run fsck.
root@devkit-e8:/var/volatile# mount
mtd:physmap-flash.0 on / type cramfs (ro,relatime)
```

```
proc on /proc type proc (rw,relatime)
sysfs on /sys type sysfs (rw,relatime)
devtmpfs on /dev type devtmpfs
(rw,relatime,size=3716k,nr_inodes=929,mode=755)
devpts on /dev/pts type devpts (rw,relatime,mode=600,ptmxmode=000)
tmpfs on /run type tmpfs (rw,nosuid,nodev,mode=755)
tmpfs on /var/volatile type tmpfs (rw,relatime)
/dev/mmcblk0p1 on /var/volatile/sd type vfat
(rw,relatime,fmask=0022,dmask=0022,codepage=437,iocharset=iso8859-
1,shortname=mi)
root@devkit-e8:/var/volatile# cd sd
root@devkit-e8:/var/volatile/sd# dd if=/dev/zero of=testfile bs=1024
count=1024
1024+0 records in
1024+0 records out
1048576 bytes (1.0MB) copied, 0.287792 seconds, 3.5MB/s
root@devkit-e8:/var/volatile/sd# dd if=/dev/zero of=testfile bs=1024
count=10240
10240+0 records in
10240+0 records out
10485760 bytes (10.0MB) copied, 1.080789 seconds, 9.3MB/s
root@devkit-e8:/var/volatile/sd# dd if=/dev/zero of=testfile bs=1024
count=102400
102400+0 records in
102400+0 records out
104857600 bytes (100.0MB) copied, 11.326372 seconds, 8.8MB/s
root@devkit-e8:/var/volatile/sd#
4. Boot the Image build with apss-sd-boot:-
Waiting for root device /dev/mmcblk0p1...
mmc0: new high speed MMC card at address 0001
mmcblk0: mmc0:0001 00064G 58.3 GiB
mmcblk0: p1
mmcblk0boot0: mmc0:0001 00064G 4.00 MiB
mmcblk0boot1: mmc0:0001 00064G 4.00 MiB
mmcblk0rpmb: mmc0:0001 00064G 4.00 MiB, chardev (250:0)
```

```
EXT4-fs (mmcblk0p1): mounted filesystem 560bb132-826e-404a-aee6-
25db56919bcc r/w with ordered data mode. Quota mode: disabled.
VFS: Mounted root (ext4 filesystem) on device 179:1.
Freeing unused kernel image (initmem) memory: 44K
This architecture does not have kernel memory protection.
Run /sbin/init as init process
with arguments: /sbin/init
with environment:
HOME=/
TERM=linux
mount: mounting sysfs on /sys failed: No such device EXT4-fs
(mmcblk0p1): re-mounted 560bb132-826e-404a-aee6-25db56919bcc r/w. Quota
mode: disabled.
 *** Press any key within 5 seconds to login into a shell prompt ***
Alif Iota - Tiny Linux Distribution 2.1.0 devkit-e8 /dev/ttyS0
devkit-e8 login: root
login[75]: root login on 'ttyS0'
root@devkit-e8:#
# cat /proc/cmdline console=ttyS0,115200n8 root=/dev/mmcblk0p1
rootfstype=ext4 rootwait rw loglevel=9 root@devkit-e8:~#
```

8.1.12 CRC

Cyclic Redundancy Check (CRC) is a widely used method to verify message integrity in the Alif DevKit's APSS Linux environment. This section explains the CRC algorithm, its implementation in the Alif CRC module, kernel configuration, and verification process. The CRC algorithm generates a digest (checksum) for a message by dividing its binary representation by a polynomial and calculating the remainder. This remainder serves as the CRC value. Different CRC types use polynomials of varying orders (8, 16, or 32), determining the result size:

- **CRC-8**: 8-bit result
- **CRC-16**: 16-bit result
- **CRC-32**: 32-bit result

8.1.12.1 Implementation

The Alif CRC module supports five CRC algorithms and integrates with the Linux kernel via a device driver. This section details the supported algorithms, hardware registers, and kernel configuration.

Supported CRC Algorithms

- **CRC-8-CCITT**
- **CRC-16**
- **CRC-16-CCITT**
- **CRC-32**
- **CRC-32C**

Hardware Registers

The Alif CRC module computes the CRC based on values in the following control registers:

- **Init Register: Selects the CRC algorithm type.**
- **Seed Register: Sets the initial seed value.**
- **Data Registers**: Accepts input data:
  - 4 bytes at a time for CRC-32.
  - 1 byte at a time for CRC-8 and CRC-16.
- **Result Register**: Provides the computed CRC output after all data is written.

8.1.12.2 Kernel Configuration

The Alif CRC module is accessed via a Linux kernel device driver registered with the crypto subsystem.

Enable the following kernel configuration options in the defconfig to build the driver into the xipImage:

```
CONFIG_INET=y
CONFIG_CRYPTO=y
CONFIG_CRYPTO_MANAGER=y
CONFIG_CRYPTO_USER=y
CONFIG_CRYPTO_CRC32C=y
CONFIG_CRYPTO_CRC32=y
CONFIG_CRYPTO_USER_API_HASH=y
CONFIG_CRYPTO_HW=y
CONFIG_CRYPTO_DEV_ALIF_CRC=y
```

8.1.12.3 Build Integration

To include CRC support in the APSS Linux project:

1. Add "apss-crc" to the DISTRO_FEATURES variable in conf/local.conf.
2. Build the project to generate:
    - xipImage with the CRC driver enabled.
    - cramfs-xip containing the CRC test program at /opt/linux_dd_test/crc/test_crc.

8.1.12.4 Verification

The Alif CRC Linux device driver can be tested using the provided test program, which interacts with the driver via the AF_ALG socket interface.

Test Program

- **Location**: /opt/linux_dd_test/crc/test_crc (available in cramfs-xip when apss-crc is enabled).
- **Purpose**: Computes the CRC for a given input string using the specified algorithm.
- **Usage**:

```
/opt/linux_dd_test/crc/test_crc <crc_type> <input_string>
```

- **Valid CRC Types**: crc32, crc32c, crc16, crc16-ccitt, crc8, crc32-alif-linux, crc32-linux, crc32c-linux

8.1.12.5 Examples

Example 1: CRC-32 Calculation

```
root@devkit-e8:~# test_crc crc32 starlight
Init
Update
result : 40437e6c
input: starlight
crc type: crc32
Result is: c1 2e 34 a5
```

Example 2: CRC-32C Calculation

```
root@devkit-e8:~# test_crc crc32c starlight
Init values
Init
Update
result : 564e87a0
input: starlight
crc type: crc32c
Result is: 46 62 6b 62
```

8.1.13 I2S

The Inter-IC Sound (I2S) Bus is a three-wire serial protocol developed by Philips for transferring stereo audio data. This section covers the I2S implementation on the Alif DevKit's APSS Linux environment, including features, kernel configuration, device tree setup, and testing.

The I2S controller in the Alif DevKit, based on the DesignWare DW_apb_i2s, supports audio transmission and reception following the Philips I2S protocol. It is optimized for embedded systems, offering flexible configurations for stereo audio processing.

8.1.13.1 Features Supported

- **Protocol**: I2S transmitter and receiver compliant with Philips I2S serial protocol.
- **Channels**: One stereo channel each for transmitter and receiver.
- **Communication**: Full duplex, with independent transmitter and receiver operation.
- **Mode**: Operates in master mode.
- **Clock Sources**: Supports 76.8 MHz internal clock or an external audio clock.
- **Sampling Frequencies**: 8, 16, 32, 44.1, 48, 88.2, 96, and 192 kHz.
- **Word-Select Clocks**: 16, 24, or 32 clocks for left/right channel selection.
- **Audio Resolution**: 12, 16, 20, 24, and 32 bits.
- **FIFO**: 16-word depth for both receiver and transmitter.
- **FIFO Thresholds**: Programmable for flexible data handling.
- **DMA**: Includes hardware handshaking interface for Direct Memory Access (DMA).
- **Bus**: 32-bit APB data bus.

Simple System Configurations for DW_apb_i2s

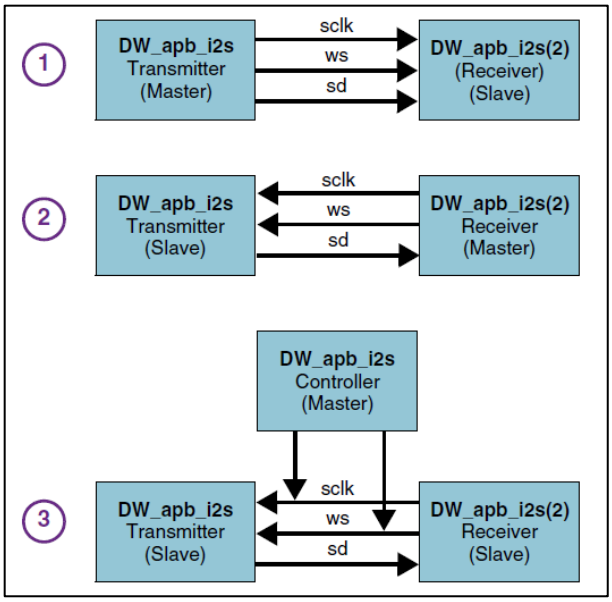


Figure 26 I2S

8.1.13.2 Kernel Configuration

Enable the following Linux kernel options to support the I2S controller:

```
CONFIG_SOUND=y
CONFIG_SND=y
CONFIG_SND_PCM=y
CONFIG_SND_SOC=y
CONFIG_SND_DESIGNWARE_I2S=y
CONFIG_SND_DESIGNWARE_PCM=y
CONFIG_SND_SOC_T5848=y
CONFIG_SND_SIMPLE_CARD_UTILS=y
CONFIG_SND_SIMPLE_CARD=y
CONFIG_IIO=y
CONFIG_ENSEMBLE_DAC=y
CONFIG_SERIAL_8250_DMA=y
CONFIG_DMADEVICES=y
CONFIG_DMA_ENGINE=y
CONFIG_DMA_OF=y
CONFIG_PL330_DMA=y
CONFIG_I2C=y
```

```
CONFIG_I2C_BOARDINFO=y
CONFIG_I2C_COMPAT=y
CONFIG_I2C_HELPER_AUTO=y
CONFIG_I2C_DESIGNWARE_PLATFORM=y
CONFIG_I2C_DESIGNWARE_CORE=y
CONFIG_REGMAP_I2C=y
CONFIG_CONFIGFS_FS=y
CONFIG_SYSFS=y
CONFIG_SYSFS_SYSCALL=y
CONFIG_SLAB=y
CONFIG_I2C_CHARDEV=y
```

8.1.13.3 Device Tree Configuration

```
sound-wm8904 {
        compatible = "simple-audio-card";
        simple-audio-card,name = "wm8904-audio";
        simple-audio-card,format = "i2s";
        simple-audio-card,frame-master = <&cpudai_0>;
        simple-audio-card,bitclock-master = <&cpudai_0>;
        simple-audio-card,widgets =
                "Headphone Jack", "Left Line Out",
                "Headphone Jack", "Right Line Out";

        cpudai_0: simple-audio-card,cpu {
                sound-dai = <&i2s3>;
        };
        acodec: simple-audio-card,codec {
                sound-dai = <&wm8904>;
        };
};
```

```
&i2c1 {
        #address-cells = <1>;
        #size-cells = <0>;
        pinctrl-names = "default";
        pinctrl-0 = <&pinctrl_i2c1>;
        status = I2C1_STATUS;
        wm8904: wm8904@1a {
                compatible = "wlf,wm8904";
                reg = <0x1a>;
                #sound-dai-cells = <0>;
                clocks = <&wm8904_mclk>;
                clock-names = "mclk";
        };
```

```
&i2s3 {
        pinctrl-names = "default";
        pinctrl-0 = <&pinctrl_i2s3>;
        status = I2S3_STATUS;
};
```

```
                i2s3: i2s3@49017000 {
                        compatible = "snps,designware-i2s";
                        reg = <0x49017000 0x1000>;
                        clocks = <&psclks ENSEMBLE_I2S3_CLK>,
                                 <&psclks ENSEMBLE_I2S3_BIT_CLK>;
                        clock-names = "pclk","i2sclk";
                        #sound-dai-cells = <0>;
/*                      interrupts = <GIC_SPI 133 IRQ_TYPE_LEVEL_HIGH>;*/
                        dmas = <&event_router 31 1>,
                               <&event_router 27 1>;
                        dma-names = "tx","rx";
                        status = "disabled";
                };
```

Configuration Notes

- **DMA Handshaking:** Enabled for I2S3 to facilitate efficient audio data transfer.
- **Disabling DMA:** To disable DMA, uncomment the interrupt line in the DTS file:

  ```
  interrupts = <GIC_SPI 133 IRQ_TYPE_LEVEL_HIGH>;
  ```

- **ALSA Support:** The ALSA arecord application can record audio from the microphone.
- **DMAs Property:** The dmas property points to the Alif event router. To configure this property, you need to pass the channel number and the corresponding event router group mapping.

8.1.13.4 Yocto Build Changes for I2S Testing

This section outlines the necessary modifications to the Yocto build configuration to enable I2S testing using ALSA tools.

Configuration Steps

1. Enable ALSA Tools for I2S Testing

To support I2S testing, ALSA tools (specifically `arecord`) must be included in the cramfs.

- Add the following line to the `local.conf` file in your Yocto build environment:

```
IMAGE_INSTALL:append = " alsa-utils-aplay"
```

Note:

Ensure there is a space before `alsa-utils-aplay` in the configuration.

---

2. Enable I2S Support in the Distribution

To build the `xipImage` with I2S functionality enabled, include the `apss-i2s` feature in the distribution configuration.

- Add the following line to the `local.conf` file:

```
DISTRO_FEATURES:append = " apss-i2s"
```

3**. Adjust Memory Configuration for MRAM (Optional)**

When testing with MRAM (and not using OSPI), including ALSA tools may increase the size of the cramfs, which could lead to memory issues with the default Yocto build addresses. To address this, configure the following memory settings in the `local.conf` file:

```
KERNEL_MTD_START_ADDR = "0x80310000"
KERNEL_MTD_LEN = "0x400000"
```

8.1.13.5 Test Summary

To test the I2S functionality, play audio from any storage device, such as an SD card. If PDM is enabled, record audio using PDM and save it to the storage device. The audio can then be played through I2S and the WM8904 audio jack using the APLAY application.

Steps

1. **Mount an SD Card**: Insert an SD card to store the recorded audio file (if **PDM** is enabled), or copy an audio file to the SD card before mounting it.
2. **Navigate to the Mounted Directory**: Change to the SD card's mount point (e.g., /mnt/sdcard).
3. **Record Audio**: Run the following aplay command:

```
aplay -Dhw:0,0 -c2 -r48000 -fS32_LE -twav -d10 -Vstereo test2.wav
```

1. Dhw:0,0: Specifies the I2S codec device.
2. c2: Sets 2 channels (stereo).
3. r48000: Uses a 48 kHz sampling rate.
4. fS32_LE: Sets 32-bit little-endian format.
5. twav: Input file is a WAV file.
6. d10: play for 10 seconds.
7. Vstereo: Enables stereo visualization.
8. Test2.wav: Input file to be played.

```
root@devkit-e7:/var/volatile/sd# aplay -Dhw:0,0 -c2 -r48000 -fS32_LE -twav -d10 -Vstereo i2s_dma.wav
Playing WAVE 'i2s_dma.wav' : Signed 32 bit Little Endian, Rate 48000 Hz, Stereo
                                + 06%|11%# +                                root@devkit-e7:/var/volatile/sd#
```

If I2S and PDM both enabled, then on boot logs will list both the ALSA devices as shown below.

```
mmc0: SDHCI controller on 48102000.sdhci [48102000.sdhci] using ADMA
clk: Disabling unused clocks
ALSA device list:
  #0: alifpcm
  #1: wm8904-audio
dw-apb-uart 4901a000.serial: forbid DMA for kernel console
```

While using arecord or aplay select the hardware accordingly. For PDM

```
arecord -Dhw:0,0 -c2 -r8000 -fS16_LE -twav -d10 eagle_pdm_8k_ch5.wav
for I2S
aplay -Dhw:1,0 -c1 -r8000 -fS32_LE -twav -d10 -Vstereo eagle_pdm_8k_ch5.wav
```

8.1.14 DAC12

The Digital-to-Analog Converter (DAC12) module in the Alif DevKit converts 12-bit digital values into analog voltage signals. This section describes the DAC12 module, its features, kernel configuration, and setup in the APSS Linux environment.

The DAC12 module outputs analog voltages between 0 V and 1.8 V in Low Power (LP) mode. The device includes two DAC12 modules, each designed for high-performance digital-to-analog conversion in embedded systems.

8.1.14.1 Features Supported

- Conversion Rate: Up to 1 MHz at 12-bit resolution.
- Data Format: Supports unsigned binary or two's complement signed digital data.
- Output Current: Programmable up to 1.5 mA.
- Load Capacitance: Includes programmable compensation for stable output.
- Voltage Reference: Internal 1.8-V reference for consistent performance.
- Power Supply Rejection Ratio (PSRR): Excellent rejection of high-frequency noise.
- Maximum Current: Capable of delivering up to 1.5 mA.

8.1.14.2 Required Kernel Configuration

Enable the following Linux kernel options to support the DAC12 module:

```
CONFIG_IIO=y
CONFIG_ENSEMBLE_DAC=y
```

8.1.14.3 Device Tree Configuration

```
        dac0: dac12_0@49028000 {
                compatible = "alif,ensemble-dac";
                reg-names = "dac", "cmp","analog";
                reg = <0x49028000 0x10>,
                        <0x49023000 0x40>,
                        <0x1A60A000 0x40>;
                clocks = <&psclks ENSEMBLE_DAC120_CLK>,
                        <&psclks ENSEMBLE_CMP0_CLK>;
                clock-names = "dac_clk", "cmp_clk";
                instance = <0>;
                status = "disabled";
        };

        dac1: dac12_1@49029000 {
                compatible = "alif,ensemble-dac";
                reg-names = "dac";
                reg = <0x49029000 0x10>;
                clocks = <&psclks ENSEMBLE_DAC121_CLK>,
                        <&psclks ENSEMBLE_CMP0_CLK>;
                clock-names = "dac_clk", "cmp_clk";
                instance = <1>;
                status = "disabled";
        };
```

```
&dac0 {
        status = DAC120_STATUS;
};

&dac1 {
        status = DAC121_STATUS;
};
```

8.1.14.4 Test Output

```
root@appkit-e7:~# cat /sys/bus/iio/devices/iio\:device0/out_voltage_raw
0
root@appkit-e7:~# echo 1000 > /sys/bus/iio/devices/iio\:device0/out_voltage_raw
root@appkit-e7:~# cat /sys/bus/iio/devices/iio\:device0/out_voltage_raw
1000
root@appkit-e7:~#
```

8.1.15 ADC12/ADC24

The Analog-to-Digital Converter (ADC) modules are 12-bit/24-bit multi-input units used for analog signals conversion into digital values. The device includes three ADC12 modules and one ADC24 module.

ADC12 modules support the following main features:

- 12-bit Successive Approximation Register (SAR) ADC
- Conversion rate of up to 1.25 MSPS

- 6 external inputs and two internal inputs:
  - external pins may be configured to 6 single-ended or 3 differential inputs
  - One input from the on-chip temperature sensor (TSENS)
  - One input from internal voltage reference

8.1.15.1 Features Supported

The ADC24 supports the following key features:

- 24-bit Sigma-Delta ADC
- Conversion rate of up to 16 kSPS
- 4 external differential inputs

The diagrams below illustrate the components involved, their connections, and the flow of data in both a 12-bit and a 24-bit ADC.

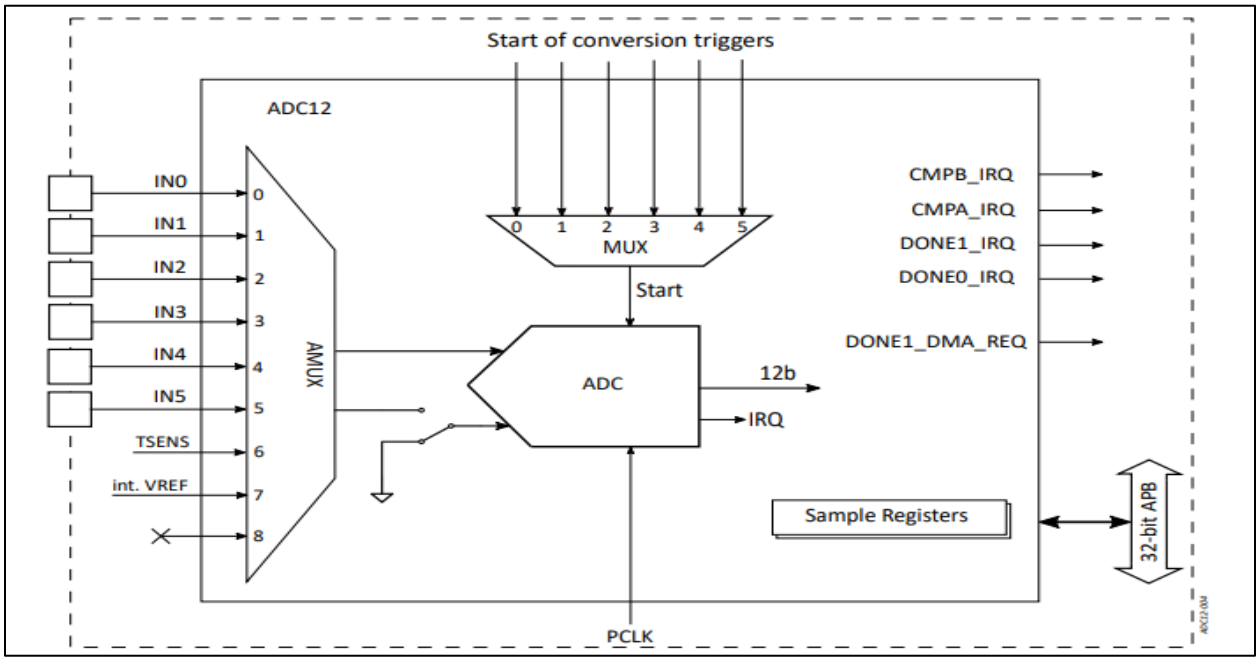


Figure 27: ADC12 Block Diagram

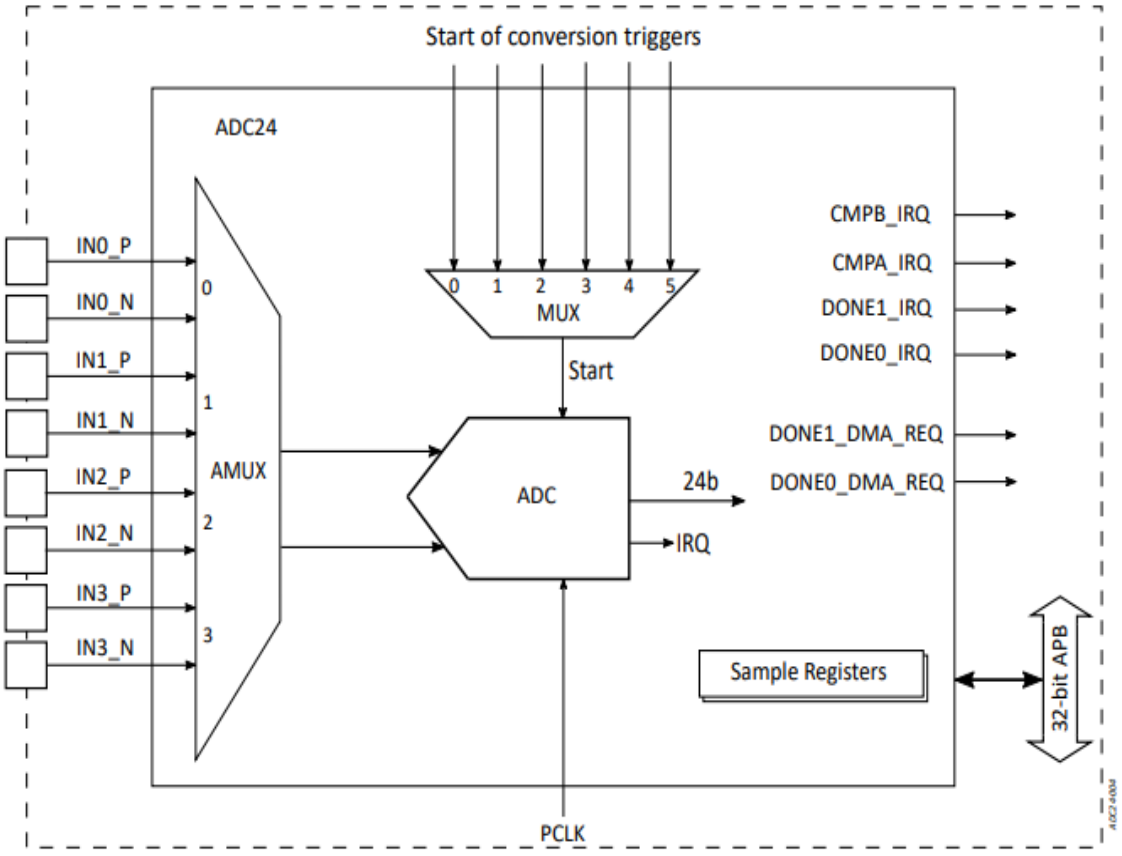


Figure 28: ADC24 Block Diagram

8.1.15.2 Kernel Configurations

```
CONFIG_IIO=y
CONFIG_IIO_CONFIGFS=y
CONFIG_ENSEMBLE_ADC=y
CONFIG_SLAB=y
```

8.1.15.3 Device Tree Configuration

```
adc0: adc12_0@49020000 {
                        compatible = "alif,adc-dev";
             reg-names = "adc_base";
                        reg = <0x49020000 0x100>;
                        interrupt-parent = <&gic>;
                        interrupts = <0 140 IRQ_TYPE_LEVEL_HIGH>,
<0 143 IRQ_TYPE_LEVEL_HIGH>,
<0 146 IRQ_TYPE_LEVEL_HIGH>,
```

```
<0 149 IRQ_TYPE_LEVEL_HIGH>;
                        clocks = <&psclks ENSEMBLE_ADC120_CLK>;
            clock-names = "adc_clk";
                        status = "disabled";
               };

  adc1: adc12_1@49021000 {
                        compatible = "alif,adc-dev";
            reg-names = "adc_base";
                        reg = <0x49021000 0x100>;
                        interrupt-parent = <&gic>;
                        interrupts = <0 141 IRQ_TYPE_LEVEL_HIGH>,
<0 144 IRQ_TYPE_LEVEL_HIGH>,
<0 147 IRQ_TYPE_LEVEL_HIGH>,
<0 150 IRQ_TYPE_LEVEL_HIGH>;
                        clocks = <&psclks ENSEMBLE_ADC121_CLK>;
            clock-names = "adc_clk";
                        status = "disabled";
               };
adc2: adc12_2@49022000 {
                        compatible = "alif,adc-dev";
            reg-names = "adc_base";
                        reg = <0x49022000 0x100>;
                        interrupt-parent = <&gic>;
                        interrupts = <0 142 IRQ_TYPE_LEVEL_HIGH>,
<0 145 IRQ_TYPE_LEVEL_HIGH>,
<0 148 IRQ_TYPE_LEVEL_HIGH>,
<0 151 IRQ_TYPE_LEVEL_HIGH>;
                        clocks = <&psclks ENSEMBLE_ADC122_CLK>;
           clock-names = "adc_clk";
                        status = "disabled";
               };
adc24: adc24@49027000 {
                        compatible = "alif,adc-dev";
            reg-names = "adc_base";
                        reg = <0x49027000 0x100>;
```

```
                        interrupt-parent = <&gic>;
                        interrupts = <0 152 IRQ_TYPE_LEVEL_HIGH>,
<0 153 IRQ_TYPE_LEVEL_HIGH>,
<0 154 IRQ_TYPE_LEVEL_HIGH>,
<0 155 IRQ_TYPE_LEVEL_HIGH>;
                        clocks = <&psclks ENSEMBLE_ADC122_CLK>;
             clock-names = "adc_clk";
                        status = "disabled";
                };
```

8.1.15.4 Test Output

ADC12 Output

```
root@devkit-e8:/sys/bus/iio/devices# ls -l
lrwxrwxrwx    1 root     root             0 Jan  1 00:00 iio:device0 ->
../../../devices/platform/49020000.apb/49020000.adc12_0/iio0
lrwxrwxrwx    1 root     root             0 Jan  1 00:00 iio:device1 ->
../../../devices/platform/49020000.apb/49021000.adc12_1/iio1
lrwxrwxrwx    1 root     root             0 Jan  1 00:00 iio:device2 ->
../../../devices/platform/49020000.apb/49022000.adc12_2/iio2
```

ADC12_0: Reading Internal Channel-6 Temperature Sensor and Channel-7 Reference Voltage as Input Channels

```
root@devkit-e8:/sys/bus/iio/devices# cd iio\:device0
root@devkit-e8:/sys/bus/iio/devices# cat in_temp_raw
1125
root@devkit-
e8:/sys/devices/platform/49020000.apb/49020000.adc12_0/iio:device0# cat
in_temp_scale
Temperature: 29.5°C
```

```
 root@devkit-
e8:/sys/devices/platform/49020000.apb/49020000.adc12_0/iio:device0# cat
in_voltage7_raw
4093
root@devkit-
e8:/sys/devices/platform/49020000.apb/49020000.adc12_0/iio:device0# cat
in_voltage_scale
1.758974358
```

To check the external channel voltage, connect the output channels listed below to a voltage meter.

```
Channel0:   P0_0 -->   J8.pin2
Channel1:   P0_1 -->   J8.pin4
Channel2:   P0_2 -->   J8.pin3
Channel3:   P0_3 -->   J8.pin5
Channel4:   P0_4 -->   J8.pin7
Channel5:   P0_5 -->   J8.pin9
```

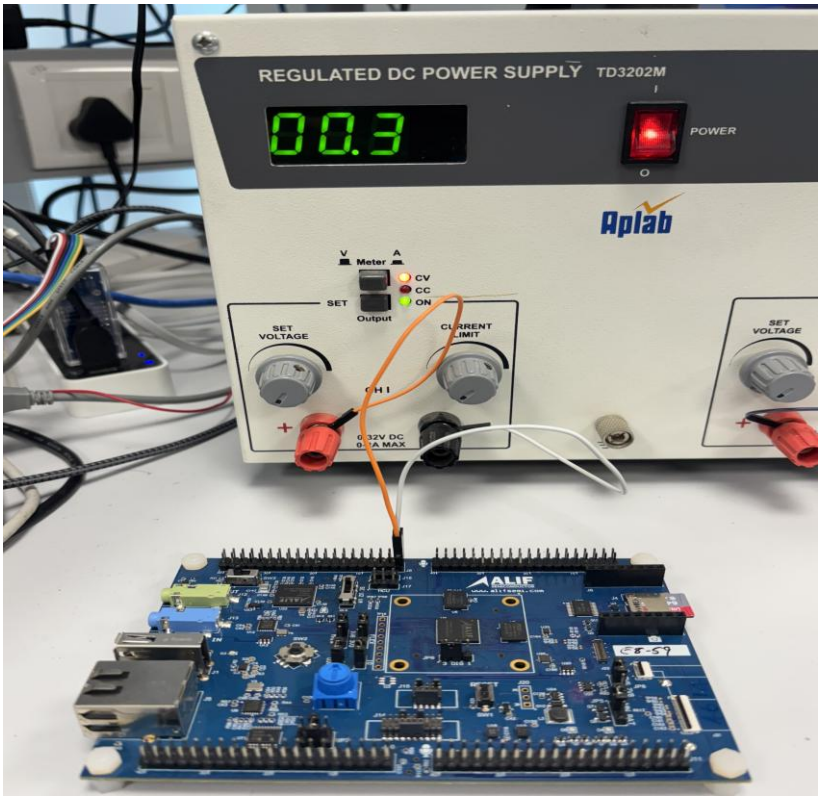


Figure 29: Setting up the ADC12 for verifying single-ended conversions from an external input source.

Now execute the command to validate the raw and scale values of all six external channels.

Channel0:

```
root@devkit-
e8:/sys/devices/platform/49020000.apb/49020000.adc12_0/iio:device0# cat
in_voltage0_raw
root@devkit-
e8:/sys/devices/platform/49020000.apb/49020000.adc12_0/iio:device0# cat
in_voltage_scale
```

Channel1:-

```
root@devkit-
e8:/sys/devices/platform/49020000.apb/49020000.adc12_0/iio:device0# cat
in_voltage1_raw
root@devkit-
e8:/sys/devices/platform/49020000.apb/49020000.adc12_0/iio:device0# cat
in_voltage_scale
```

Channel2: -

```
root@devkit-
e8:/sys/devices/platform/49020000.apb/49020000.adc12_0/iio:device0# cat
in_voltage2_raw
root@devkit-
e8:/sys/devices/platform/49020000.apb/49020000.adc12_0/iio:device0# cat
in_voltage_scale
```

Channel3: -

```
root@devkit-
e8:/sys/devices/platform/49020000.apb/49020000.adc12_0/iio:device0# cat
in_voltage3_raw
root@devkit-
e8:/sys/devices/platform/49020000.apb/49020000.adc12_0/iio:device0# cat
in_voltage_scale
```

Channel4: -

```
root@devkit-
e8:/sys/devices/platform/49020000.apb/49020000.adc12_0/iio:device0# cat
in_voltage4_raw
root@devkit-
e8:/sys/devices/platform/49020000.apb/49020000.adc12_0/iio:device0# cat
in_voltage_scale
```

Channel5: -

```
root@devkit-
e8:/sys/devices/platform/49020000.apb/49020000.adc12_0/iio:device0# cat
in_voltage5_raw
root@devkit-
e8:/sys/devices/platform/49020000.apb/49020000.adc12_0/iio:device0# cat
in_voltage_scale
```

To enable differential mode, follow these steps and connect output channels in pairs to the voltage meter.

```
root@devkit-
e8:/sys/devices/platform/49020000.apb/49020000.adc12_0/iio:device0# echo 1 >
differential_var
```

Channel0: P0_0-->J8.pin2 and P0_1-->J8.pin4
Channel1: P0_2-->J8.pin3 and P0_3-->J8.pin5
Channel2: P0_4-->J8.pin7 and P0_5-->J8.pin9

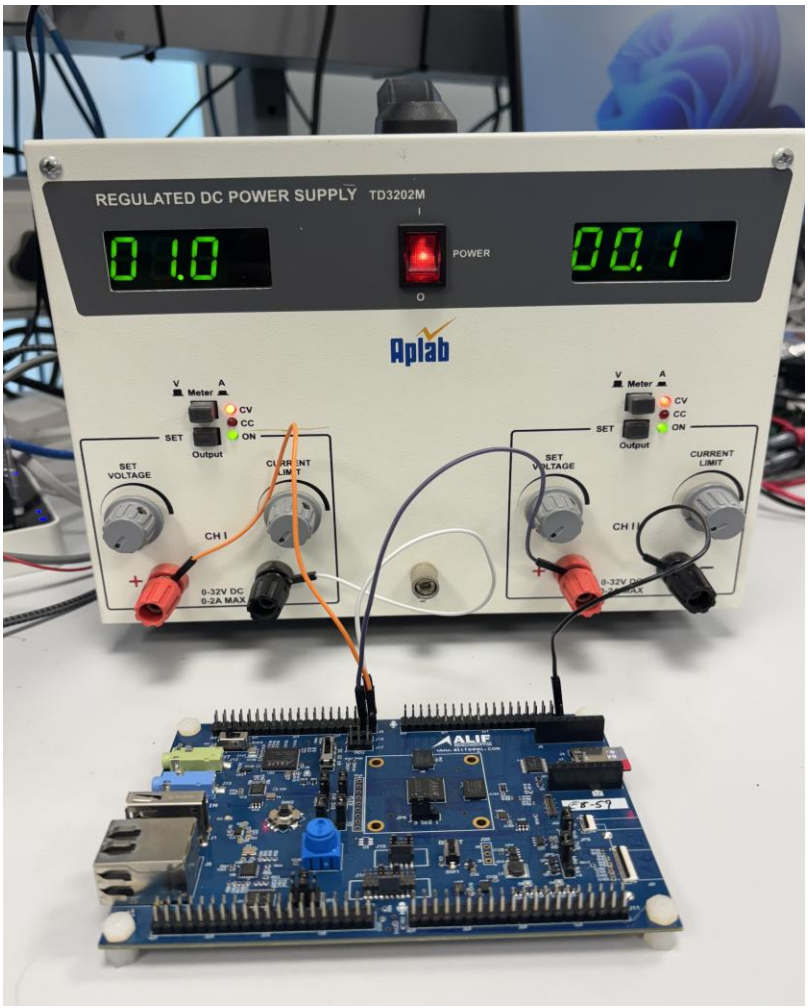


Figure 29: ADC12 Setup for checking differential input conversion from external input source

Channel0:

```
root@devkit-e8:/sys/devices/platform/49020000.apb/49020000.adc12_0/iio:device0# cat in_voltage0_raw
root@devkit-e8:/sys/devices/platform/49020000.apb/49020000.adc12_0/iio:device0# cat in_voltage_scale
```

Channel1:

```
root@devkit-e8:/sys/devices/platform/49020000.apb/49020000.adc12_0/iio:device0# cat in_voltage2_raw
```

```
root@devkit-
e8:/sys/devices/platform/49020000.apb/49020000.adc12_0/iio:device0# cat
in_voltage_scale

Channel2:
root@devkit-
e8:/sys/devices/platform/49020000.apb/49020000.adc12_0/iio:device0# cat
in_voltage4_raw
root@devkit-
e8:/sys/devices/platform/49020000.apb/49020000.adc12_0/iio:device0# cat
in_voltage_scale
```

ADC12_1: Reading Internal Channel-7 Reference Voltage as Input Channels

```
 root@devkit-
e8:/sys/devices/platform/49020000.apb/49021000.adc12_1/iio:device1# cat
in_voltage7_raw
4092
root@devkit-
e7:/sys/devices/platform/49020000.apb/49021000.adc12_1/iio:device1# cat
in_voltage_scale
1.758485958
```

To check the external channel voltage, please connect the specified output channels to a voltmeter.

Channel 0: P0_6 → J8.pin8
(Note: You need to connect the click board to J8.pin8 to measure the correct voltage.)
Channel 1: P1_2 → J8.pin13
Channel 2: P1_3 → J8.pin15

Channel0:

```
root@devkit-
e8:/sys/devices/platform/49020000.apb/49021000.adc12_1/iio:device1# cat
in_voltage0_raw
root@devkit-
e8:/sys/devices/platform/49020000.apb/49021000.adc12_1/iio:device1# cat
in_voltage_scale
```

Channel1:-

```
root@devkit-
e8:/sys/devices/platform/49020000.apb/49021000.adc12_1/iio:device1# cat
in_voltage4_raw
root@devkit-
e8:/sys/devices/platform/49020000.apb/49021000.adc12_1/iio:device1# cat
in_voltage_scale
```

Channel2:-

```
root@devkit-
e8:/sys/devices/platform/49020000.apb/49021000.adc12_1/iio:device1# cat
in_voltage5_raw
root@devkit-
e8:/sys/devices/platform/49020000.apb/49021000.adc12_1/iio:device1# cat
in_voltage_scale
```

ADC12_2: Reading the temperature from the Internal Channel-6 sensor and the reference voltage from Channel-7 as input channels.

```
root@devkit-
e8:/sys/devices/platform/49020000.apb/49022000.adc12_2/iio:device2# cat
in_temp_raw
1125
root@devkit-
e8:/sys/devices/platform/49020000.apb/49022000.adc12_2/iio:device2# cat
in_temp_scale
Temperature: 29.5°C

root@devkit-
e8:/sys/devices/platform/49020000.apb/49022000.adc12_2/iio:device2# cat
in_voltage7_raw
4092
root@devkit-
e8:/sys/devices/platform/49020000.apb/49022000.adc12_2/iio:device2# cat
in_voltage_scale
1.758485958
```

Note:

To check the voltage of external channels, connect the output channels to a voltmeter.

Channel 0: P1_4 --> J8. pin17

```
Channel0:
root@devkit-
e8:/sys/devices/platform/49020000.apb/49022000.adc12_2/iio:device2# cat
in_voltage0_raw
root@devkit-
e8:/sys/devices/platform/49020000.apb/49022000.adc12_2/iio:device2# cat
in_voltage_scale
```

ADC24 Output

To measure the external channel voltage, connect the differential channels to a voltmeter.

1 **Channel 0**: P0_0 → J8.pin2 and P0_4 → J8.pin7
2 **Channel 1**: P0_1 → J8.pin4 and P0_5 → J8.pin9

(Note: You need to connect the click board to J8.pin8 and J8.pin10 to measure the correct voltage.)

3 **Channel 2**: P0_2 → J8.pin3 and P0_6 → J8.pin8
4 **Channel 3**: P0_3 → J8.pin5 and P0_7 → J8.pin10

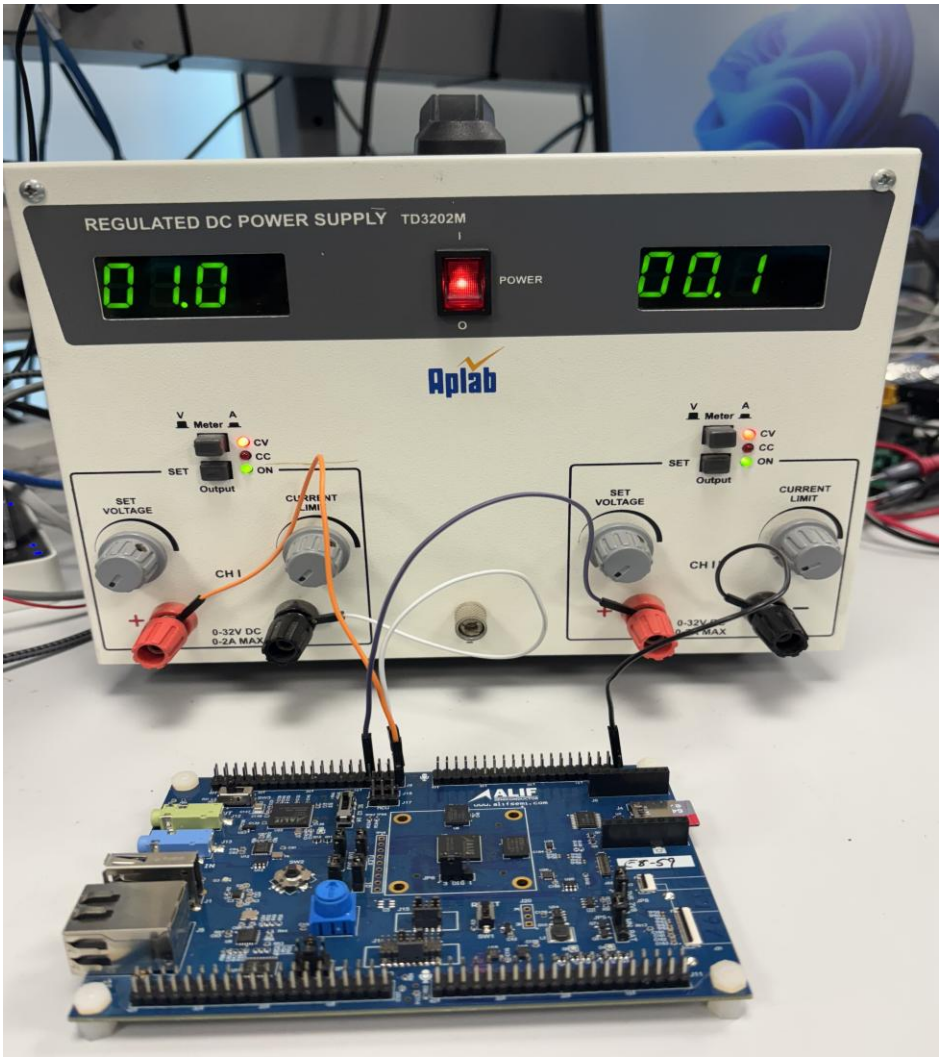


Figure 30: ADC24 Setup for checking differential input conversion from external input source

```
Channel0:
root@devkit-e8:/sys/devices/platform/49020000.apb/49027000.adc24/iio:device0#
cat in_voltage0-voltage4_raw
```

```
4855147
root@devkit-
e8:/sys/devices/platform/49020000.apb/49027000.adc24/iio:device30# cat
in_voltage-voltage_scale
0.509325177

Channel1:
root@devkit-e8:/sys/devices/platform/49020000.apb/49027000.adc24/iio:device0#
cat in_voltage1-voltage5_raw
root@devkit-
e8:/sys/devices/platform/49020000.apb/49027000.adc24/iio:device30# cat
in_voltage-voltage_scale

Channel2:
root@devkit-e8:/sys/devices/platform/49020000.apb/49027000.adc24/iio:device0#
cat in_voltage2-voltage6_raw
root@devkit-
e8:/sys/devices/platform/49020000.apb/49027000.adc24/iio:device30# cat
in_voltage-voltage_scale

Channel3:
root@devkit-e8:/sys/devices/platform/49020000.apb/49027000.adc24/iio:device0#
cat in_voltage3-voltage8_raw
root@devkit-
e8:/sys/devices/platform/49020000.apb/49027000.adc24/iio:device30# cat
in_voltage-voltage_scale
```

8.1.16 SPI ENV3-CLICK Temperature Sensor

The ENV3-CLICK chemical sensor with an SPI interface measures device temperature. Readings are transmitted via a Motorola SPI interface.

8.1.16.1 Features

- Uses DWC SSI Master Driver
- Measures temperatures from -40 °C to +85 °C
- Data read/write via Motorola Serial Peripheral Interface (SPI)

- Thermometer accuracy: ±2.0 °C

8.1.16.2 Verification

After booting Linux on the APSS, run this command on the target to read real-time temperature:

```
root@devkit-e8:~# cat
/sys/class/spi_master/spi3/spi3.0/iio:device0/in_temp_input
```

Example output: 26.3125 °C

8.1.16.3 Summary of the Test

1. ENV3-CLICK sensor connects as a client to DWC SPI (instances 0–3), using spi3.
2. Registered interrupt for DWC SPI3 is 163. Interrupt log:

```
root@devkit-e8:~# cat /proc/interrupts
CPU0 33: 96 GIC-0 129 Level 48103000.spi, 48104000.spi, 48105000.spi,
48106000.spi
```

3. Reads real-time temperature; values change over time with subsequent reads.
4. Configured for 16-bit resolution mode to detect minimal temperature changes.
5. Operates in Forced Mode for each temperature reading.

8.1.16.4 Kernel Configurations

```
CONFIG_SPI=y
CONFIG_SPI_MASTER=y
CONFIG_SPI_DESIGNWARE=y
CONFIG_SPI_DW_MMIO=y
CONFIG_BME680=y
CONFIG_BME680_SPI=y
CONFIG_IIO=y
CONFIG_IIO_CONFIGFS=y
CONFIG_HWSEM_ALIF=n
CONFIG_MAILBOX=n
CONFIG_ARM_MHUv2=n
CONFIG_RPMSG=n
CONFIG_SLAB=y
CONFIG_BLK_DEV_NULL_BLK=y
CONFIG_CONFIGFS_FS=y
```

8.1.17 CMP

The High-Speed Comparator (CMP) module is a rail-to-rail, multi-input, analog comparator with programmable reference voltage and hysteresis. The device includes up to four CMP modules.

Each CMP module supports:

- Reference voltage from DAC6, internal Vref, or external pins
- Programmable hysteresis
- Comparator result inverter
- Configurable number of taps for filtering
- Interrupt generation after filtering
- Response time: < 5 ns
- Power supply from internal 1.8-V LDO (LDO-5)

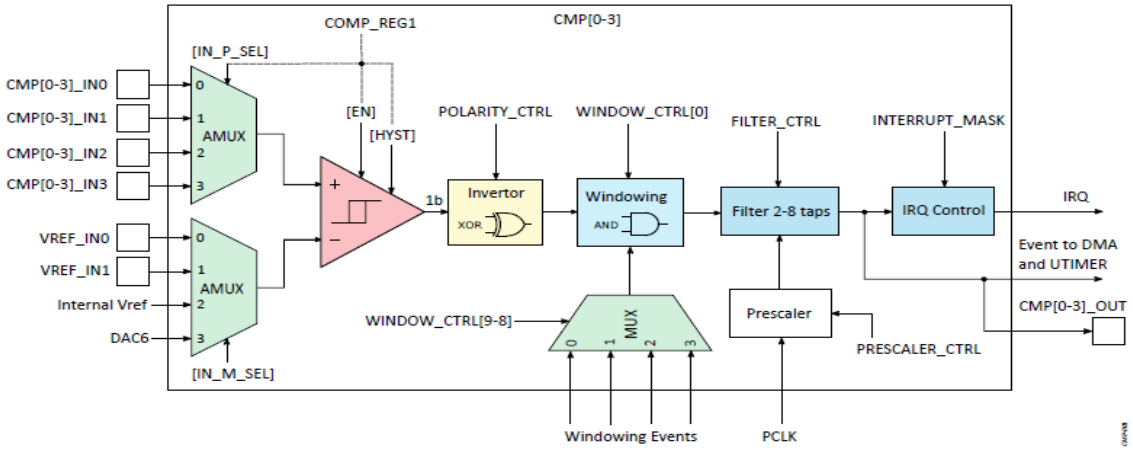


Figure 7 High Speed Comparator Block Diagram

The core of the module is an analog comparator equipped with programmable hysteresis. It is designed to compare two voltage levels:

- **Positive input (+)**: The voltage to evaluate.
- **Negative input (-)**: The reference voltage.

The module outputs:

- A logic '1' if the positive input voltage is higher than the negative input.
- A logic '0' if the positive input voltage is lower.

If input voltages are very close, the output may switch rapidly. Programmable hysteresis can stabilize the output.

Input Options

Each CMP module can compare one of four external signals against:

- Internal Vref.
- DAC6 output.
- Two external reference pins.

Monitoring Modes

Inputs can be monitored:

- **Continuously**: For real-time comparisons.
- **In time frames**: Defined by external modules like QEC inputs or UTIMER intervals.

Output Filtering and Events

The output can be filtered to remove high-frequency glitches using a configurable number of taps. When the filtered output changes (from 0 to 1 or 1 to 0):

- An interrupt may be generated.
- The result is sent to the device event router and to the CMP_STATUS register.

The comparator result inverter can reverse the output logic if needed.

8.1.17.1 Kernel Configurations

To enable the cmp module, ensure the following kernel settings:

```
CONFIG_ALIF_COMPARATOR=y
CONFIG_SYSFS=y
```

8.1.17.2 Device Tree Configuration

The device tree entries for the four CMP modules are as follows:

```
cmp0: cmp0@49023000 {
compatible = "alif,cmp-module";
reg = <0x49023000 0x1000>;
reg-names = "cmp_reg";
interrupt-parent = <&gic>;
interrupts = <0 156 IRQ_TYPE_LEVEL_HIGH>;
clocks = <&psclks ENSEMBLE_CMP0_CLK>;
led-gpios = <&port13 3 GPIO_ACTIVE_HIGH>;
instance = <0>;
status = "disabled";
```

```
};
cmp1: cmp1@49024000 {
 compatible = "alif,cmp-module";
 reg = <0x49024000 0x1000>;
 reg-names = "cmp_reg";
 interrupt-parent = <&gic>;
 interrupts = <0 157 IRQ_TYPE_LEVEL_HIGH>;
 clocks = <&psclks ENSEMBLE_CMP1_CLK>;
 instance = <1>;
 status = "disabled";
         };

            cmp2: cmp2@49025000 {
                    compatible = "alif,cmp-module";
                    reg = <0x49025000 0x1000>;
                    reg-names = "cmp_reg";
                    interrupt-parent = <&gic>;
                    interrupts = <0 158 IRQ_TYPE_LEVEL_HIGH>;
                    clocks = <&psclks ENSEMBLE_CMP2_CLK>;
                    instance = <2>;

                    status = "disabled";
            };

            cmp3: cmp3@49026000 {
                    compatible = "alif,cmp-module";
                    reg = <0x49026000 0x1000>;
                    reg-names = "cmp_reg";
                    interrupt-parent = <&gic>;
                    interrupts = <0 159 IRQ_TYPE_LEVEL_HIGH>;
                    clocks = <&psclks ENSEMBLE_CMP3_CLK>;
                     instance = <3>;
                    status = "disabled";
            };
```

8.1.17.3 Observation

LED ON: GPIO LED-D4 glows blue when the positive input is greater than the negative input.

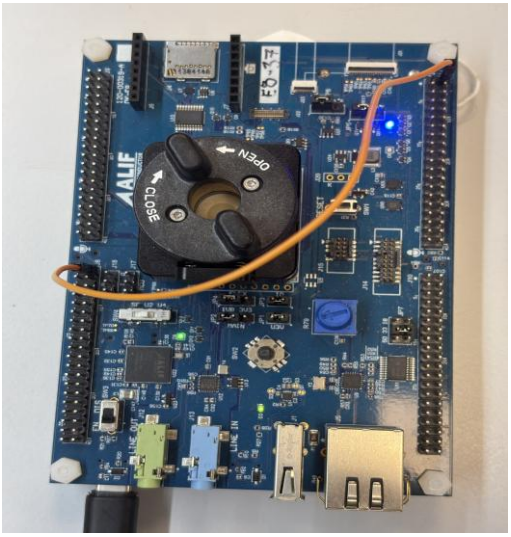


Figure 8 : When Positive Input is Greater than Negative Input, GPIO LED D4 turns ON

LED OFF: GPIO LED-D4 turns off when the negative input is greater than the positive input.

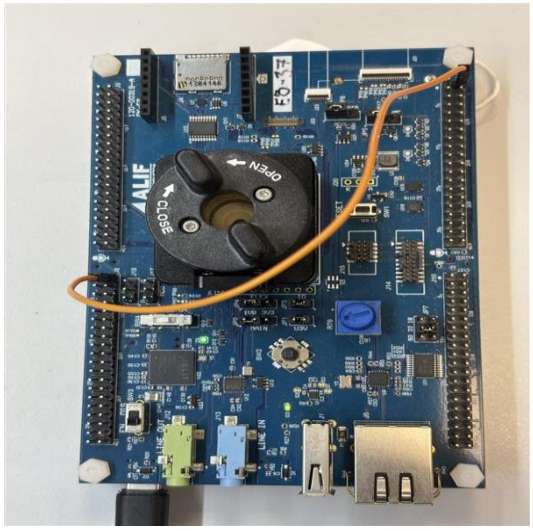


Figure 33: When Negative Input is Greater than Positive Input, GPIO LED D4 turns OFF

Fig: Board setup, Gpio LED-D4 will off when -ve input > +ve input

8.1.17.4 Verification

After booting Linux on the APSS, follow these steps to validate CMP0 to CMP3. The expected output is that GPIO LED-D4 will glow blue when the positive input voltage is greater than the negative input and turn off when the positive input voltage is less than the negative input.

1. CMP0 Verification

a. CMP0_IN0

- Connect a female-to-female jumper wire from P0_0(J8.P2) to P12_0(J11.P2)
- Run

```
# echo 1 > /sys/bus/platform/drivers/cmp-module/49023000.cmp0/status
cmp-module 49023000.cmp0: LED ON
# cat /sys/bus/platform/drivers/cmp-module/49023000.cmp0/status
cmp-module 49023000.cmp0: +Ve > -Ve
1
```

- LED Off

```
# echo 0 > /sys/bus/platform/drivers/cmp-module/49023000.cmp0/status
cmp-module 49023000.cmp0: LED OFF
# cat /sys/bus/platform/drivers/cmp-module/49023000.cmp0/status
cmp-module 49023000.cmp0: -Ve > +Ve
0
```

b. CMP0_IN1:

- Connect a female-to-female jumper wire from P0_6(J8.P8) to P12_0(J11.P2)
- Change input to positive and re-start

```
# echo 1 > /sys/bus/platform/drivers/cmp-module/49023000.cmp0/pos_input
cmp-module 49023000.cmp0: Positive input set to 1
# echo 1 > /sys/bus/platform/drivers/cmp-module/49023000.cmp0/start
cmp-module 49023000.cmp0: Comparator started 2
# echo 1 > /sys/bus/platform/drivers/cmp-module/49023000.cmp0/status
cmp-module 49023000.cmp0: LED ON
# cat /sys/bus/platform/drivers/cmp-module/49023000.cmp0/status
cmp-module 49023000.cmp0: +Ve > -Ve
1
```

- LED Off

```
# echo 0 > /sys/bus/platform/drivers/cmp-module/49023000.cmp0/status
cmp-module 49023000.cmp0: LED OFF
# cat /sys/bus/platform/drivers/cmp-module/49023000.cmp0/status
cmp-module 49023000.cmp0: -Ve > +Ve
0
```

2. CMP1 Verification

a. CMP1_IN0

- Connect a female-to-female jumper wire from P0_1(J8.P4) to P12_0(J11.P2)
- Run

```
# echo 1 > /sys/bus/platform/drivers/cmp-module/49024000.cmp1/status
cmp-module 49024000.cmp1: LED ON
# cat /sys/bus/platform/drivers/cmp-module/49024000.cmp1/status
cmp-module 49024000.cmp1: +Ve > -Ve
1
```

- LED Off

```
# echo 0 > /sys/bus/platform/drivers/cmp-module/49024000.cmp1/status
cmp-module 49024000.cmp1: LED OFF
# cat /sys/bus/platform/drivers/cmp-module/49024000.cmp1/status
cmp-module 49024000.cmp1: -Ve > +Ve
0
```

b. CMP1_IN1

- Connect a female-to-female jumper wire from P0_7(J8.P10) to P12_0(J11.P2)
- Change input to positive and re-start

```
# echo 1 > /sys/bus/platform/drivers/cmp-module/49024000.cmp1/pos_input
cmp-module 49024000.cmp1: Positive input set to 1
# echo 1 > /sys/bus/platform/drivers/cmp-module/49024000.cmp1/start
cmp-module 49024000.cmp1: Comparator started 2
# echo 1 > /sys/bus/platform/drivers/cmp-module/49024000.cmp1/status
cmp-module 49024000.cmp1: LED ON
# cat /sys/bus/platform/drivers/cmp-module/49024000.cmp1/status
cmp-module 49024000.cmp1: +Ve > -Ve
1
```

- LED Off

```
# echo 0 > /sys/bus/platform/drivers/cmp-module/49024000.cmp1/status
cmp-module 49024000.cmp1: LED OFF
# cat /sys/bus/platform/drivers/cmp-module/49024000.cmp1/status
cmp-module 49024000.cmp1: -Ve > +Ve
0
```

c. CMP1_IN2

- Connect a female-to-female jumper wire from P1_5(J8.P19) to P12_0(J11.P2)
- Change input to positive and re-start

```
# echo 2 > /sys/bus/platform/drivers/cmp-module/49024000.cmp1/pos_input
cmp-module 49024000.cmp1: Positive input set to 2
# echo 1 > /sys/bus/platform/drivers/cmp-module/49024000.cmp1/start
cmp-module 49024000.cmp1: Comparator started 2
# echo 1 > /sys/bus/platform/drivers/cmp-module/49024000.cmp1/status
cmp-module 49024000.cmp1: LED ON
# cat /sys/bus/platform/drivers/cmp-module/49024000.cmp1/status
cmp-module 49024000.cmp1: +Ve > -Ve
1
```

- LED Off

```
# echo 0 > /sys/bus/platform/drivers/cmp-module/49024000.cmp1/status
cmp-module 49024000.cmp1: LED OFF
# cat /sys/bus/platform/drivers/cmp-module/49024000.cmp1/status
cmp-module 49024000.cmp1: -Ve > +Ve
0
```

d. CMP1_IN3

- Connect a female-to-female jumper wire from P0_5(J8.P9) to P12_0(J11.P2)
- Change input to positive and re-start

```
# echo 3 > /sys/bus/platform/drivers/cmp-module/49024000.cmp1/pos_input
```

```
cmp-module 49024000.cmp1: Positive input set to 3
# echo 1 > /sys/bus/platform/drivers/cmp-module/49024000.cmp1/start
cmp-module 49024000.cmp1: Comparator started 2
# echo 1 > /sys/bus/platform/drivers/cmp-module/49024000.cmp1/status
cmp-module 49024000.cmp1: LED ON
# cat /sys/bus/platform/drivers/cmp-module/49024000.cmp1/status
cmp-module 49024000.cmp1: +Ve > -Ve
1
```

- LED Off

```
# echo 0 > /sys/bus/platform/drivers/cmp-module/49024000.cmp1/status
cmp-module 49024000.cmp1: LED OFF
# cat /sys/bus/platform/drivers/cmp-module/49024000.cmp1/status
cmp-module 49024000.cmp1: -Ve > +Ve
0
```

3. CMP2 Verification

a. CMP2_IN0

- Connect a female-to-female jumper wire from P0_2(J8.P3) to P12_0(J11.P2)

```
# echo 1 > /sys/bus/platform/drivers/cmp-module/49025000.cmp2/status
cmp-module 49025000.cmp2: LED ON
# cat /sys/bus/platform/drivers/cmp-module/49025000.cmp2/status
cmp-module 49025000.cmp2: +Ve > -Ve
1
```

- LED Off

```
# echo 0 > /sys/bus/platform/drivers/cmp-module/49025000.cmp2/status
cmp-module 49025000.cmp2: LED OFF
# cat /sys/bus/platform/drivers/cmp-module/49025000.cmp2/status
cmp-module 49025000.cmp2: -Ve > +Ve
0
```

b. CMP2_IN1:

- Connect a female-to-female jumper wire from P1_2(J8.P13) to P12_0(J11.P2)
- Change input to positive and re-start

```
# echo 3 > /sys/bus/platform/drivers/cmp-module/49025000.cmp2/pos_input
```

```
cmp-module 49025000.cmp2: Positive input set to 3
# echo 1 > /sys/bus/platform/drivers/cmp-module/49025000.cmp2/start
cmp-module 49025000.cmp2: Comparator started 2
# echo 1 > /sys/bus/platform/drivers/cmp-module/49025000.cmp2/status
cmp-module 49025000.cmp2: LED ON
# cat /sys/bus/platform/drivers/cmp-module/49025000.cmp2/status
cmp-module 49025000.cmp2: +Ve > -Ve
1
```

- LED Off

```
# echo 0 > /sys/bus/platform/drivers/cmp-module/49025000.cmp2/status
cmp-module 49025000.cmp2: LED OFF
# cat /sys/bus/platform/drivers/cmp-module/49025000.cmp2/status
cmp-module 49025000.cmp2: -Ve > +Ve
0
```

4. CMP3 Verification

a. CMP3_IN0

- Connect a female-to-female jumper wire from P0_3(J8.P5) to P12_0(J11.P2)

```
# echo 1 > /sys/bus/platform/drivers/cmp-module/49026000.cmp3/status
cmp-module 49026000.cmp3: LED ON
# cat /sys/bus/platform/drivers/cmp-module/49026000.cmp3/status
cmp-module 49026000.cmp3: +Ve > -Ve
1
```

- LED Off

```
# echo 0 > /sys/bus/platform/drivers/cmp-module/49026000.cmp3/status
cmp-module 49026000.cmp3: LED OFF
# cat /sys/bus/platform/drivers/cmp-module/49026000.cmp3/status
cmp-module 49026000.cmp3: -Ve > +Ve
0
```

b. CMP3_IN1

- Connect a female-to-female jumper wire from P1_7(J8.P20) to P12_0(J11.P2)

- change the input to positive and re-start

```
# echo 2 > /sys/bus/platform/drivers/cmp-module/49026000.cmp3/pos_input
cmp-module 49026000.cmp3: Positive input set to 2
# echo 1 > /sys/bus/platform/drivers/cmp-module/49026000.cmp3/start
cmp-module 49026000.cmp3: Comparator started 2
# echo 1 > /sys/bus/platform/drivers/cmp-module/49026000.cmp3/status
cmp-module 49026000.cmp3: LED ON
# cat /sys/bus/platform/drivers/cmp-module/49026000.cmp3/status
cmp-module 49026000.cmp3: +Ve > -Ve
1
```

- LED Off

```
# echo 0 > /sys/bus/platform/drivers/cmp-module/49026000.cmp3/status
cmp-module 49026000.cmp3: LED OFF
# cat /sys/bus/platform/drivers/cmp-module/49026000.cmp3/status
cmp-module 49026000.cmp3: -Ve > +Ve
0
```

c. CMP3_IN2

- Connect a female-to-female jumper wire from P1_3(J8.P15) to P12_0(J11.P2)
- Change the input to positive and re-start

```
# echo 3 > /sys/bus/platform/drivers/cmp-module/49026000.cmp3/pos_input
cmp-module 49026000.cmp3: Positive input set to 3
# echo 1 > /sys/bus/platform/drivers/cmp-module/49026000.cmp3/start
cmp-module 49026000.cmp3: Comparator started 2
# echo 1 > /sys/bus/platform/drivers/cmp-module/49026000.cmp3/status
cmp-module 49026000.cmp3: LED ON
# cat /sys/bus/platform/drivers/cmp-module/49026000.cmp3/status
cmp-module 49026000.cmp3: +Ve > -Ve
1
```

- LED Off

```
# echo 0 > /sys/bus/platform/drivers/cmp-module/49026000.cmp3/status
cmp-module 49026000.cmp3: LED OFF
# cat /sys/bus/platform/drivers/cmp-module/49026000.cmp3/status
```

```
cmp-module 49026000.cmp3: -Ve > +Ve
0
```

8.1.17.5 Summary of the Test

5. All four CMP instances (CMP0 to CMP3) were validated by comparing positive and negative input voltages.
6. To verify interrupt counts on CPU0 for all CMP instances, run:

```
# cat /proc/interrupts  | grep cmp
1 GIC-0 188 Level     cmp-module
1 GIC-0 189 Level     cmp-module
1 GIC-0 190 Level     cmp-module
18 GIC-0 191 Level     cmp-module
```

This confirms that interrupts are generated as expected when the filtered output changes.

8.1.18 PDM

The Pulse Density Modulation (PDM) Audio Module processes 1-bit PDM streams from up to eight microphones, converting them into 16-bit Pulse Code Modulated (PCM) data. The module supports configurable clock frequencies and a FIFO buffer for efficient data management. This section details the module's functionality, configuration, and verification in a Yocto Linux environment using ALSA utilities. Each PDM module supports:

- Eight audio channels, each interfacing with one PDM microphone
- Conversion of 1-bit PDM input to 16-bit PCM output
- Nine clock frequency modes (512 kHz to 4.8 MHz)
- FIFO buffer storing up to eight PCM samples per channel
- Data access via Advanced Peripheral Bus (APB) registers
- Interrupt generation for data-ready events
- Power supply from internal 1.8-V LDO (LDO-5)

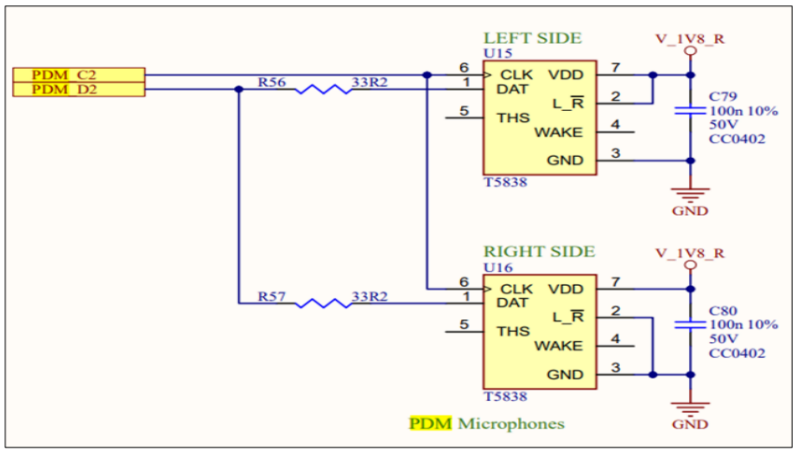


Figure 33 : PDM Audio Module Block Diagram

The module converts PDM streams to PCM data, storing 16-bit samples in per-channel FIFO buffers. The host accesses data via APB registers, with clock frequencies determining the sampling rate.

8.1.18.1 Kernel Configurations

To enable the PDM module, configure the following kernel settings:

```
CONFIG_SOUND=y
CONFIG_SND=y
CONFIG_SND_SUPPORT_OLD_API=y
CONFIG_SND_PROC_FS=y
CONFIG_SND_DRIVERS=y
CONFIG_SND_SOC=y
CONFIG_ALIF_PDM=y
CONFIG_SND_TIMER=y
CONFIG_SND_PCM=y
CONFIG_SND_JACK=y
CONFIG_SND_JACK_INPUT_DEV=y
CONFIG_SND_PCM_TIMER=y
```

```
CONFIG_XIP_PHYS_ADDR=0x80020000
CONFIG_MTD_PHYSMAP_START=0x80280000
CONFIG_MTD_PHYSMAP_LEN=0x400000
```

8.1.18.2 Yocto Configurations

1. Add the following to *`conf/local.conf`* in the Yocto build:

   Add the settings below only if all binaries are flashed in the MRAM and you plan to build the PDM driver alone. Adjust the size to fit cramfs and xipImage.

```
KERNEL_MTD_START_ADDR = "0x80310000"
XIP_KERNEL_LOAD_ADDR = "0x80020000"
KERNEL_MTD_LEN = "0x400000"
DISTRO_FEATURES_append += "apss-pdm"
# Default value: "1"
```

2. Generate images by running:

```
bitbake -c compile -f trusted-firmware-a ; bitbake trusted-firmware-a
bitbake -c compile -f linux-alif;bitbake linux-alif
bitbake alif-tiny-image
```

Output files *(`bl32.bin`, `alif-tiny-image-devkit-e8.cramfs-xip`, `xipImage`, `devkit-e8.dtb`)* are located in *`build/tmp/deploy/images/devkit-e8`*.

8.1.18.3 Device Tree Configuration

The PDM module's device tree entry is:

```
pdm0: pdm@4902d000 {
        compatible = "alif,alif-pcm";
        reg = <0x4902d000 0x1000>;
        clocks = <&psclks ENSEMBLE_SYST_PCLK>,
                 <&psclks ENSEMBLE_PDM_CLK>;
        clock-names = "pclk", "pdm_clk";
        interrupt-parent = <&gic>;
        interrupts = <0 134 IRQ_TYPE_LEVEL_HIGH>;
        status = "disabled";
};

pcmcard: pcmcard@4902d800 {
        compatible = "alif,alif-pcm-card";
        clocks = <&psclks ENSEMBLE_SYST_PCLK>;
};
```

```
&pdm0 {
        pinctrl-names = "default";
        pinctrl-0 = <&pinctrl_pdm0>;
        status = PDM_STATUS;
};
```

```
pinctrl_pdm0: pdm0grp {
        pinmux = <
                PIN_P6_7__PDM_C2_A      0x0
                PIN_P5_4__PDM_D2_B      0x1
                PIN_P0_5__PDM_C0_A      0x0
                PIN_P0_4__PDM_D0_A      0x1
                PIN_P6_3__PDM_C1_C      0x0
                PIN_P6_2__PDM_D1_C      0x1
        >;
};
```

Figure 34: PDM Device Tree

Channel 6 and 7 Pinmux Configuration

Configure the pinmux settings as follows for PDM audio interfaces on Channels 6 and 7:

Channel 6: *PIN_P5_2__PDM_C3_A* (PDM clock output)

Channel 7: *PIN_P5_1__PDM_D3_A* (PDM data input)

Append the below in the pincontrol node of pdm, pinctrl_pdm0: pdm0grp

```
PIN_P5_2__PDM_C3_A      0x0
PIN_P5_1__PDM_D3_A      0x1
```

Channel 2 and 3 , PDM_C1_C and PDM_D1_C is used in the pincontrol by default,

Change it to PDM_C1_A and PDM_D1_A since C instance pins are not available on Eagle devkit.

```
PIN_P0_7__PDM_C1_A      0x0
PIN_P0_6__PDM_D1_A      0x1
```

Note:

These settings have not been tested due to a known pinmux conflict between the SD card and SPI interface. To avoid operational issues, please disable both the SD card and SPI0 before applying these settings.

8.1.18.4 Hardware Setup

The PDM module needs an Eagle Flat board with internal PDM microphones on channels 4 and 5. Channels 0–3 and 6–7 require external connections. A debugger (Ulink or JLink) is needed for flashing and debugging.

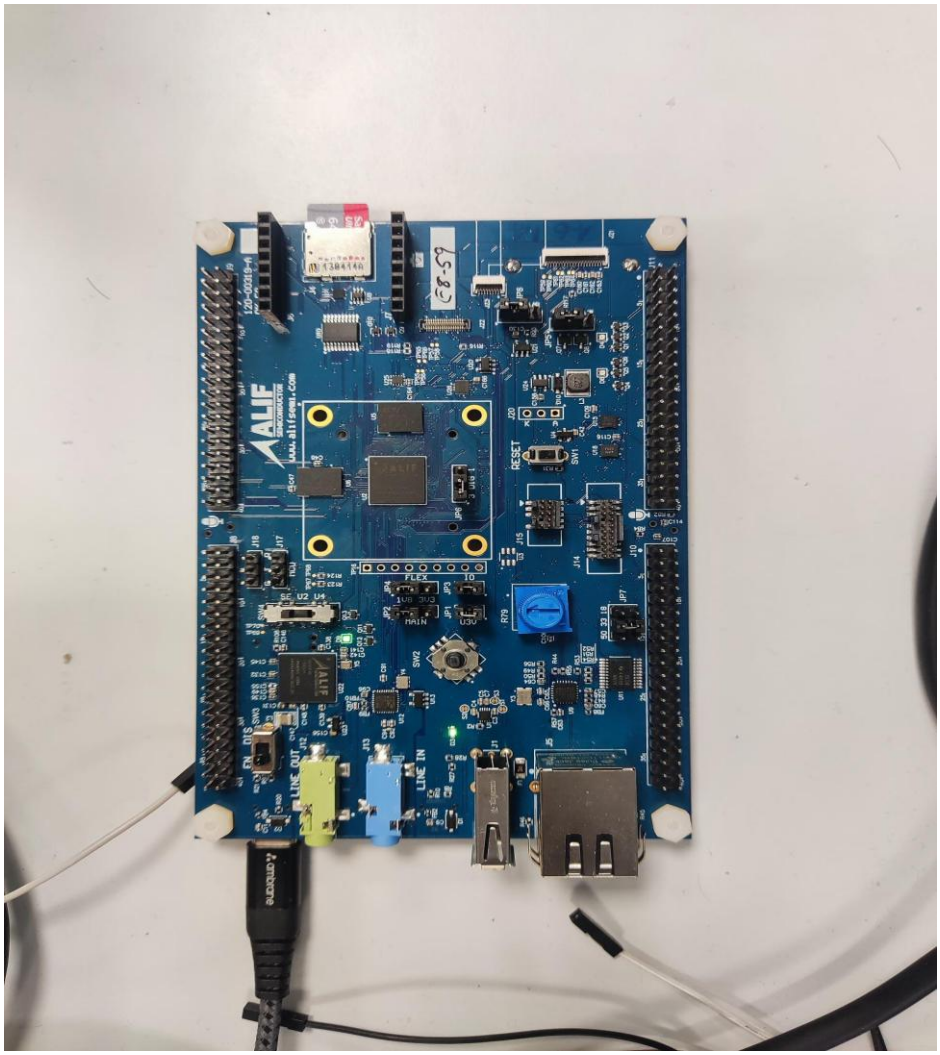


Figure 35 : PDM Hardware Setup (no external connections required as channels are internally connected to mic)

8.1.18.5 Channel Configurations

1. **Channels 4 and 5** (Internal Microphones):
   - **Data Line:** PDM_D2_B to P5_4
   - **Clock Line:** PDM_C2_A to P6_7

   No external connections required

2. **Channels 0 and 1:**
   - **Data Line:** Connect P5_4 (J9_7) to P0_4 (J8_7)
   - **Clock Line**: Connect P6_7 (J11_20) to P0_5 (J8_9)

3. Channels 2 and 3:
   - **Data Line:** Connect P5_4 (J9_7) to P0_6 (J8_8)
   - **Clock Line:** Connect P6_7 (J11_20) to P0_7 (J8_10)

4. **Channels 6 and 7:**
   - **Data Line:** Connect P5_4 (J9_7) to P5_1 (J10_15)
   - **Clock Line**: Connect P6_7 (J11_20) to P5_2 (J10_17)

Multi-Channel Setup

Testing channels 0 to 3 require two Eagle flat boards, each with two microphones. Connect the microphones for channels 2 and 3 to the second board as specified above.

8.1.18.6 Test Procedure

To configure and record audio on the devkit-e8 system, follow these steps:

1. Navigate to the */var/volatile/* directory and create a directory *sd*

```
cd /var/volatile/
root@devkit-e8:/var/volatile# mkdir sd
```

2. Mount the SD card *(/dev/mmcblk0p1)* to the sd directory

```
mount /dev/mmcblk0p1 sd
FAT-fs (mmcblk0p1): Volume was not properly unmounted. Some data may be
corrupt. Please run fsck.
```

Note:

If a warning appears about an improperly unmounted volume, consider running “fsck” to check for data corruption.

3. Navigate to the sysfs directory for audio configuration:

```
cd /sys/devices/platform/49020000.apb/4902d000.pdm/
```

4. Set the mode and sampling frequency to 16 kHz:

```
echo 2 > modefreq
```

5. Enable channels 4 and 5:

```
echo 48 > channelsel
```

Note:

The decimal value 48 (which is 0x30 in hexadecimal) enables channels 4 and 5 for this specific hardware configuration. To activate different channels, provide the corresponding decimal value, which the hardware interprets as a hexadecimal channel mask.

6. Move to the /var/volatile/sd directory to store the audio file:

```
cd /var/volatile/sd
```

7. Record audio using the `arecord` command with the following parameters:
   - Device: hw:0,0
   - Channels(c): 2: *change c to 1 for monochannel*
   - Sampling rate: 16 kHz
   - Format: Signed 16-bit Little Endian
   - Output: WAV file
   - Duration: 15 seconds

```
arecord -Dhw:0,0 -c2 -r16000 -fS16_LE -twav -d15
mode2_ch4_ch5_20_03_13_38.wav
```

Refer to the PDM mode table below for additional guidance on configuration settings.

PDM Mode	PDM Clock Frequency (PDM_Ci) kHz	Decimation Ratio	Baseband Sampling Rate (F_S) kHz[(1)]	Audio Signal Bandwidth (F_S/2) kHz	Application
9	4800	25	192	96	Ultrasound
8	4800	50	96	48	
7	3072	64	48	24	Full-bandwidth audio
6	2400	50	48	24	
5	1536	48	32	16	Wide-bandwidth audio
4	1024	64	16	8	High-quality voice
3	768	48	16	8	
2	512	32	16	8	
1	512	64	8	4	Standard voice
0	128	N/A	N/A	N/A	Microphone sleep

Figure 36 : PDM Mode table

8.1.18.7 Test Output

The following commands show how to mount an SD card, adjust audio settings, and record audio on the devkit-e8 system:

```
# Navigate to /var/volatile and create a directory for the SD card
root@devkit-e8:~# cd /var/volatile/
root@devkit-e8:/var/volatile# mkdir sd

# Mount the SD card
root@devkit-e8:/var/volatile# mount /dev/mmcblk0p1 sd
FAT-fs (mmcblk0p1): Volume was not properly unmounted. Some data may be
corrupt. Please run fsck.

# Display mounted filesystems
root@devkit-e8:/var/volatile# mount
mtd:physmap-flash.0 on / type cramfs (ro,relatime)
proc on /proc type proc (rw,relatime)
sysfs on /sys type sysfs (rw,relatime)
devtmpfs on /dev type devtmpfs
(rw,relatime,size=3000k,nr_inodes=750,mode=755)
devpts on /dev/pts type devpts (rw,relatime,mode=600,ptmxmode=000)
tmpfs on /run type tmpfs (rw,nosuid,nodev,mode=755)
```

```
tmpfs on /var/volatile type tmpfs (rw,relatime)
/dev/mmcblk0p1 on /var/volatile/sd type vfat
(rw,relatime,fmask=0022,dmask=0022,codepage=437,iocharset=iso8859-
1,shortname=mixed,utf8,errors=remount-ro)

# Navigate to the SD card directory
root@devkit-e8:/var/volatile# cd sd

# Configure audio settings
root@devkit-e8:/var/volatile/sd# cd
/sys/devices/platform/49020000.apb/4902d000.pdm/
root@devkit-e8:/sys/devices/platform/49020000.apb/4902d000.pdm# echo 2 >
modefreq
root@devkit-e8:/sys/devices/platform/49020000.apb/4902d000.pdm# echo 48 >
channelsel

# Return to the SD card directory
root@devkit-e8:/sys/devices/platform/49020000.apb/4902d000.pdm# cd
/var/volatile/sd

# Record audio with specified parameters
root@devkit-e8:/var/volatile/sd# arecord -Dhw:0,0 -c2 -r16000 -fS16_LE -twav
-d15 mode2_ch4_ch5_20_03_13_38.wav
alif pcm startup
selected channels : 0x30
Recording WAVE 'mode2_ch4_ch5_20_03_13_38.wav' : Signed 16 bit Little Endian,
Rate 16000 Hz, Stereo

# Record another audio file for 10 seconds
root@devkit-e8:/var/volatile/sd# arecord -Dhw:0,0 -c2 -r16000 -fS16_LE -twav
-d10 mode2_ch4_ch5_20_03_13_38_10.wav
alif pcm startup
selected channels : 0x30
Recording WAVE 'mode2_ch4_ch5_20_03_13_38_10.wav' : Signed 16 bit Little
Endian, Rate 16000 Hz, Stereo
```

```
Advanced 12nKB Sound Architecture Driver Initialized.
clocksource: Switched to clocksource arch_sys_counter
workingset: timestamp_bits=30 max_order=11 bucket_order=0
io scheduler mq-deadline registered
io scheduler kyber registered
Serial: 8250/16550 driver, 8 ports, IRQ sharing disabled
printk: legacy console [ttyS0] disabled
4901a000.serial: ttyS0 at MMIO 0x4901a000 (irq = 30, base_baud = 6250000) is a 16550A
printk: legacy console [ttyS0] enabled
physmap-flash physmap-flash.0: physmap platform flash device: [mem 0x80280000-0x8067ffff]
sdhci: Secure Digital Host Controller Interface driver
sdhci: Copyright(c) Pierre Ossman
sdhci-pltfm: SDHCI platform and OF driver helper
SMCCC: SOC_ID: ARCH_SOC_ID not implemented, skipping ....
clk: Disabling unused clocks
ALSA device list:
  #0: alifpcm
mmc0: SDHCI controller on 48102000.sdhci [48102000.sdhci] using ADMA
dw-apb-uart 4901a000.serial: forbid DMA for kernel console
cramfs: checking physical address 0x80280000 for linear cramfs image
cramfs: linear cramfs image on mtd:physmap-flash.0 appears to be 2420 KB in size
VFS: Mounted root (cramfs filesystem) readonly on device 31:0.
Freeing unused kernel image (initmem) memory: 28K
This architecture does not have kernel memory protection.
Run /sbin/init as init process
  with arguments:
    /sbin/init
```

```
login[52]: root login on 'ttyS0'
root@devkit-e7:~#
root@devkit-e7:~#
root@devkit-e7:~#
root@devkit-e7:~#
root@devkit-e7:~# cd /var/volatile/
root@devkit-e7:/var/volatile# mkdir sd
root@devkit-e7:/var/volatile# mount /dev/mmcblk0p1 sd
FAT-fs (mmcblk0p1): Volume was not properly unmounted. Some data may be corrupt. Please run fsck.
root@devkit-e7:/var/volatile# mount
mtd:physmap-flash.0 on / type cramfs (ro,relatime)
proc on /proc type proc (rw,relatime)
sysfs on /sys type sysfs (rw,relatime)
devtmpfs on /dev type devtmpfs (rw,relatime,size=3000k,nr_inodes=750,mode=755)
devpts on /dev/pts type devpts (rw,relatime,mode=600,ptmxmode=000)
tmpfs on /run type tmpfs (rw,nosuid,nodev,mode=755)
tmpfs on /var/volatile type tmpfs (rw,relatime)
/dev/mmcblk0p1 on /var/volatile/sd type vfat (rw,relatime,fmask=0022,dmask=0022,codepage=437,iocharset=iso8859-1,shortname=mixed,utf8,errors=remount-ro)
root@devkit-e7:/var/volatile# cd sd
root@devkit-e7:/var/volatile/sd# cd /sys/devices/platform/49020000.apb/4902d000.pdm/
root@devkit-e7:/sys/devices/platform/49020000.apb/4902d000.pdm# echo 2 > modefreq
root@devkit-e7:/sys/devices/platform/49020000.apb/4902d000.pdm# echo 48 > channelsel
root@devkit-e7:/sys/devices/platform/49020000.apb/4902d000.pdm# cd -
/var/volatile/sd
```

Figure 37 : Screen Capture of Test Output

8.1.19 ETHOS_U85 (NPU-HG)

Alif Semiconductor's Ensemble and Balletto families feature flexible compute architectures that combine Arm® Cortex®-A32 application cores, Cortex-M55 real-time cores with Armv8.1-M + Helium™ MVE, and Arm Ethos™ microNPUs for accelerated ML inference. All devices pair the Cortex-M55 with an Ethos-U55 microNPU, while the Ensemble E4/E8 variants additionally include a higher-performance Ethos-U85 microNPU for efficient transformer neural network (TNN) acceleration.

This application note provides practical guidance for building ML inference applications on Ensemble devices using the Linux OS and TensorFlow Lite framework. Introducing one sample application: ethosu_vww2_arm for running a VELA compiled Visual wake word-2 CNN model on Ensemble device with U85 NPU. The document serves as a streamlined guide to leveraging hardware-accelerated ML across Alif's Ensemble devices.

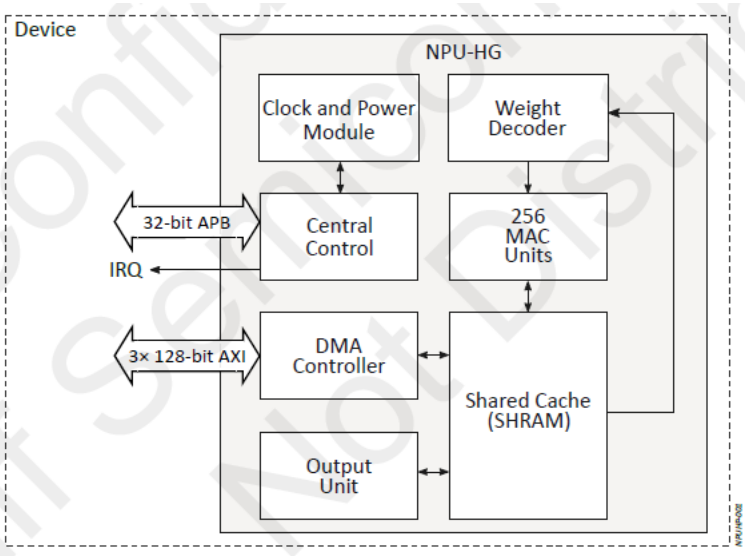


Figure 34: NPU-HG overview

8.1.19.1 Kernel Configurations

To enable the Ethos NPU module, configure the following kernel settings:

```
CONFIG_STAGING=y
CONFIG_ETHOSU_ARM=y
CONFIG_ETHOSU_DIRECT=y
CONFIG_DMA_SHARED_BUFFER=y
```

8.1.19.2 Yocto Configurations to build Ethos

Add the configurations below in the conf/local.conf file.

```
BASE_IMAGE4  = "1"
IMAGE_INSTALL:append = " tensorflow-lite"
DISTRO_FEATURES:append = " apss-ethosu"
```

8.1.19.3 Device Tree

```
reserved-memory {
        #address-cells = <1>;
        #size-cells = <1>;
        ranges;

        /* Reserved memory for Ethos-U85 NPU operations - WITHIN HyperRAM */
        ethosu_reserved: ethosu@a2000000 {
                compatible = "shared-dma-pool";
                reg = <0xa2000000 0x1000000>; /* 16MB within HyperRAM */
                no-map;
        };

        /* Ethos-U85 dedicated SRAM pool */
        ethosu_sram: ethosu_sram@a3000000 {
                compatible = "shared-dma-pool";
                reg = <0xa3000000 0x60000>; /* 384KB SRAM within HyperRAM */
                no-map;
        };
};


 /* Ethos-U85 NPU - ARM Driver Stack */
 ethosu_npu: ethosu@49042000 {
         #address-cells = <1>;
         #size-cells = <1>;
         compatible = "arm,ethosu-direct";
         reg = <0x49042000 0x2000>;
         interrupts = <GIC_SPI 355 IRQ_TYPE_LEVEL_HIGH>;
         interrupt-names = "irq";
         clocks = <&psclks ENSEMBLE_SYST_ACLK>, <&psclks ENSEMBLE_SYST_PCLK>;
         clock-names = "core", "apb";
         memory-region = <&ethosu_reserved>;
         sram = <&ethosu_sram>;
         dma-ranges = <0xa2000000 0xa2000000 0x1000000>,
                      <0xa3000000 0xa3000000 0x60000>;
         region-cfgs = <0 0 0 0 0 0 0 0>;
         cs-region = <1>;
         status = "disabled";
         ethosu_mem_config {
                 compatible = "arm,ethosu-mem-config";
                 /* <beats outstanding_read outstanding_write> */
                 sram = <0 64 32>;
                 ext  = <1 64 32>;
                 /* <mem_domain mem_type axi_port> */
                 configs = <0 0 1>,
                           <0 0 1>,
                           <0 0 1>;
         };
 };
```

Note: The reserved memories have been allocated in the HyperRAM region, even though the memory node indicates SRAM. Ethos has been tested solely with HyperRAM, not utilizing the actual SRAM available on the Alif boards.

8.1.19.4 Vela Compiler Install and Compilation Vela Compiler Installation Guide

This guide provides instructions for installing the **Ethos-U Vela 4.4.0** compiler in a Python virtual environment.

System Requirements

- **Operating System**: Ubuntu 20.04 or later (Linux x86_64)

- **Python**: Python 3.8 or later (3.12 recommended)

- **Disk Space**: ~500 MB for virtual environment

- **Internet Connection**: Required for downloading packages

Update package list

```
sudo apt update
```

Install Python and virtual environment tools

```
sudo apt install -y python3 python3-pip python3-venv
```

Install development tools (required for some Python packages)

```
sudo apt install -y build-essential python3-dev
```

Verify Python version (should be 3.8+)

```
python3 --version
```

Installation Steps

Step 1: Create Installation Directory

Create directory for Vela environment

```
mkdir -p ~/vela_compiler
cd ~/vela_compiler
```

Step 2: Create Python Virtual Environment

Create virtual environment named 'vela_env'

```
python3 -m venv vela_env
```

Activate the virtual environment

```
source vela_env/bin/activate
```

Verify activation (you should see (vela_env) in prompt)

Your prompt should now show: (vela_env) user@host:~$

Step 3: Upgrade pip

Upgrade pip to latest version

```
pip install --upgrade pip
```

Verify pip version

```
pip --version
```

Step 4: Install Vela Compiler

Install Vela 4.4.0

```
pip install ethos-u-vela==4.4.0
```

This will automatically install required dependencies:

- flatbuffers

- lxml

- numpy

Step 5: Install Additional Dependencies

Install packages used by supporting scripts

pip install numpy matplotlib pillow

These are used for:

- numpy: Array processing, image handling

- matplotlib: Image visualization

- pillow: Image format conversion

Step 6: Verify Installation

Check Vela version

```
vela --version
```

Expected output: 4.4.0

Check installed packages

```
pip list
```

Expected packages:

ethos-u-vela 4.4.0

flatbuffers 24.3.25

lxml 6.0.0

matplotlib 3.10.7

numpy 2.3.3

pillow 12.0.0

Step 7: Test Vela Compiler

Display Vela help

```
vela --help
```

Check available configuration files

```
vela --list-config-files
```

Expected output:

Available config files:

Arm/vela.ini

Check available accelerator configurations

```
vela --help | grep "accelerator-config"
```

Available accelerators include:

- ethos-u55-32, ethos-u55-64, ethos-u55-128, ethos-u55-256

- ethos-u65-256, ethos-u65-512

- ethos-u85-128, ethos-u85-256, ethos-u85-512, ethos-u85-1024, ethos-u85-2048

Complete Package List

Here is the specific package configuration utilized in this project:

```
Package         Version
--------------- -----------
contourpy       1.3.3
cycler          0.12.1
ethos-u-vela    4.4.0         ← Main Vela compiler
flatbuffers     24.3.25       ← TFLite format support
fonttools       4.60.1
kiwisolver      1.4.9
lxml            6.0.0         ← XML parsing (Vela dependency)
matplotlib      3.10.7        ← Visualization
numpy           2.3.3         ← Array processing
packaging       25.0
pillow          12.0.0        ← Image processing
pip             24.3.1
pyparsing       3.2.5
python-dateutil 2.9.0.post0
six             1.17.0
```

Vela Compilation

To compile our model using the VELA compiler, please make sure to activate the VELA environment first.

```
source vela_env/bin/activate
```

Your prompt should change to:

```
(vela_env) user@host:~$
```

Please execute the VELA compilation command as outlined below.

```
vela \
    --accelerator-config "ethos-u85-256" \
    --optimise "Performance" \
    --config "$PROJECT_DIR/scripts/alif_ensemble_e8.ini" \
    --system-config "Alif_Ensemble_E8_HyperRAM" \
    --memory-mode "Alif_Dynamic_HyperRAM" \
```

```
    --tensor-allocator "HillClimb" \
    --separate-io-regions \
    --cop-format "COP2" \
    --arena-cache-size 393216 \
    --cpu-tensor-alignment 16 \
    --hillclimb-max-iterations 99 \
    --verbose-config \
    --verbose-allocation \
    --output-dir "$OUTPUT_DIR" \
    "$INPUT_MODEL"
```

Set the $PROJECT_DIR, $OUTPUT_DIR, and $INPUT_MODEL paths properly.

The alif_ensemble_e8.ini file is our custom configuration file. The content of the file is attached below.

Please keep this configuration file in a directory and provide the correct path to the --config flag.

Alif_ensemble_e8.ini

```
; SPDX-FileCopyrightText: Copyright 2024 Alif Semiconductor
; SPDX-License-Identifier: Apache-2.0
;
; Custom Vela configuration for Alif Ensemble E8 SoC
; Matches the device tree memory layout exactly

; ------------------------------------------------------------------------
--
; System Configuration for Alif Ensemble E8

[System_Config.Alif_Ensemble_E8_HyperRAM]
core_clock=4e8
axi0_port=Sram
axi1_port=Dram
; SRAM characteristics (384KB at 0xa3000000)
Sram_clock_scale=1.0
Sram_burst_length=32
Sram_read_latency=32
```

```
Sram_write_latency=32
Sram_max_reads=8
Sram_max_writes=8
; HyperRAM characteristics (16MB at 0xa2000000) - using Dram port
Dram_clock_scale=0.25
Dram_burst_length=32
Dram_read_latency=500
Dram_write_latency=250
Dram_max_reads=16
Dram_max_writes=8

; ------------------------------------------------------------------------
--
; Memory Mode for Alif Device Tree Layout

[Memory_Mode.Alif_Dynamic_HyperRAM]
; Memory mode optimized for dynamic allocation
; Constants in HyperRAM (Axi1), arena/intermediates in SRAM (Axi0)
const_mem_area=Axi1
arena_mem_area=Axi0
cache_mem_area=Axi0
; Match device tree SRAM size exactly
arena_cache_size=393216

[Memory_Mode.Alif_HyperRAM_Only]
; Alternative: Everything in HyperRAM for testing
const_mem_area=Axi1
arena_mem_area=Axi1
cache_mem_area=Axi1
arena_cache_size=393216
```

8.1.19.5 Test Setup

Required binaries

1. Libtensorflowlite.so

This is a TensorFlow Lite library that has been cross-compiled and can be built using Yocto.

TensorFlow version 2.16.1 is utilized in Yocto and must be stored in the filesystem under /usr/lib.
This will be automatically included in the file system when you build with the specified Yocto configurations.

2. Libethosu_delegate.so

This can be built with the ethos-u-linux-driver-stack available at artificial-intelligence / ethos-u / Ethos-U Linux Driver Stack · GitLab
Commit Id used : 3fe05bfdac4b1841d77c5d038dadaf59326f025c

Keep this library in the filesystem under /usr/lib
Refer the below note.

3. Application binary

A user application to run an inference with a vela compiled tensorflowlite model.

Keep it in the file system under /usr/bin/ or keep it in SDcard

User application should use the same version of tensorflowlite header files which used to create the libtensorflowlite.so and libethosu_delegate.so.

Git repository of Tensorflowlite: https://github.com/tensorflow/tensorflow.git

Tensorflow version used : 2.16.1

Tensorflowlite headers for building a custom user application can be taken by cloning it from above git repositories as shown below.

```
git clone https://github.com/tensorflow/tensorflow.git
cd tensorflow/
git fetch --tags
git tag -l
git checkout v2.16.1
```

Use the relevant headers from this repo along with the libtensorflowlite.so library for building the user application.

An already built sample application is attached along with this release please refer the below note.

4. A VELA compiled. tflite model

   Download a INT8 fully quantized model (eg: https://github.com/Arm-Examples/ML-zoo/tree/master/models/visual_wake_words/micronet_vww2/tflite_int8)

   Compile it with the VELA and store it in the SD Card.

The application loads the model, attaches the delegate, and sets up the input and output tensors and buffers. After the setup is complete, it runs the inference using the TensorFlow Lite runtime library together with the Arm Ethos-U delegate library, the Arm Ethos-U driver library (both of which are built into the delegate), and the Ethos-U kernel driver.

Note: The shared library **libtensorflowlite.so** is automatically included in the file system when you build with the specified Yocto configurations. However, the **libethosu_delegate.so** library must be added manually. Because cross-compiling the Ethos-U delegate requires additional external steps, the cross-compiled binary is already provided for you in the **Final_ETHOS_Files** folder included with this release.

This folder also contains a sample application, the application build scripts, the Vela compiler, and tools for generating input images for the VWW2 TensorFlow Lite model.

Running the Application

The Application included in the ***Final_ETHOS_Files/Bin/*** named as ***ethosu_vww2_arm***

This application has two arguments, .tflite model and a raw bin file of a 50x50 size grayscale image.

Converting a .jpg image into a binary file of 50x50 grayscale image can be done by using the script ***convert_images_to_vww_format.py*** included in ***Final_ETHOS_Files/Scripts/InputImage*** folder.
For converting an image rename the image into person1.jpg and keep it in the same folder as the script and run the script with python. The converted image will be generated in a folder named ***vww_test_images.***

```
python3 convert_images_to_vww_format.py
```

Users can keep the input image and model in SDcard and run the ethosu_vww2_arm application with them.

Recommended: Use ***libethosu_delegate.so, ethosu_vww2_arm*** and ***vww2_50_50_INT8_vela.tflite*** files from the attached ***Final_ETHOS_files*** folder for initial ETHOSU NPU driver verification. Later users can create custom user applications keeping the libraries as such.

8.1.19.6 Test output

Attaching the console output from running a sample application, ethosu_vww2_arm_9, with the Vela-compiled model and a raw grayscale image of size 50x50, as specified for the Visual Wake Word-2 Int8 model. The output tensor is an array containing two quantized confidence values: one for "person not detected" and the other for "person detected." The output indicates a higher confidence value for "person detected." Attaching the actual image and the converted image used as input below.

Person3.jpg

Person3.jpg image is converted to a grayscale image of size 50x50

person3_50x50_int8.bin

```
root@devkit-e8:~# ethosu_vww2_arm_9 /mnt/vww2_50_50_INT8_vela_COP2_withseperate-io-region.tflite /mnt/vww_test_images/person2_50x50_int8.bin
VWW2 with Ethos-U NPU Acceleration
========================================
Setting up signal handler for debugging...
Signal handlers installed
Loading TensorFlow Lite model with Ethos-U acceleration...
Loading Ethos-U delegate...
Checking for Ethos-U device...
Found /dev/ethosu0 device
Checking Ethos-U device status...
/dev/ethosu0 device file exists
Device permissions OK (read/write access granted)
Testing device open/close...
Device opens/closes successfully
Trying: libethosu_delegate_fixed.so
  Loading library: libethosu_delegate_fixed.so
  Checking library file access...
  Library file not readable: No such file or directory
Trying: libethosu_delegate.so
  Loading library: libethosu_delegate.so
  Checking library file access...
  Library file not readable: No such file or directory
Trying: ./libethosu_delegate.so
  Loading library: ./libethosu_delegate.so
  Checking library file access...
  Library file not readable: No such file or directory
Trying: /usr/lib/libethosu_delegate.so
  Loading library: /usr/lib/libethosu_delegate.so
  Checking library file access...
  Library file is readable
  Opening library with dlopen...
  Library loaded successfully, handle = 0xb6f8c830
  Looking for tflite_plugin_create_delegate symbol...
  Found tflite_plugin_create_delegate at address: 0xb696bfb8
  Looking for tflite_plugin_destroy_delegate symbol...
  Found tflite_plugin_destroy_delegate at address: 0xb696b218
  Function symbols found
  Creating delegate instance...
  Debug: Function pointer address = 0xb696bfb8
  Debug: Preparing options array...
  Debug: Options prepared:
    - Key[0]: device_name
    - Value[0]: /dev/ethosu0
    - Num options: 1
  Debug: About to call create_delegate_func_...
  Debug: Parameters: keys=0xbef4cb34, values=0xbef4cb38, num=1, error_callback=nullptr
  Test 1: Trying with NULL options first...
INFO: EthosuOp: Version 1.0
INFO: EthosuOp: Device name = /dev/ethosu0
INFO: EthosuOp: Timeout = 60000000000
INFO: EthosuOp: Enable cycle counter = false
  NULL options worked! Destroying test delegate...
  Test delegate destroyed
  Test 2: Trying with device_name option...
INFO: EthosuOp: Version 1.0
INFO: EthosuOp: Device name = /dev/ethosu0
INFO: EthosuOp: Timeout = 60000000000
INFO: EthosuOp: Enable cycle counter = false
  Debug: create_delegate_func_ returned successfully
  Debug: Returned delegate pointer = 0xb6ef2e30
  Delegate instance created successfully
Ethos-U delegate loaded from: /usr/lib/libethosu_delegate.so
Model loaded successfully
Interpreter created successfully
Total tensors in graph: 5
Execution plan size: 1
Tensor details:
  Tensor[0]: ethos_u_command_stream, type=3, bytes=12964
  Tensor[1]: read_only, type=3, bytes=153200
  Tensor[2]: scratch, type=3, bytes=35696
  Tensor[3]: input, type=9, bytes=2500
  Tensor[4]: Identity, type=9, bytes=2
Adding Ethos-U delegate to interpreter...
INFO: Ethos-U Operator Delegate delegate: 1 nodes delegated out of 1 nodes with 1 partitions.

Ethos-U delegate added successfully!
Model will run on Ethos-U NPU hardware
Allocating tensors...
ERROR: EthosuOp: Warning - tensor count less than expected minimum
Tensors allocated successfully

Model Information:
  Input tensors: 1
  Output tensors: 1
  Execution plan nodes: 1
  Hardware acceleration: Ethos-U NPU
  Input tensor shape: [1, 50, 50, 1]
  Input tensor type: 9

  Loading image: /mnt/vww_test_images/person2_50x50_int8.bin
Image file size: 2500 bytes
Input tensor size: 2500 elements
Raw file size matches int8 tensor size exactly!
Loaded raw int8 image data directly

Running inference...
Executing on Ethos-U NPU hardware!
Debug: First 10 input values: -71 -83  ethosu0: Inference create. No IFMs given. Continuing anyway
-74 -77 -71 -70 -27 11 -25 -44
INFO: EthosuOp: Node has 5 inputs, 1 outputs
INFO: EthosuOp: Input indices:
INFO:   [0] = tensor 0 (ethos_u_command_stream, 12964 bytes)
INFO:   [1] = tensor 1 (read_only, 153200 bytes)
INFO:   [2] = tensor 2 (scratch, 35696 bytes)
INFO:   [3] = tensor 2 (scratch, 35696 bytes)
INFO:   [4] = tensor 3 (input, 2500 bytes)
INFO: EthosuOp: Extracted 1 IFMs starting from offset 4
INFO: EthosuOp: Read back 1 output tensors
INFO: EthosuOp: Output[0]: tensor 4, 2 bytes
INFO: EthosuOp: Output data: 22 29
Inference completed successfully
  Inference time: 362 ms
Debug: Output tensor address: 0xb6921b00
Debug: Output tensor bytes: 2
Debug: Output tensor allocation_type: 2

Results:
Output tensor shape: [1, 2]
Debug: Raw output bytes (first 32): 22 29
Output probabilities (int8 quantized):
  Class 0: 34
  Class 1: 41
Person raw score: 41
No-person raw score: 34
 Person detected!

 VWW2 inference with Ethos-U completed successfully!
```

8.2 Linux Kernel Features

8.2.1 This overview details the default Linux kernel features found in the DevKit_e8_defconfig configuration file for the Alif DevKit, along with the features enabled via DISTRO_FEATURES in the Yocto build system.

8.2.2 Kernel Features and Configurations

The table below outlines the kernel features, their default status in DevKit_e8_defconfig, and their dependency on DISTRO_FEATURES for enabling them during the build process.

Kernel Feature	Enabled in Default Config	Enabled with DISTRO_FEATURES	DISTRO_FEATURES Value
Core Features			
Arm System Type (ARMv7-A)	Yes	Yes	-
Arm Uniprocessor	Yes	Yes	-
Arm SMP	No	No	-
Tick Based CPU Accounting	Yes	Yes	-
PREEMPT_NONE	Yes	Yes	-
SLUB	Yes	Yes	-
SLAB_CACHE_MERGE	Yes	Yes	-
VMSPLIT_3G	Yes	Yes	-
THUMB2 – KERNEL	Yes	Yes	-
VFP	Yes	Yes	-
NEON	Yes	Yes	-
KERNEL MODE NEON	Yes	Yes	-
File Systems			
CRAMFS	Yes	Yes	-
SYSFS	No	Yes	apss-spi, apss-i2c, apss-usb
INOTIFY & DNOTIFY	Yes	Yes	-
EXT3/EXT4	No	Yes	apss-sd, apss-sd-boot
Drivers			
DWC - UART2-B	Yes	Yes	-

Kernel Feature	Enabled in Default Config	Enabled with DISTRO_FEATURES	DISTRO_FEATURES Value
PINMUX	Yes	Yes	-
RTC	Yes	Yes	-
DWC SPI	No	Yes	apss-spi
ENV3-Click Temp Sensor (SPI)	No	Yes	apss-spi
WATCHDOG – ARM SBSA	Yes	Yes	-
DWC3 USB	No	Yes	apss-usb
USB GADGET – CDC-ACM	No	Yes	apss-usb
MTD	Yes	Yes	-
BLOCK Device	Yes	Yes	-
MHU	No	No	apss-mhu
HWSEM	No	Yes	apss-hwsem
SDHCI Boot	No	Yes	apss-sd-boot
SDHCI Mount	Yes	Yes	apss-sd-share
ETHERNET	No	Yes	apss-ethernet
I2C-I3C	No	Yes	apss-i2c-i3c
CDC-200	No	Yes	apss-cdc200/MIPI-DSI
Utimer	No	Yes	apss-utimer
I2C	No	Yes	apss-i2c
CRC	No	Yes	apss-crc
TOUCH-SCREEN	No	Yes	apss-touch
USB_HOST	No	Yes	apss-usb-host
I3C	No	Yes	apss-i3c
I2S	No	Yes	apss-i2s
MIPI-CSI	No	Yes	apss-mipicsi

9. Performance and Resource Usage

9.1 Linux Device Drivers RAM Usage

This section provides an overview of the memory utilization associated with the Linux kernel (xipImage) and SRAM when specific device drivers are enabled on the Application Processor Subsystem (APSS). The stitched SRAM possesses a total capacity of 8 MB.

APSS Drivers/Features Memory Usage			
Drivers	**XipImage Size (MB)**	**Dynamic RAM Used (MB)**	**Free (MB)**
Basic Drivers UART2, Pinmux, GPIO, RTC, WDT	2.3	2.3	4.4

APSS Drivers/Features with Stitched SRAM (8 MB)

The following table lists memory usage for drivers and features using STICHED SRAM (address 0x0200_0000), with a total capacity of 8 MB.

APSS Drivers/Features Memory Usage			
Drivers	**Image Size (MB)**	**Dynamic RAM (MB)**	**Free (MB)**
Basic Drivers + MHU + HWSEM + USB Pheri	2.4	**3.4**	**3.2**
Basic Drivers + MHU + HWSEM + SD Share	2.8	3.7	3.0
Basic Drivers + MHU + HWSEM + Ethernet	3.2	3.6	3.1
Basic Drivers + MHU + HWSEM + debug	2.3	2.4	4.3
Basic Drivers + MHU + HWSEM + Ethernet + SD Share	3.4	3.1	3.3

Note: "Basic Drivers" refers to UART2, Pinmux, GPIO, RTC, and WDT.

9.2 Linux CPU Performance and Memory Test

This overview assesses the CPU performance and memory features of the Linux kernel on the Alif DevKit's A32 core. CPU performance is evaluated using the Dhrystone benchmark, and memory tests are performed on STITCHED SRAM.

9.2.1 CPU Performance (A32 DMIPS)

The Dhrystone benchmark measures CPU performance in DMIPS (Dhrystone Millions of Instructions Per Second) on the A32 core at maximum frequency with caches enabled.

9.2.1.1 Linux Booting from MRAM (Max Frequency, No Compiler Optimization)

```
ensemble-pinctrl 1a603000.pinctrl: Ensemble pinctrl initialized
alif-adc-dac6 4902a000.adc-dac6: initialized successfully
clocksource: Switched to clocksource arch_sys_counter
NET: Registered PF_UNIX/PF_LOCAL protocol family
workingset: timestamp_bits=30 max_order=11 bucket_order=0
io scheduler mq-deadline registered
io scheduler kyber registered
CPU0: Dhrystones per Second: 1462738 (832 DMIPS)
```

Figure 96:Linux Booting from MRAM with Max Frequency without Compiler Optimization

9.2.1.2 Linux Boot Time (No Compiler Optimization)

```
[    0.085893] io scheduler mq-deadline registered
[    0.085902] io scheduler kyber registered
[    0.095343] Serial: 8250/16550 driver, 8 ports, IRQ sharing disabled
[    0.097011] printk: legacy console [ttyS0] disabled
[    0.097233] 4901a000.serial: ttyS0 at MMIO 0x4901a000 (irq = 54, base_baud = 6250000) is a 16550A
[    0.097279] printk: legacy console [ttyS0] enabled
[    0.553595] physmap-flash physmap-flash.0: physmap platform flash device: [mem 0x80380000-0x8057ffff]
[    0.565531] dw-apb-rtc 42000000.rtc: registered as rtc0
[    0.571189] SMCCC: SOC_ID: ARCH_SOC_ID not implemented, skipping ....
[    0.581938] Registering SWP/SWPB emulation handler
[    0.592135] clk: Disabling unused clocks
[    0.596361] dw-apb-uart 4901a000.serial: forbid DMA for kernel console
[    0.603035] cramfs: checking physical address 0x80380000 for linear cramfs image
[    0.610475] cramfs: linear cramfs image on mtd:physmap-flash.0 appears to be 1824 KB in size
[    0.618998] VFS: Mounted root (cramfs filesystem) readonly on device 31:0.
[    0.626057] Freeing unused kernel image (initmem) memory: 56K
[    0.631833] This architecture does not have kernel memory protection.
[    0.638303] Run /sbin/init as init process
[    0.642420]   with arguments:
[    0.645393]     /sbin/init
[    0.648120]   with environment:
[    0.651285]     HOME=/
[    0.653667]     TERM=linux
mount: mounting sysfs on /sys failed: No such device
*** Press any key within 5 seconds to login into a shell prompt ***

Alif Iota - Tiny Linux Distribution 2.0.0 devkit-e8 /dev/ttyS0

devkit-e8 login: root
login[70]: root login on 'ttyS0'
root@devkit-e8:~#
```

Figure 37: Linux Boot Time without Compiler Optimization

- Without compiler optimization, the Linux boot time is 0.6 seconds.

9.2.1.3 Linux DMIPS with Compiler Optimization

```
[    0.011069] NET: Registered protocol family 1
[    0.011270] workingset: timestamp_bits=30 max_order=11 bucket_order=0
[    0.011403] Block layer SCSI generic (bsg) driver version 0.4 loaded (major 250)
[    7.699364] CPU0: Dhrystones per Second: 1873157 (1066 DMIPS)
[    7.701986] Serial: 8250/16550 driver, 8 ports, IRQ sharing disabled
[    7.702604] printk: console [ttyS0] disabled
[    7.702644] 4901a000.uart2: ttyS0 at MMIO 0x4901a000 (irq = 126, base_baud = 6250000) is a 16550A
```

Figure 38: Linux DMIPIS with Compiler Optimization

16.2 Memory Testing

Memory tests assess the performance and reliability of Stitched SRAM, which is utilized by the Linux kernel and drivers on the DevKit.

16.2.1 Stitched SRAM Testing

```
Booting Linux on physical CPU 0x0
Linux version 6.12.6+ (alif@alif) (arm-poky-linux-gnueabi-gcc (GCC) 13.3.0, GNU ld (GNU Binutils)
2.42.0.20240723) #1 SMP PREEMPT Thu Aug 14 12:40:55
CPU: ARMv8-a aarch32 Processor [411fd010] revision 0 (ARMv8), cr=50c5383d
CPU: PIPT / VIPT nonaliasing data cache, VIPT aliasing instruction cache
OF: fdt: Machine model: devkit-e8
Memory policy: Data cache writealloc
early_memtest: # of tests: 17
0x02000000 - 0x02004000 pattern 4c494e5558726c7a
0x02051ba8 - 0x025ff000 pattern 4c494e5558726c7a
0x02600000 - 0x027d8000 pattern 4c494e5558726c7a
0x02000000 - 0x02004000 pattern eeeeeeeeeeeeeeee
0x02051ba8 - 0x025ff000 pattern eeeeeeeeeeeeeee
0x02600000 - 0x027d8000 pattern eeeeeeeeeeeeeee
0x02000000 - 0x02004000 pattern dddddddddddddddd
0x02051ba8 - 0x025ff000 pattern ddddddddddddddd
0x02600000 - 0x027d8000 pattern dddddddddddddddd
0x02000000 - 0x02004000 pattern bbbbbbbbbbbbbbbb
0x02051ba8 - 0x025ff000 pattern bbbbbbbbbbbbbbb
0x02600000 - 0x027d8000 pattern bbbbbbbbbbbbbbbb
0x02000000 - 0x02004000 pattern 7777777777777777
0x02051ba8 - 0x025ff000 pattern 7777777777777777
0x02600000 - 0x027d8000 pattern 7777777777777777
0x02000000 - 0x02004000 pattern cccccccccccccccc
0x02051ba8 - 0x025ff000 pattern ccccccccccccccc
0x02600000 - 0x027d8000 pattern cccccccccccccccc
0x02000000 - 0x02004000 pattern 9999999999999999
0x02051ba8 - 0x025ff000 pattern 999999999999999
0x02600000 - 0x027d8000 pattern 9999999999999999
0x02000000 - 0x02004000 pattern 6666666666666666
0x02051ba8 - 0x025ff000 pattern 666666666666666
0x02600000 - 0x027d8000 pattern 6666666666666666
0x02000000 - 0x02004000 pattern 3333333333333333
```

```
0x02051ba8 - 0x025ff000 pattern 3333333333333333
0x02600000 - 0x027d8000 pattern 3333333333333333
0x02000000 - 0x02004000 pattern 8888888888888888
0x02051ba8 - 0x025ff000 pattern 8888888888888888
0x02600000 - 0x027d8000 pattern 8888888888888888
0x02000000 - 0x02004000 pattern 4444444444444444
0x02051ba8 - 0x025ff000 pattern 4444444444444444
0x02600000 - 0x027d8000 pattern 4444444444444444
0x02000000 - 0x02004000 pattern 2222222222222222
0x02051ba8 - 0x025ff000 pattern 2222222222222222
0x02600000 - 0x027d8000 pattern 2222222222222222
0x02000000 - 0x02004000 pattern 1111111111111111
0x02051ba8 - 0x025ff000 pattern 1111111111111111
0x02600000 - 0x027d8000 pattern 1111111111111111
0x02000000 - 0x02004000 pattern aaaaaaaaaaaaaaaa
0x02051ba8 - 0x025ff000 pattern aaaaaaaaaaaaaaaa
0x02600000 - 0x027d8000 pattern aaaaaaaaaaaaaaaa
0x02000000 - 0x02004000 pattern 5555555555555555
0x02051ba8 - 0x025ff000 pattern 5555555555555555
0x02600000 - 0x027d8000 pattern 5555555555555555
0x02000000 - 0x02004000 pattern ffffffffffffffff
0x02051ba8 - 0x025ff000 pattern ffffffffffffffff
0x02600000 - 0x027d8000 pattern ffffffffffffffff
0x02000000 - 0x02004000 pattern 0000000000000000
0x02051ba8 - 0x025ff000 pattern 0000000000000000
0x02600000 - 0x027d8000 pattern 0000000000000000
```

Figure 39: Linux Memory Testing STITCHED SRAM (0x0200_0000).

10. Advanced Build Options

10.1 Building APSS Linux Images in Docker Container on Any Linux Host Machine

This overview details the process of building APSS Linux images using a Docker container on a Linux host machine, specifically utilizing Ubuntu 22.04 as the build environment.

10.1.1 Install Docker Package/Application in Linux Host Machine

Install the necessary packages (git, docker, and docker.io) on the Linux host. Refer to https://docs.docker.com/engine/install/ for distribution-specific instructions.

- Example (Ubuntu 22.04):

```
bash
sudo apt install git docker docker.io -y
```

10.1.2 Run Ubuntu 22.04 Distribution in Docker Container

Follow these steps to set up and use an Ubuntu 22.04 Docker container to build APSS Linux images.

1. **Clone the Repository:**

- Clone the AlifSemiDev repository and checkout the scarthgap_yocto_5.0 branch:

```
bash
git clone https://github.com/AlifSemi/alif_linux-apss-build-setup
```

- Provide GitHub credentials (use personal access tokens as password):

```
bash
cd alif_linux-apss-build-setup
git checkout scarthgap_yocto_5.0
```

***Note**: See Generating Personal Access Tokens for Cloning from GitHub for token creation.*

2. **Fetch Docker Image and Install Prerequisites:**

- Run the script to fetch the Ubuntu 22.04 Docker image and install prerequisites:

```
bash
./alif_linux-apss-build-setup/ubuntu-docker/build-ubuntu-image.sh
```

Note: This step may take some time to complete.

---

3. **Start the Docker Container:**

- Launch the Ubuntu 22.04 container:

```
bash
./alif_linux-apss-build-setup/ubuntu-docker/ubuntu-builder
```

4. Install Host Packages:

- Install required packages for building Linux images:

```
bash
sudo alif_linux-apss-build-setup/scripts/host_setup.sh
```

5. Fetch Yocto and BSP Layers:

- Navigate to the repository and fetch layers:

```
bash
cd alif_linux-apss-build-setup
git checkout scarthgap_yocto_5.0
sh scripts/fetch-layers.sh
```

Note: Run sh scripts/fetch-layers.sh update for the latest changes.

---

6. **Set Up Build Environment**:

- Configure the build environment:

```
bash
source scripts/setup.sh
```

7. Build Linux Images:

- Generate APSS images using BitBake:

```
bash
bitbake alif-tiny-image
```

8. Extract Images:

- Use the following files from tmp/deploy/images/devkit-e8 to boot the APSS:
  - xipImage
  - bl32.bin
  - devkit-e8.dtb
  - alif-tiny-image-devkit-e8.cramfs-xip

9. Exit Container:

- Press Ctrl+D or type logout to exit the Docker container.

10. Re-enter Container:

- Restart the container to continue work:

```
bash
./alif_linux-apss-build-setup/ubuntu-docker/ubuntu-builder
```

11. Navigate to Build Directory:

- Change to the previously created Yocto build directory:

```
bash
cd build
```

12. Rebuild Images (Optional):

- Generate APSS images again if needed:

```
bash
bitbake alif-tiny-image
```

13. **Shared Directory**:

- The alif_linux-apss-build-setup directory is shared between the host and container, allowing access to all files from the native Linux host.

10.2 Building APSS Linux Images in Docker Container on a Windows 11 Host Machine

The overview provides guidance on enabling essential Windows features, installing Docker Desktop, and building APSS Linux images using an Ubuntu 22.04 Docker container on a Windows 11 host.

10.2.1 Enable Hyper-V and Windows Subsystem for Linux Features

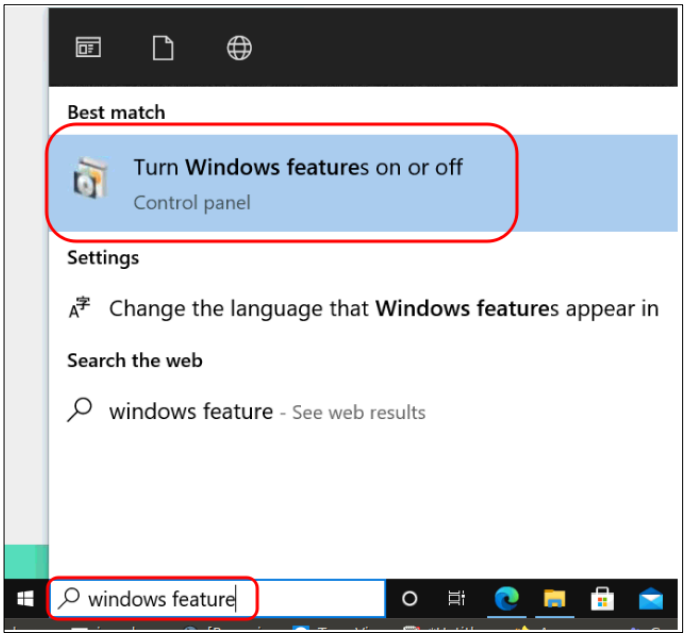
Steps

1. Enable Virtualization in BIOS:

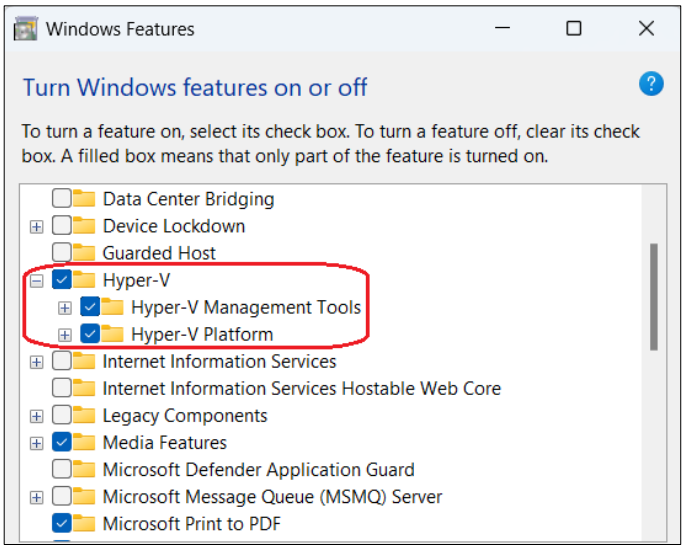
- Restart the computer and enter BIOS setup (use the appropriate key, e.g., F2, Del).
- Navigate to **Advanced BIOS Features** > **CPU Features** > **Virtualization Technology** > **Enabled**.
- Save and exit.

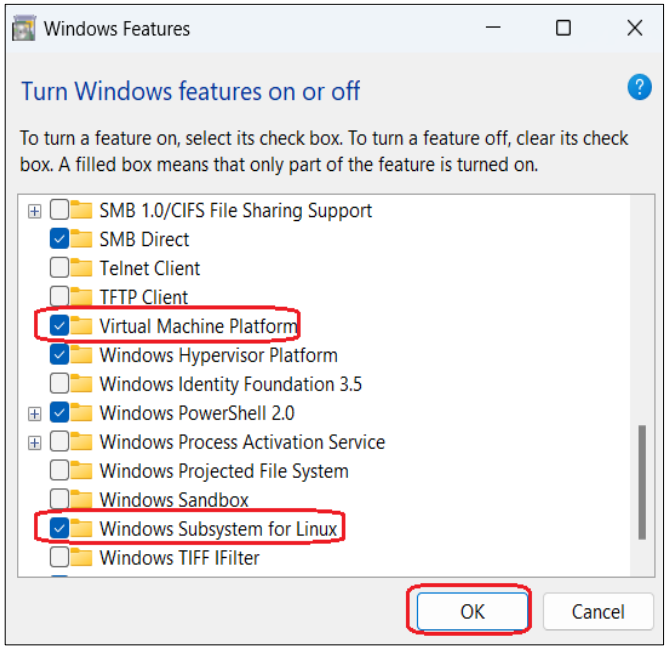
2. Enable Windows Features:

- Search for "Windows Features" in the Windows 11 search bar and select **Turn Windows features on or off**.



- Check **Hyper-V**, **Virtual Machine Platform**, and **Windows Subsystem for Linux**, then click **OK**.



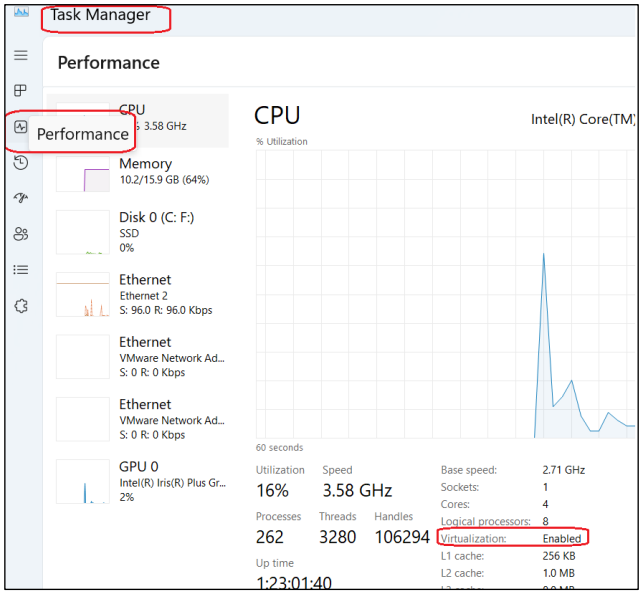


2. **Restart if Prompted:**

- Restart the PC if required.

3. **Verify Virtualization:**

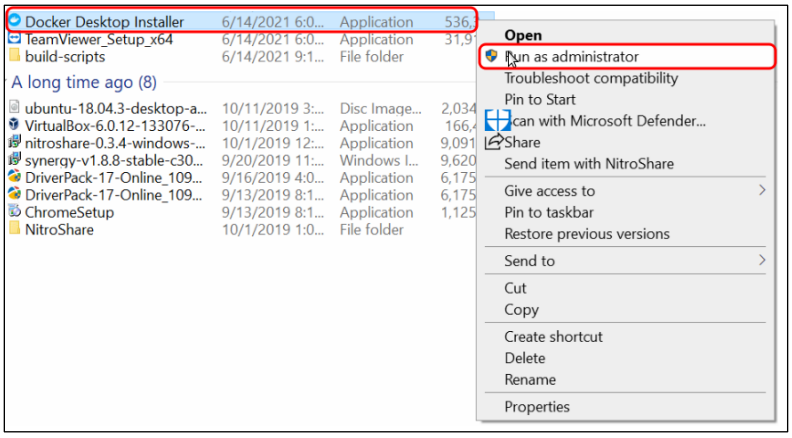
- Open Task Manager, go to the **Performance** tab, and confirm virtualization is enabled.

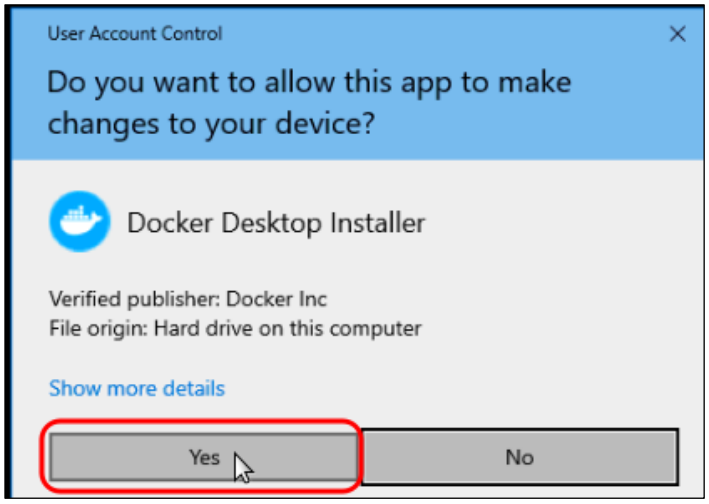


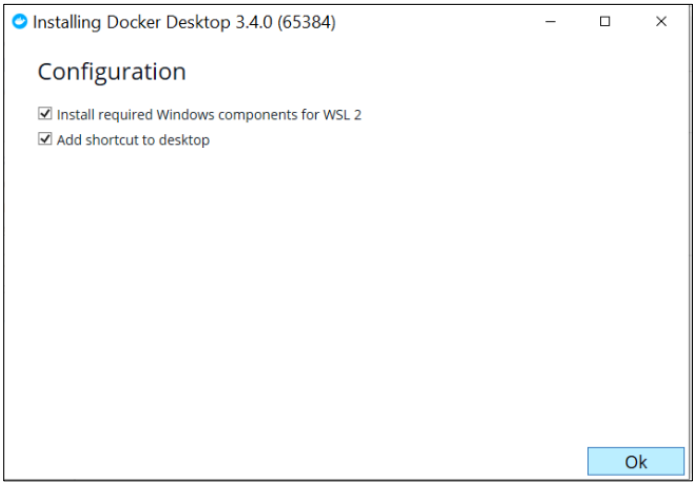
10.2.2 Install Docker Desktop

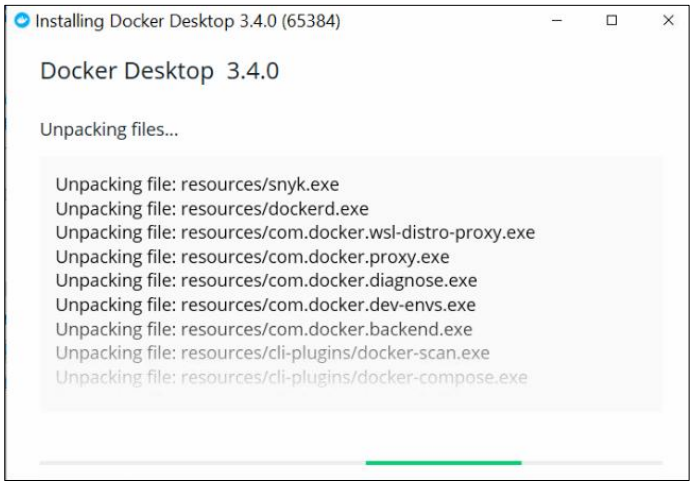
Steps

1. **Download and Install:**
   - Download the Docker Desktop installer v3.4.0 for Windows 11 and run it as an administrator.



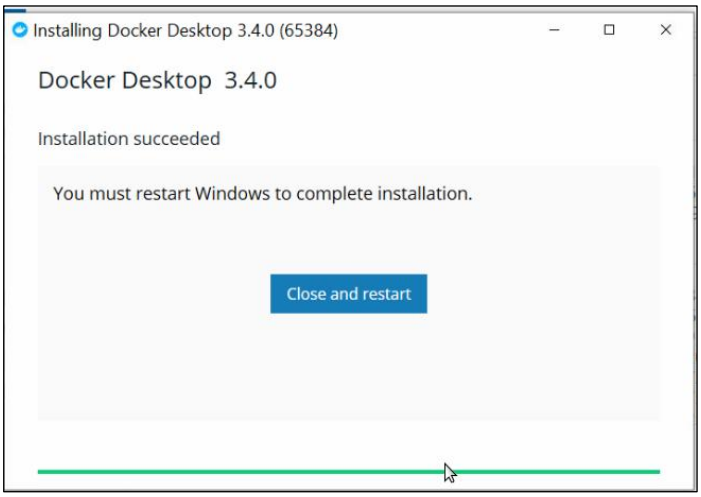






2. **Complete Installation:**

   - Click **Close and restart** if prompted to finalize the installation.



3. **Verify Startup:**
    - Launch Docker Desktop and ensure it starts successfully.

10.2.3 Run Ubuntu 22.04 Distribution in Docker Container

Steps

1. **Download and Extract Repository:**
    - Download scarthgap_yocto_5.0.zip from https://github.com/AlifSemiDev/alif_linux-apss-build-setup/archive/refs/heads/scarthgap_yocto_5.0.zip and extract it to the alif_linux-apss-build-setup directory.

Note:

Use your GitHub username and a personal access token as the password

(see Create a GitHub Personal Access Token)

---

2. **Open Command Prompt**:
    - Launch the Command Prompt on Windows 11.
4. **Navigate to Docker Folder:**
    - Change to the extracted directory:

```
cd alif_linux-apss-build-setup\windows-docker
```

5. **Build Docker Image:**

- Build the Ubuntu 22.04 image with dependent packages:

```
docker-container-build.bat
```

Note:

This process takes time as it fetches packages from the internet.

---

6. **Create and Run Container:**

- Start the container and mount the alif_linux-apss-build-setup directory to /opt:

```
docker-container-create.bat
```

7. **Initialize Git Repository:**

- Inside the container, switch to /opt and set up a Git branch:

```
bash
cd /opt/
git init
git checkout -b scarthgap_yocto_5.0
git add *
git commit -m "initial commit"
```

Note:

This step compensates for the missing .git directory in the ZIP file.

---

8. **Set Up Build Environment:**

- Configure the build environment:

```
bash
git checkout -b scarthgap_yocto_5.0
sudo scripts/host_setup.sh
source scripts/setup.sh
```

Note:

Default build directory is $HOME/build.

Use source scripts/setup.sh build_new for a custom directory (e.g., $HOME/build_new).

---

9. **Build Linux Images:**

- Generate APSS images:

```
bash
bitbake alif-tiny-image
```

10. **Copy Images:**

- Copy xipImage, bl32.bin, devkit-e8.dtb, and alif-tiny-image-devkit-e8.cramfs-xip from tmp/deploy/images/devkit-e8 to /opt for Windows access and APSS booting.

11. **Exit Container:**

- Press Ctrl+D or type logout to exit.

12. **Re-enter Container:**

- Restart the container:

```
docker-container-start.bat
```

13. **Reconfigure Environment:**

- Set up the environment again:

```
bash
source /opt/scripts/setup.sh
```

or

```
bash
source $HOME/build/setup.sh
```

14. **Remove Container:**

- Delete the container (removes all internal data):

```
docker rm ubuntu
```

15. **Remove Docker Image:**

- Delete the built Ubuntu 22.04 image:

```
docker rmi apss
```

11. Legal and Support Information

11.1 Disclaimers

Legal Notice – Please Read

Alif Semiconductor™ reserves the right, without notice, to alter, edit, update, make corrections, and improvements to Alif documentation and products at any time. It is the responsibility of customers to maintain the most current versions of documentation before making any purchases from Alif. The information found in this documentation is provided to purchasers solely for the purpose of enabling hardware and software implementation of Alif products.

Alif neither takes any responsibility for, nor guarantees the appropriateness of, its products for a specific purpose. Customers accept the responsibility for the selection and incorporation of Alif products into their systems and Alif has no liability with respect thereto. ALIF ALSO DISCLAIMS ANY AND ALL LIABILITY WITH RESPECT THERETO, INCLUDING WITHOUT LIMITATION, DIRECT, CONSEQUENTIAL, INDIRECT, SPECIAL AND INCIDENTAL DAMAGES. ADDITIONALLY, ALIF DISCLAIMS AND EXCLUDES ALL WARRANTIES, WHETHER STATUTORY, EXPRESS OR IMPLIED, INCLUDING ANY IMPLIED WARRANTY OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NON-INFRINGEMENT AND THOSE ARISING FROM COURSE OF DEALING AND USAGE OF TRADE.

Alif Semiconductor and the Alif logo are trademarks of Alif. For more information about our trademarks please visit our website at https://alifsemi.com/legal/. The omission of any Alif trademark, product name or any other name from this list does not constitute a waiver of Alif's intellectual property rights.

The recipient of this document does not have permission to copy, reprint, reproduce, duplicate, share, in any form, in whole or in part, unless prior written consent from Alif is obtained.

Please contact an Alif representative at contact@alifsemi.com if you have any questions regarding the information in this document.

Alif sells products according to standards terms and conditions of sales, which can be found at: https://alifsemi.com/legal/

11.2 Related Documents and Tools

- **Yocto Reference Manual and BitBake Reference Manual**
  - **Source:** Official Yocto Project documentation and Bitbake Reference Manual
- **Cramfs-tools**

- **Source:** CramFS utility package

▪ **Application_XIP_Instructions_For_OMAP**

- **Source:** Texas Instruments documentation

11.3 Contact Information

For more information visit our website Alif Semiconductor or contact us: contact@alifsemi.com

US HQ – Silicon Valley, CA

7901 Stoneridge Drive, Suite 300

Pleasanton, CA 94588

12. Document History

Version	Change Log
2.0.0	Eagle Devkit E8 Release
2.1.0	Eagle Devkit E8 Release